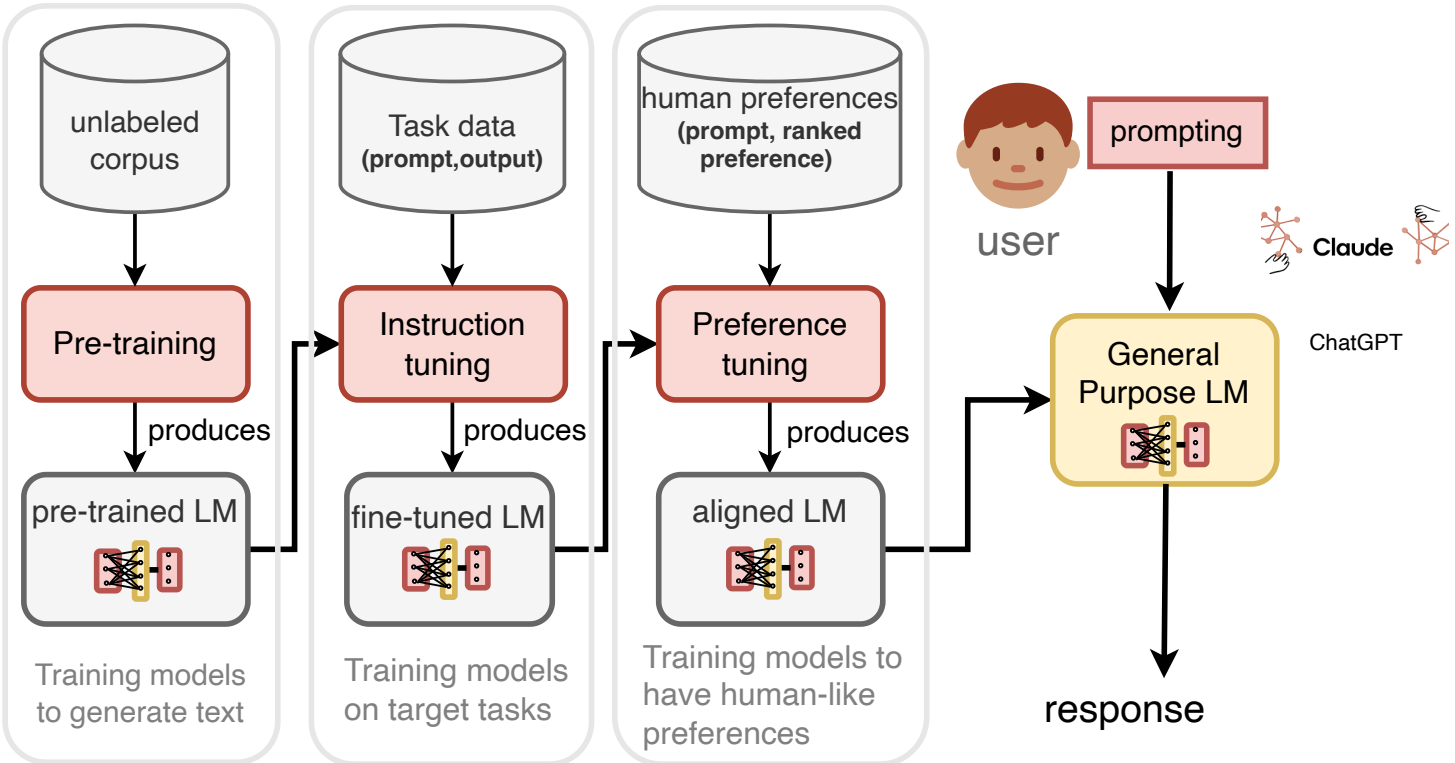
Discrete Methods for Modern AI: lecture 3

Kyle Richardson

Nov 16 2026

Reminder of what the course is about

How do we get to general-purpose LLMs? **recipe**



- This is the **modern AI** part of the course: **large language models** (LLMs), how they are built, trained and used.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

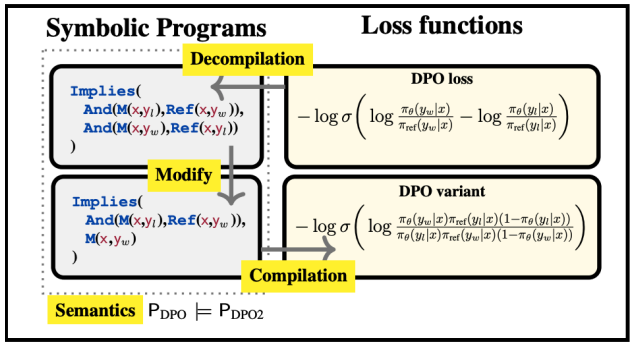
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

Last lecture: the basics of LLMs (Vaswani et al., 2017)

Language Model Programming




```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

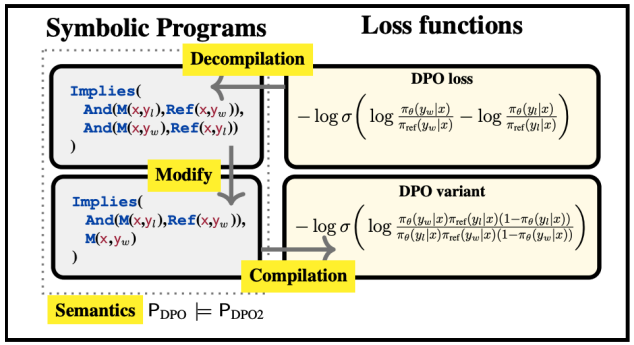
    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **LM programming**: The study and design of languages for specifying, composing and reasoning about LLM computations.

Discrete Modeling Techniques




```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

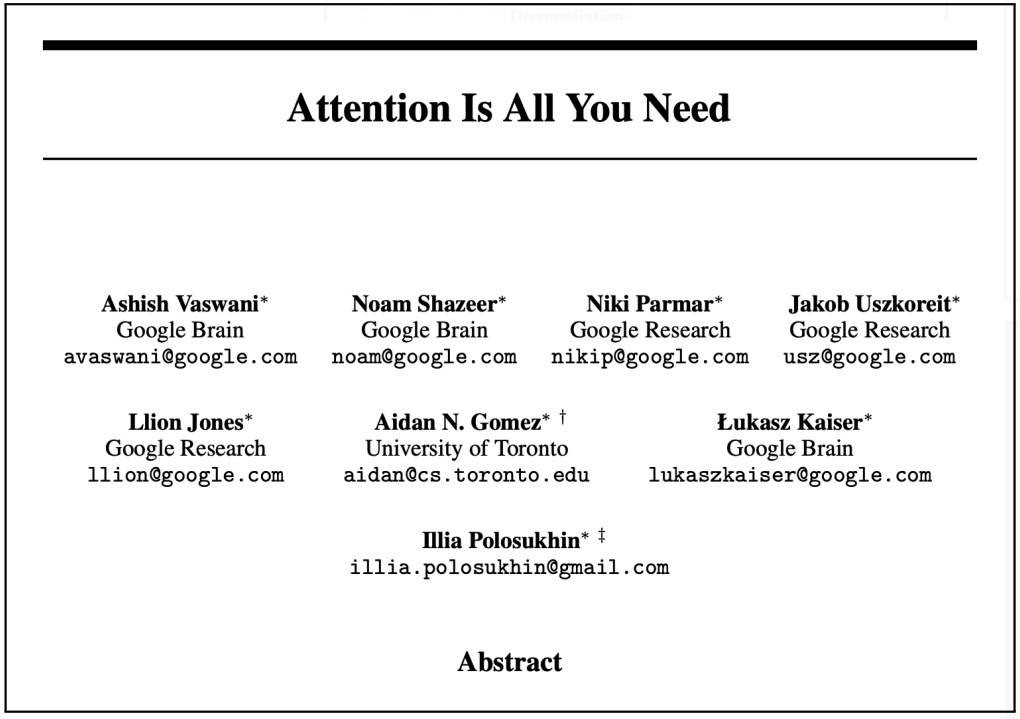
    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **What this course is about**: The formal techniques needed for this kind of modeling, using and implementing symbolic programming languages.



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

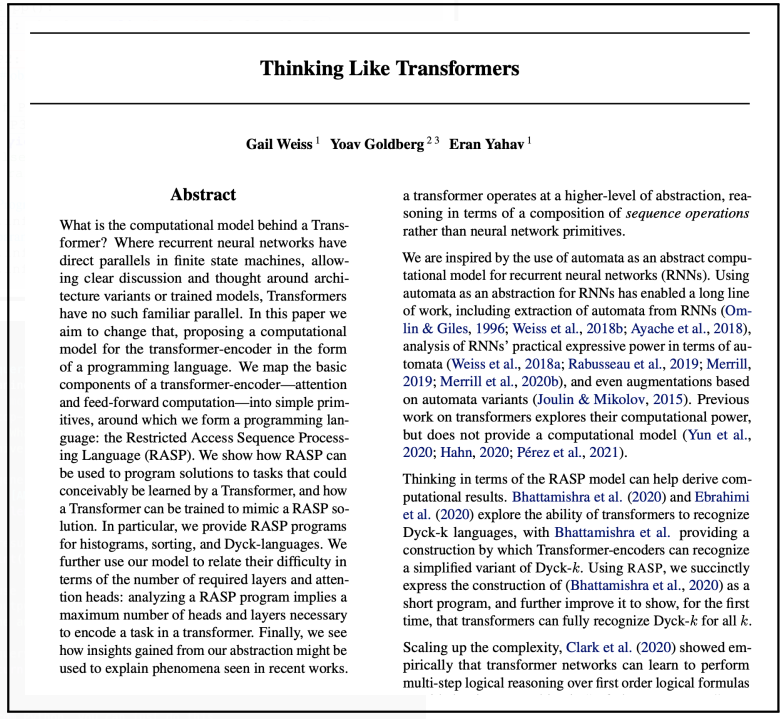
Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract



Thinking Like Transformers

Gail Weiss [1] **Yoav Goldberg** [2 3] **Eran Yahav** [1]

Abstract

What is the computational model behind a Transformer? Where recurrent neural networks have direct parallels in finite state machines, allowing clear discussion and thought around architecture variants or trained models, Transformers have no such familiar parallel. In this paper we aim to change that, proposing a computational model for the transformer-encoder in the form of a programming language. We map the basic components of a transformer-encoder—attention and feed-forward computation—into simple primitives, around which we form a programming language: the Restricted Access Sequence Processing Language (RASP). We show how RASP can be used to program solutions to tasks that could conceivably be learned by a Transformer, and how a Transformer can be trained to mimic a RASP solution. In particular, we provide RASP programs for histograms, sorting, and Dyck-languages. We further use our model to relate their difficulty in terms of the number of required layers and attention heads: analyzing a RASP program implies a maximum number of heads and layers necessary to encode a task in a transformer. Finally, we see how insights gained from our abstraction might be used to explain phenomena seen in recent works.

a transformer operates at a higher-level of abstraction, reasoning in terms of a composition of *sequence operations* rather than neural network primitives.

We are inspired by the use of automata as an abstract computational model for recurrent neural networks (RNNs). Using automata as an abstraction for RNNs has enabled a long line of work, including extraction of automata from RNNs (Omlin & Giles, 1996; Weiss et al., 2018b; Ayache et al., 2018), analysis of RNNs' practical expressive power in terms of automata (Weiss et al., 2018a; Rabusseau et al., 2019; Merrill, 2019; Merrill et al., 2020b), and even augmentations based on automata variants (Joulin & Mikolov, 2015). Previous work on transformers explores their computational power, but does not provide a computational model (Yun et al., 2020; Hahn, 2020; Pérez et al., 2021).

Thinking in terms of the RASP model can help derive computational results. Bhattamishra et al. (2020) and Ebrahimi et al. (2020) explore the ability of transformers to recognize Dyck-k languages, with Bhattamishra et al. providing a construction by which Transformer-encoders can recognize a simplified variant of Dyck-k. Using RASP, we succinctly express the construction of (Bhattamishra et al., 2020) as a short program, and further improve it to show, for the first time, that transformers can fully recognize Dyck-k for all k.

Scaling up the complexity, Clark et al. (2020) showed empirically that transformer networks can learn to perform multi-step logical reasoning over first order logical formulas

A first taste of a programming language for LLMs

(Vaswani et al., 2017; Weiss et al., 2021)

Discrete Modeling Techniques

Discrete Methods for Modern AI: Foundations

Kyle Richardson
University of Tübingen
Draft: work in progress (November 1, 2026)

1 Introduction

These notes are a companion to a course on *Discrete Methods for Modern AI* [1], which examines applications of discrete modeling techniques – specifically, techniques drawn from logic, automated reasoning and classical probabilistic inference – to problems in modern language modeling and generative AI. Throughout the course, we study how these techniques can support deeper semantic analysis of current systems, provide a means of incorporating structured knowledge and constraints into learning and inference, and offer theoretical tools for characterizing the capabilities and limitations of these systems. In these notes, we focus on the logical foundations shared by each of these directions and introduce the basic propositional calculus. In particular, our review aims to be *algorithmic* in nature by focusing on the technical tools needed for *building* the kinds of automated reasoning systems used throughout this area of the machine learning literature (see review in Marra et al. (2021); Feldstein et al. (2024)).

The structure of this note is as follows: starting from a review of basic concepts in propositional logic (§ 2), we then state some fundamental algorithmic questions involved with *using* propositional logic (§ 3)[2], including the *Boolean Satisfiability* (SAT) problem and some variants of this problem, which are at the heart of many problems in automated reasoning and probabilistic inference. To help build intuitions for how build automated reasoning systems, we derive a classical search-based algorithm for SAT, called the DPLL Algorithm (Davis et al., 1962), and relate its generalized algebraic form to solutions to these different variants of SAT (e.g., *model counting* and its weighted variants). Building on these intuitions and algorithmic details, we next (§ 5) give an overview of representations of logic based on tractable classes of Boolean and arithmetic circuits that are at the core of many systems that combine logic and learning.

While working in this literature often requires considerable technical sophistication and knowledge of logic, researchers often rely on a broad set of tools from the automated reasoning community (e.g., SAT solving and other theorem proving technology). We therefore provide code examples using such tools and encourage readers to work through these examples. We also use examples from a library built by the author, `symprog` [3], which aims to make it easy to quickly try out the ideas covered in these notes. For advanced readers who want to explore specific topics in more detail, we also provide pointers to related work and various *technical asides* that address topics not directly covered in our main survey.

- What this [illegible] the [illegible] needed for this kind of modeling, using and implementing symbolic programming languages.

We will go deeper into this

Recap of last lecture: LMs, transformers, Rasp

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Recap of last lecture: LMs, transformers, Rasp

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Recap of last lecture: LMs, transformers, Rasp

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Recap of last lecture: LMs, transformers, Rasp

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Rasp: why it matters

- ✓ program depth $\rightarrow$ layers
- ✓ existence, not learnability
- ✓ variants: provable claims

Recap of last lecture: LMs, transformers, Rasp

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: why it matters

- ✓ program depth $\rightarrow$ layers
- ✓ existence, not learnability
- ✓ variants: provable claims

Today: constrained fine-tuning, encoders, semantic loss, WMC.

Quick check on lecture 2: true or false?

Recall: a **bigram** model uses only the previous word: $p(w_j \mid w_{j-1})$

1. bigram: $p(\textit{sat} \mid \textit{the cat}) \neq p(\textit{sat} \mid \textit{a cat})$
2. bigram, 10,000 words: about 10^8 parameters
3. contextual: *bank* has one vector in all sentences
4. contextual LMs model the full joint $p(w_1, \ldots, w_n)$

Quick check on lecture 2: true or false?

Recall: a **bigram** model uses only the previous word: $p(w_j \mid w_{j-1})$

1. bigram: $p(\textit{sat} \mid \textit{the cat}) \neq p(\textit{sat} \mid \textit{a cat})$ **false** only *cat*
2. bigram, 10,000 words: about 10^8 parameters
3. contextual: *bank* has one vector in all sentences
4. contextual LMs model the full joint $p(w_1, \ldots, w_n)$

Quick check on lecture 2: true or false?

Recall: a **bigram** model uses only the previous word: $p(w_j \mid w_{j-1})$

1. bigram: $p(\textit{sat} \mid \textit{the cat}) \neq p(\textit{sat} \mid \textit{a cat})$ — **false** — only *cat*
2. bigram, 10,000 words: about 10^8 parameters — **true** — one per pair
3. contextual: *bank* has one vector in all sentences
4. contextual LMs model the full joint $p(w_1, \ldots, w_n)$

Quick check on lecture 2: true or false?

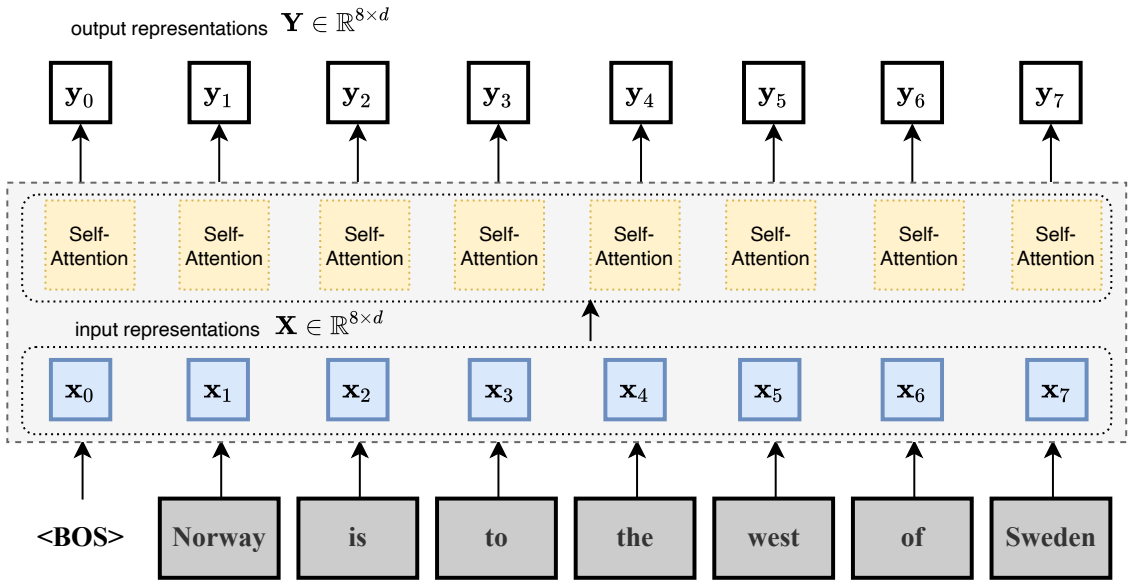
Recall: a **bigram** model uses only the previous word: $p(w_j \mid w_{j-1})$

1.	bigram: $p(\textit{sat} \mid \textit{the cat}) \neq p(\textit{sat} \mid \textit{a cat})$	**false**	only *cat*
2.	bigram, 10,000 words: about 10^8 parameters	**true**	one per pair
3.	contextual: *bank* has one vector in all sentences	**false**	it varies
4.	contextual LMs model the full joint $p(w_1, \ldots, w_n)$		

Quick check on lecture 2: true or false?

Recall: a **bigram** model uses only the previous word: $p(w_j \mid w_{j-1})$

1.	bigram: $p(\textit{sat} \mid \textit{the cat}) \neq p(\textit{sat} \mid \textit{a cat})$	**false**	only *cat*
2.	bigram, 10,000 words: about 10^8 parameters	**true**	one per pair
3.	contextual: *bank* has one vector in all sentences	**false**	it varies
4.	contextual LMs model the full joint $p(w_1, \ldots, w_n)$	**true**	all of $w_{<j}$



Remember: each input $\mathbf{x}_i$ becomes a contextual $\mathbf{y}_i$

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5. the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$
6. after the softmax, the $\alpha_{i,j}$ for word i sum to 1
7. $\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight
8. attention over n words computes n^2 scores
9. full attention (encoder): word 2 sees word 5
10. causal attention (decoder): word 2 sees word 5

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5. the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$ **true** one number
6. after the softmax, the $\alpha_{i,j}$ for word i sum to 1
7. $\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight
8. attention over n words computes n^2 scores
9. full attention (encoder): word 2 sees word 5
10. causal attention (decoder): word 2 sees word 5

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5. the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$ **true** one number
6. after the softmax, the $\alpha_{i,j}$ for word i sum to 1 **true** softmax
7. $\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight
8. attention over n words computes n^2 scores
9. full attention (encoder): word 2 sees word 5
10. causal attention (decoder): word 2 sees word 5

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5.	the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$	**true**	one number
6.	after the softmax, the $\alpha_{i,j}$ for word i sum to 1	**true**	softmax
7.	$\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight	**false**	$\sum_j \alpha_{i,j}\mathbf{x}_j$
8.	attention over n words computes n^2 scores		
9.	full attention (encoder): word 2 sees word 5		
10.	causal attention (decoder): word 2 sees word 5		

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5.	the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$	**true**	one number
6.	after the softmax, the $\alpha_{i,j}$ for word i sum to 1	**true**	softmax
7.	$\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight	**false**	$\sum_j \alpha_{i,j}\mathbf{x}_j$
8.	attention over n words computes n^2 scores	**true**	every pair
9.	full attention (encoder): word 2 sees word 5		
10.	causal attention (decoder): word 2 sees word 5		

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5.	the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$	**true**	one number
6.	after the softmax, the $\alpha_{i,j}$ for word i sum to 1	**true**	softmax
7.	$\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight	**false**	$\sum_j \alpha_{i,j}\mathbf{x}_j$
8.	attention over n words computes n^2 scores	**true**	every pair
9.	full attention (encoder): word 2 sees word 5	**true**	sees all
10.	causal attention (decoder): word 2 sees word 5		

Quick check on lecture 2: true or false? (attention)

Recall: raw score $\alpha'_{i,j}$, weight $\alpha_{i,j}$, output $\mathbf{y}_i$ for word i.

5.	the raw score is a dot product: $\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j$	**true**	one number
6.	after the softmax, the $\alpha_{i,j}$ for word i sum to 1	**true**	softmax
7.	$\mathbf{y}_i$ is the $\mathbf{x}_j$ with the highest weight	**false**	$\sum_j \alpha_{i,j}\mathbf{x}_j$
8.	attention over n words computes n^2 scores	**true**	every pair
9.	full attention (encoder): word 2 sees word 5	**true**	sees all
10.	causal attention (decoder): word 2 sees word 5	**false**	only ≤ 2

Remember: reversing a string in Rasp

```
1  tokens = [1, 2, 3, 0]    ## 0 is the end symbol
2  pos = indices(tokens)
3  EOS = 0
4
5  length = aggr(
6   select(tokens,tokens,lambda q,k : k == EOS),
7   pos
8  )                          ## => [3, 3, 3, 3]
9  targets = length - pos     ## => [3, 2, 1, 0]
10
11 output = aggr(
12  select(pos,targets,lambda k,v: k == v),
13  tokens
14 )                          ## => [0, 3, 2, 1]
```

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer
12. `select` decides which positions attend to which
13. `aggr` averages the values at selected positions
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`
15. element-wise operations correspond to attention
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer **true** by design
12. `select` decides which positions attend to which
13. `aggr` averages the values at selected positions
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`
15. element-wise operations correspond to attention
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer **true** by design
12. `select` decides which positions attend to which **true** the pattern
13. `aggr` averages the values at selected positions
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`
15. element-wise operations correspond to attention
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer **true** by design
12. `select` decides which positions attend to which **true** the pattern
13. `aggr` averages the values at selected positions **true** a mean
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`
15. element-wise operations correspond to attention
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer — **true** — by design
12. `select` decides which positions attend to which — **true** — the pattern
13. `aggr` averages the values at selected positions — **true** — a mean
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]` — **true** — prefix means
15. element-wise operations correspond to attention
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11. Rasp's operations mirror parts of a transformer **true** by design
12. `select` decides which positions attend to which **true** the pattern
13. `aggr` averages the values at selected positions **true** a mean
14. i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]` **true** prefix means
15. element-wise operations correspond to attention **false** to the MLP
16. our program for *reverse* uses one layer
17. depth-2 program: a 2-layer transformer can do it
18. so training a 2-layer transformer will find it

Quick check on lecture 2: true or false? (Rasp)

11.	Rasp's operations mirror parts of a transformer	**true**	by design
12.	`select` decides which positions attend to which	**true**	the pattern
13.	`aggr` averages the values at selected positions	**true**	a mean
14.	i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`	**true**	prefix means
15.	element-wise operations correspond to attention	**false**	to the MLP
16.	our program for *reverse* uses one layer	**false**	two
17.	depth-2 program: a 2-layer transformer can do it		
18.	so training a 2-layer transformer will find it		

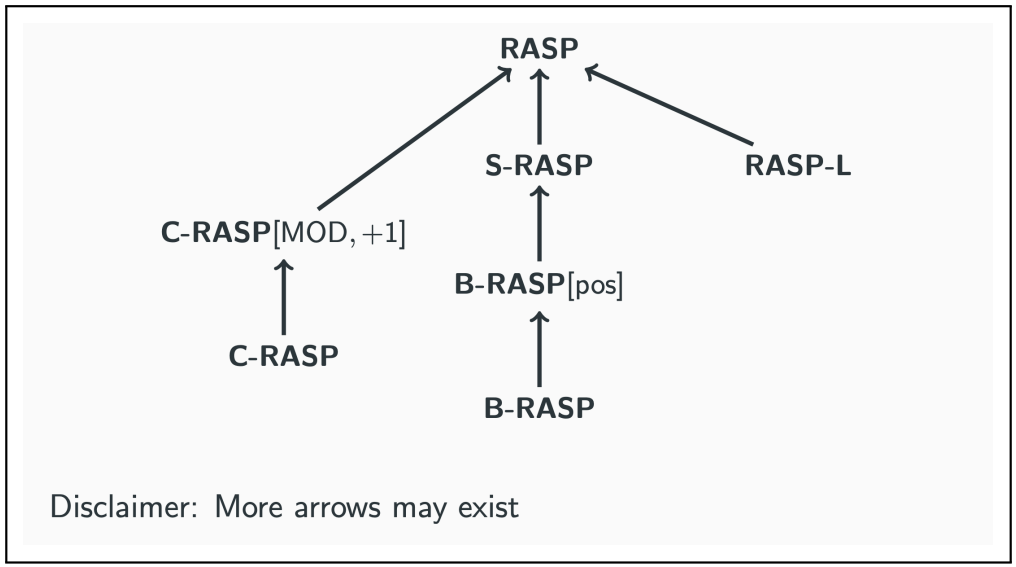
Quick check on lecture 2: true or false? (Rasp)

11.	Rasp's operations mirror parts of a transformer	**true**	by design
12.	`select` decides which positions attend to which	**true**	the pattern
13.	`aggr` averages the values at selected positions	**true**	a mean
14.	i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`	**true**	prefix means
15.	element-wise operations correspond to attention	**false**	to the MLP
16.	our program for *reverse* uses one layer	**false**	two
17.	depth-2 program: a 2-layer transformer can do it	**true**	existence
18.	so training a 2-layer transformer will find it		

Quick check on lecture 2: true or false? (Rasp)

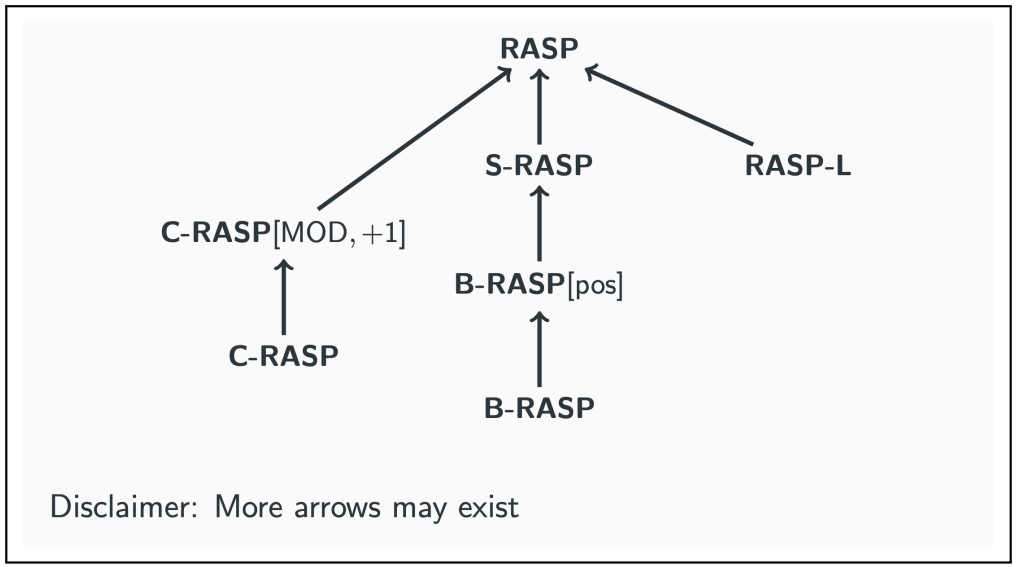
11.	Rasp's operations mirror parts of a transformer	**true**	by design
12.	`select` decides which positions attend to which	**true**	the pattern
13.	`aggr` averages the values at selected positions	**true**	a mean
14.	i selects all $j \leq i$; `aggr` on `[2,4,6]`: `[2,3,4]`	**true**	prefix means
15.	element-wise operations correspond to attention	**false**	to the MLP
16.	our program for *reverse* uses one layer	**false**	two
17.	depth-2 program: a 2-layer transformer can do it	**true**	existence
18.	so training a 2-layer transformer will find it	**false**	not shown

Different variants of Rasp



Taken from Andy Yang https://pentagonalize.github.io/files/CLAY_slides.pdf

Different variants of Rasp



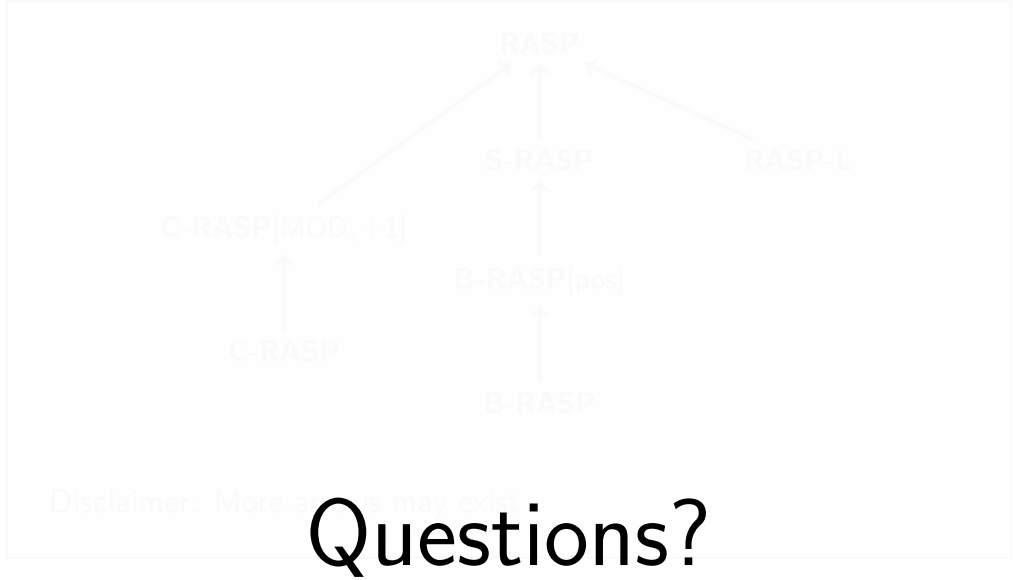
Taken from Andy Yang https://pentagonalize.github.io/files/CLAY_slides.pdf

Why variants? Each matches a precise kind of transformer, so claims become **theorems**: what transformers can and cannot express, with provable bounds.

e.g., B-RASP = masked hard-attention transformers = star-free languages (Yang et al., 2024)

e.g., C-RASP adds counting; compiles to softmax transformers (Yang and Chiang, 2024)

Different variants of Rasp



Taken from Andy Yang https://pentagonalize.github.io/files/CLAY_slides.pdf

Why variants? Each matches a precise kind of transformer, so claims become **theorems**: what transformers can and cannot express, with provable bounds.

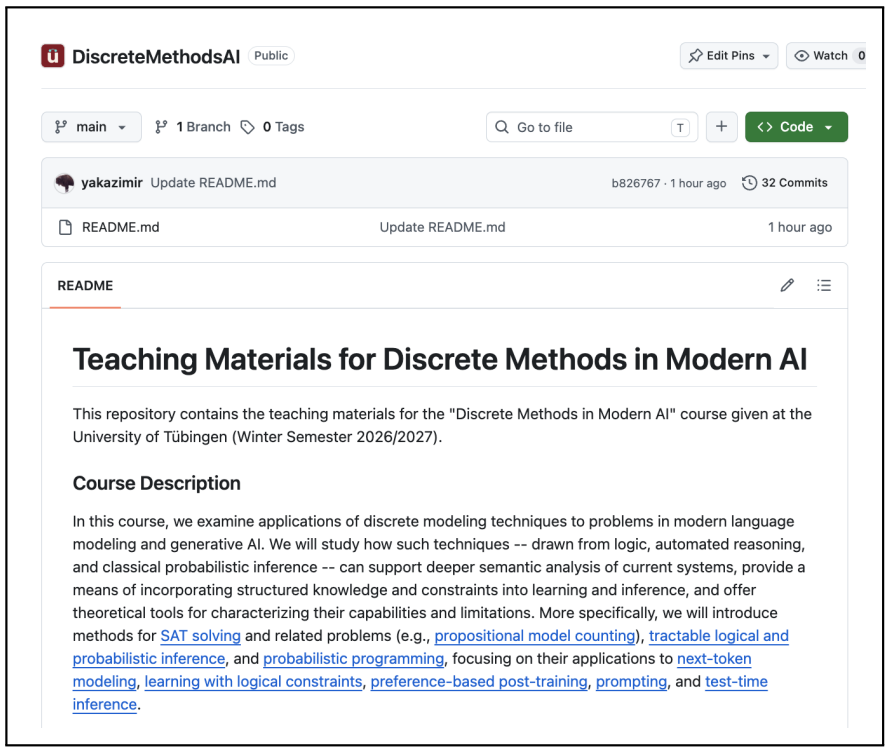
e.g., B-RASP = masked hard-attention transformers = star-free languages (Yang et al., 2024)

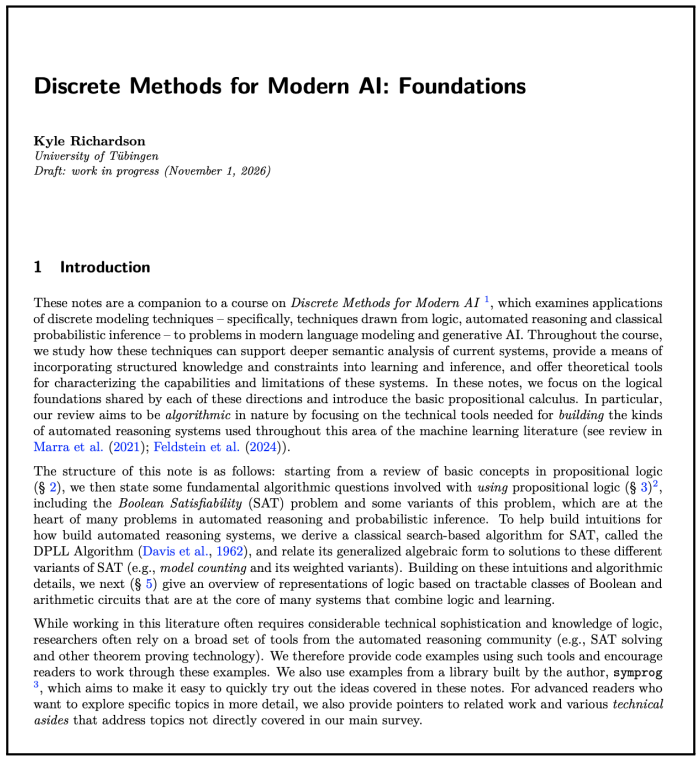
e.g., C-RASP adds counting; compiles to softmax transformers (Yang and Chiang, 2024)

Questions?

Course materials and website

https://github.com/TuLLI-AI/DiscreteMethodsAI

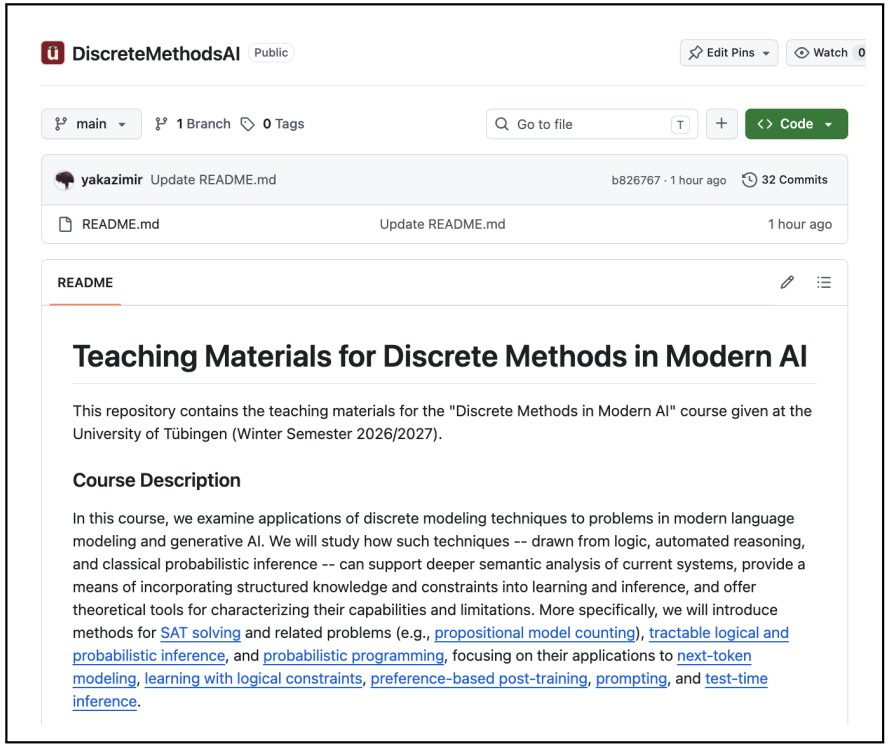


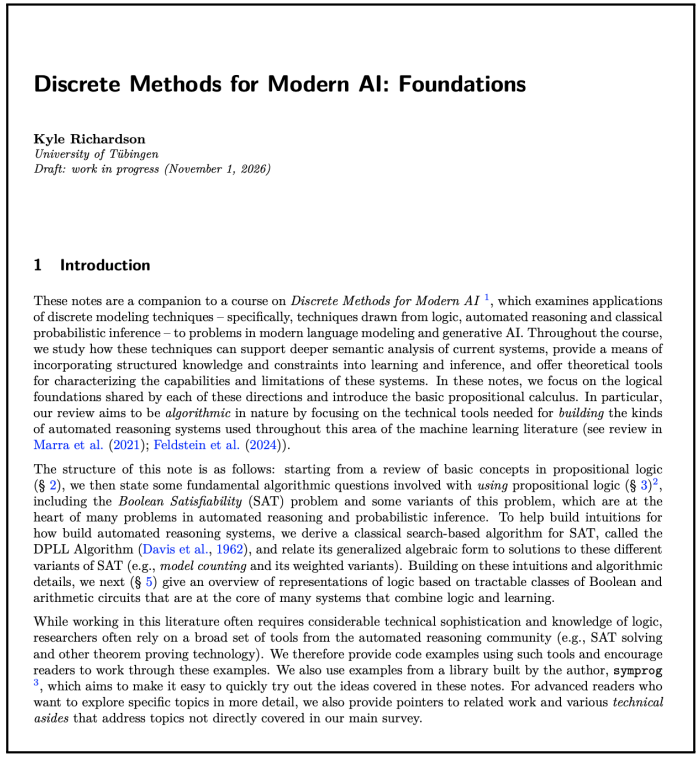


- **Expectations**: Attend weekly lectures, follow the readings and Python notebooks (on GitHub), quickly learn new topics.

Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI





- **Evaluation**: In-class comprehension exam (30%, Dec. 7), coding project + short presentation (60%), attendance (10%), **6 credit** course.

Did everyone manage to install `symprog`?

```
$ pip install symprog          github.com/TuLLI-AI/symprog
```

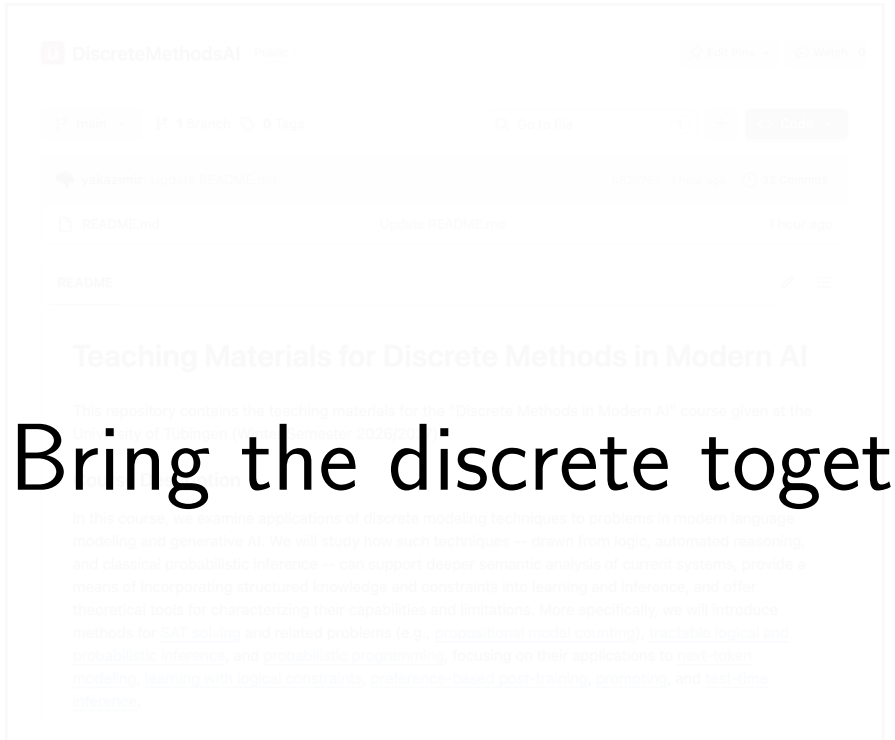
```python
from symprog import Solver

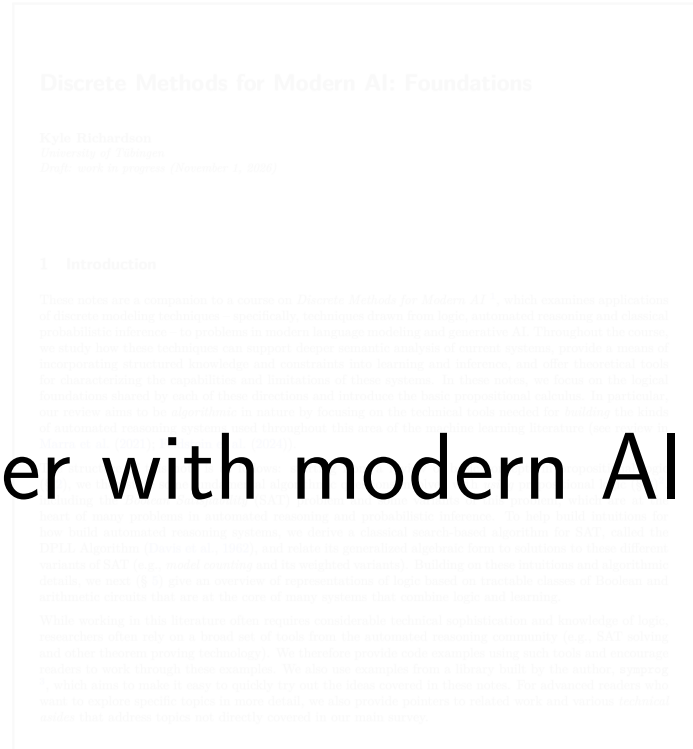
solver = Solver()
solver.add(["Or(Not(p1), p2)", "p3"])          ## F
solver.add_weights({"p1": 0.8, "p2": 0.75, "p3": 0.4})

solver.query(mode="SAT").inference        ## => yes
solver.query(mode="Count").inference      ## => 3.0
solver.query(mode="Prob").inference       ## => 0.32  WMC
solver.query(mode="ViterbiAssignment").variables   ## MPE
## => {'p1': True, 'p2': True, 'p3': True}
```

Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI

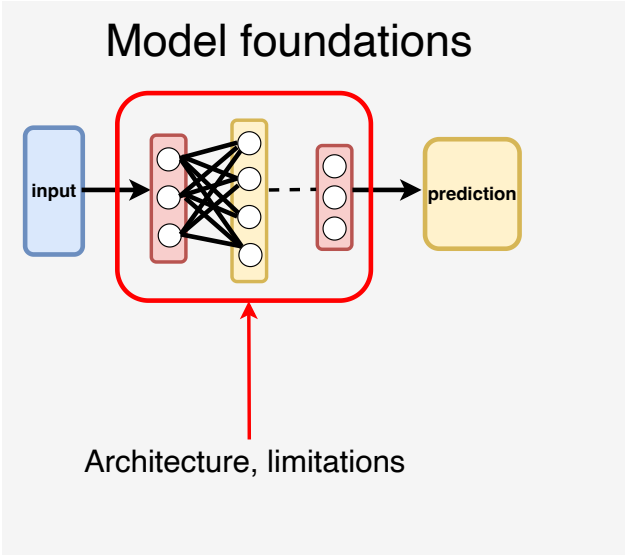


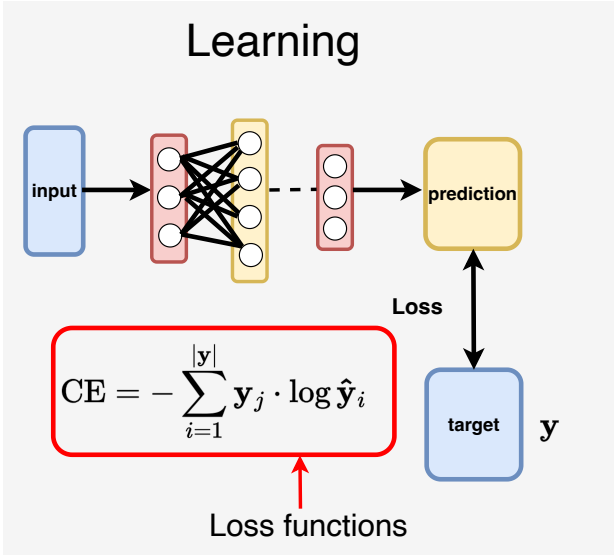


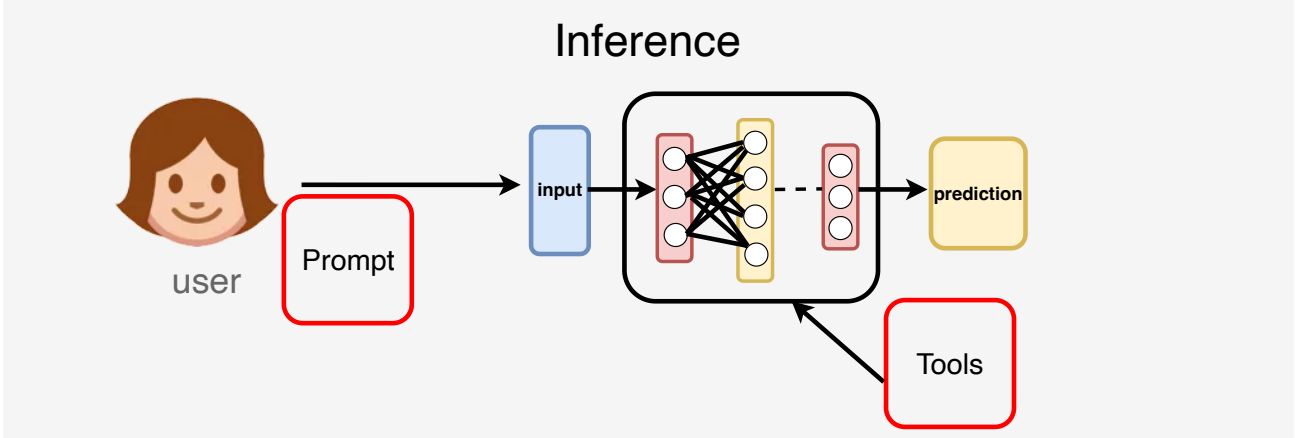
Bring the discrete together with modern AI

- **Evaluation**: In-class comprehension exam (30%, Dec. 7), coding project + short presentation (60%), attendance (10%), **6 credit** course.

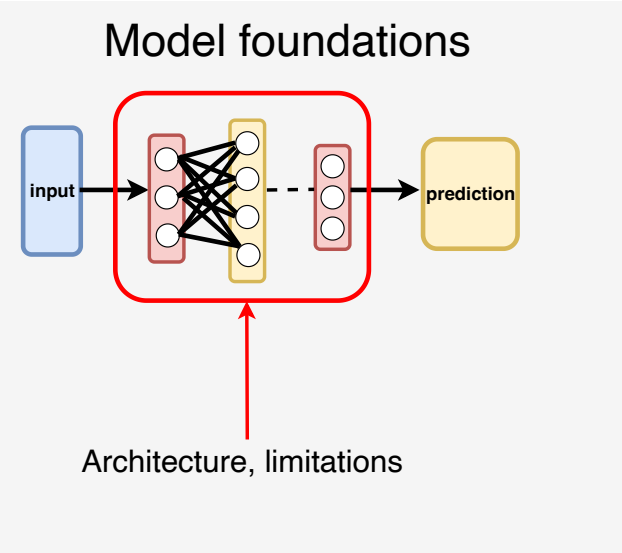
Language model programming





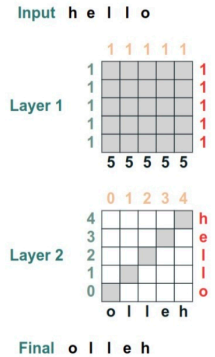


Language model programming

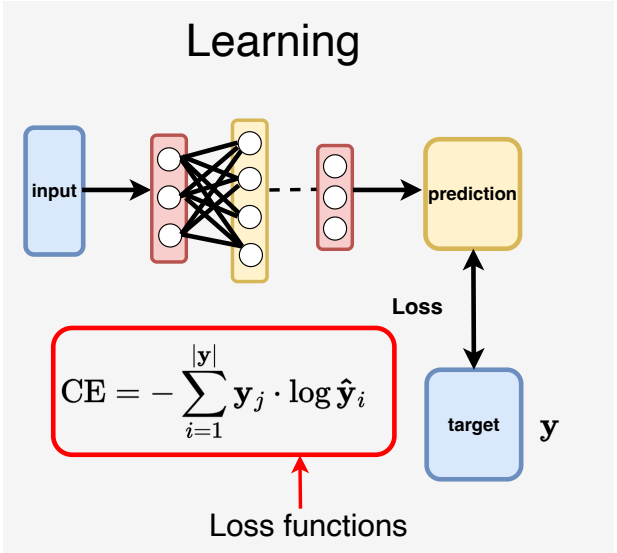


RASPy

```
def flip():
    length = (key(1) == query(1)).value(1)
    flip = (key(length - indices - 1) == query(indices)).value(tokens)
    return flip
flip()
```



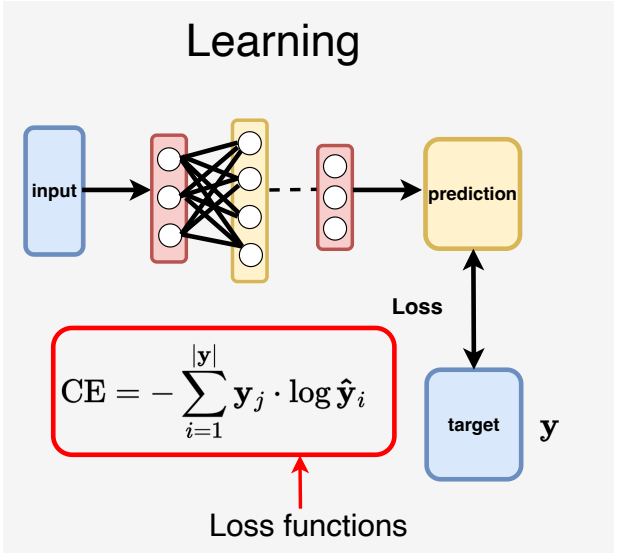
Goals for lecture today




```
1  // File path_planner.scl
2  type actor(x: i32, y: i32), goal(x: i32, y: i32), enemy(x: i32, y: i32)
3
4  const UP = 0, DOWN = 1, RIGHT = 2, LEFT = 3
5  rel safe_cell(x, y) = range(0, 5, x), range(0, 5, y), not enemy(x, y)
6  rel edge(x, y, x, yp, UP) = safe_cell(x, y), safe_cell(x, yp), yp == y + 1
7  // Rules for DOWN, RIGHT, and LEFT edges are omitted...
8
9  rel next_pos(p, q, a) = actor(x, y), edge(x, y, p, q, a)
10 rel path(x, y, x, y) = next_pos(x, y, _)
11 rel path(x1, y1, x3, y3) = path(x1, y1, x2, y2), edge(x2, y2, x3, y3, _)
12 rel next_action(a) = next_pos(p, q, a), path(p, q, r, s), goal(r, s)
```

Fig. 3. The logic program of the PacMan-Maze application in Scallop.

Goals for lecture today



Declarative Programming

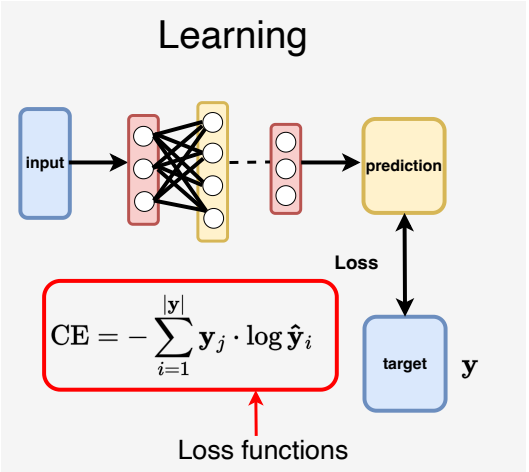
```
1  // File path_planner.scl
2  type actor(x: i32, y: i32), goal(x: i32, y: i32), enemy(x: i32, y: i32)
3
4  const UP = 0, DOWN = 1, RIGHT = 2, LEFT = 3
5  rel safe_cell(x, y) = range(0, 5, x), range(0, 5, y), not enemy(x, y)
6  rel edge(x, y, x, yp, UP) = safe_cell(x, y), safe_cell(x, yp), yp == y + 1
7  // Rules for DOWN, RIGHT, and LEFT edges are omitted...
8
9  rel next_pos(p, q, a) = actor(x, y), edge(x, y, p, q, a)
10 rel path(x, y, x, y) = next_pos(x, y, _)
11 rel path(x1, y1, x3, y3) = path(x1, y1, x2, y2), edge(x2, y2, x3, y3, _)
12 rel next_action(a) = next_pos(p, q, a), path(p, q, r, s), goal(r, s)
```

Fig. 3. The logic program of the PacMan-Maze application in Scallop.

Goals for lecture today

```
1  // File path_planner.scl
2  type actor(x: i32, y: i32), goal(x: i32, y: i32), enemy(x: i32, y: i32)
3
4  const UP = 0, DOWN = 1, RIGHT = 2, LEFT = 3
5  rel safe_cell(x, y) = range(0, 5, x), range(0, 5, y), not enemy(x, y)
6  rel edge(x, y, x, yp, UP) = safe_cell(x, y), safe_cell(x, yp), yp == y + 1
7  // Rules for DOWN, RIGHT, and LEFT edges are omitted...
8
9  rel next_pos(p, q, a) = actor(x, y), edge(x, y, p, q, a)
10 rel path(x, y, x, y) = next_pos(x, y, _)
11 rel path(x1, y1, x3, y3) = path(x1, y1, x2, y2), edge(x2, y2, x3, y3, _)
12 rel next_action(a) = next_pos(p, q, a), path(p, q, r, s), goal(r, s)
```

Fig. 3. The logic program of the PacMan-Maze application in Scallop.



- **Declarative programming**: programs as constraints on predictions.
- **Fine-tuning encoders**: our running example, natural language inference.
- **Semantics of declarative knowledge**: probabilities of formulas via WMC.
- **Semantic loss**: turning a formula into a loss function for fine-tuning.

Basic properties of declarative programming

```
from symprog import Parser   ## pip install symprog

@Parser(language="plp")
def program():
    stress[ann]: 0.8
    stress[bob]: 0.4
    influences[ann, bob]: 0.6
    influences[bob, carl]: 0.2

    smokes[X] :- stress[X]
    smokes[X] :- influences[Y, X] & smokes[Y]

    query(smokes[carl])

program.inference().inference   ## => 0.1376
```

Basic properties of declarative programming

```
1  from symprog import Parser    ## pip install symprog
2
3  @Parser(language="plp")
4  def program():
5      stress[ann]: 0.8
6      stress[bob]: 0.4
7      influences[ann, bob]: 0.6
8      influences[bob, carl]: 0.2
9
10     smokes[X] :- stress[X]
11     smokes[X] :- influences[Y, X] & smokes[Y]
12
13     query(smokes[carl])
14
15 program.inference().inference    ## => 0.1376
```

1. We say what we want to do without saying how to do it.

Basic properties of declarative programming

```
1  from symprog import Parser    ## pip install symprog
2
3  @Parser(language="plp")
4  def program():
5      stress[ann]: 0.8
6      stress[bob]: 0.4
7      influences[ann, bob]: 0.6
8      influences[bob, carl]: 0.2
9
10     smokes[X] :- stress[X]
11     smokes[X] :- influences[Y, X] & smokes[Y]
12
13     query(smokes[carl])
14
15 program.inference().inference    ## => 0.1376
```

1. We say what we want to do without saying how to do it.
2. What we say can be made to be formally precise.

Basic properties of declarative programming

```
from symprog import Parser   ## pip install symprog

@Parser(language="plp")
def program():
    stress[ann]: 0.8
    stress[bob]: 0.4
    influences[ann, bob]: 0.6
    influences[bob, carl]: 0.2

    smokes[X] :- stress[X]
    smokes[X] :- influences[Y, X] & smokes[Y]

    query(smokes[carl])

program.inference().inference   ## => 0.1376
```

- **Logic programming**: Framework for programming based on (a subset of) logic, `Prolog`, we will revisit this in later lectures.

The same program in ordinary logic (symprog)

```
from symprog import Parser   ## pip install symprog

@Parser(language="constraint")
def program():           ## a, b, c = ann, bob, carl
    stress_a: 0.8; stress_b: 0.4   ## facts
    infl_ab: 0.6;  infl_bc: 0.2

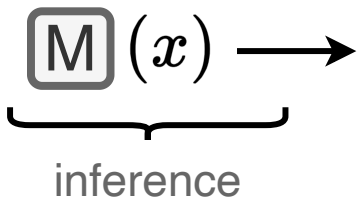
    ## grounded rules:  smokes_a <-> stress_a
    ##   smokes_b <-> stress_b or (infl_ab and smokes_a)
    ##   smokes_c <-> infl_bc and smokes_b
    ## unfold them into one formula for smokes_c:
    infl_bc and (stress_b or (infl_ab and stress_a))

program.inference().inference   ## => 0.1376
```

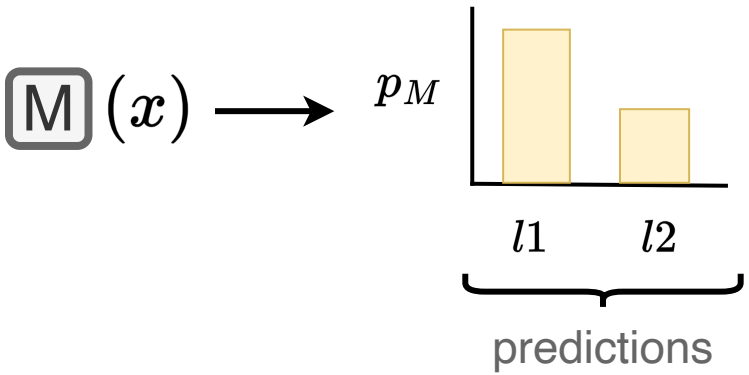
Ultimately such programs are compiled down to (weighted) propositional logic, inference reducible to model counting.

Thinking declaratively about model behavior

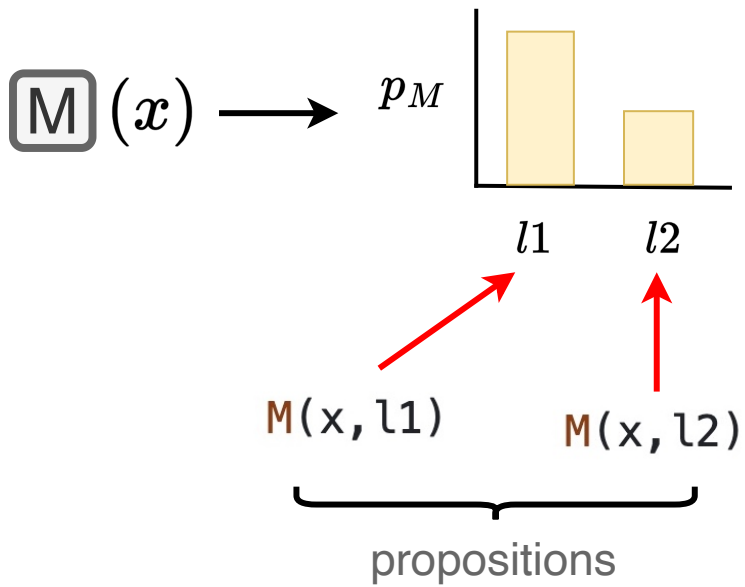
Declarative Modeling: Predictions as Propositions



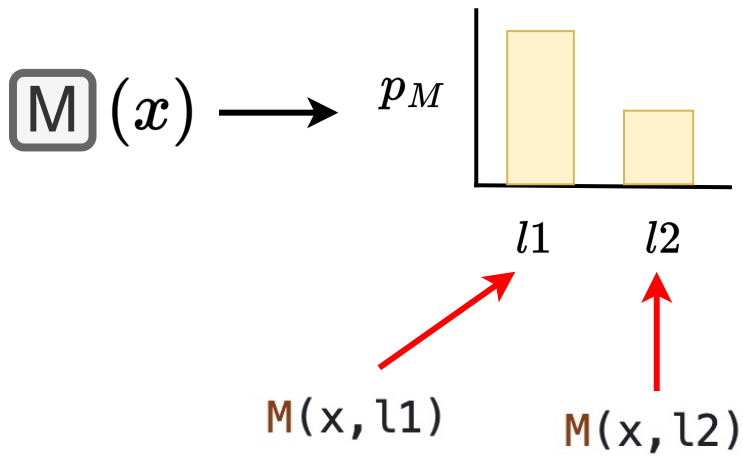
Declarative Modeling: Predictions as Propositions



Declarative Modeling: Predictions as Propositions

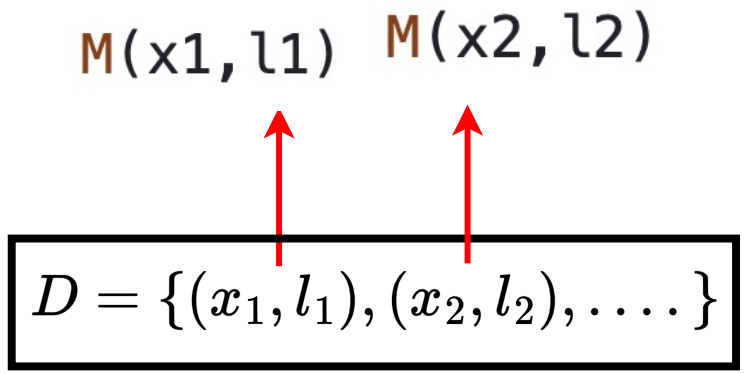


Declarative Modeling: Predictions as Propositions

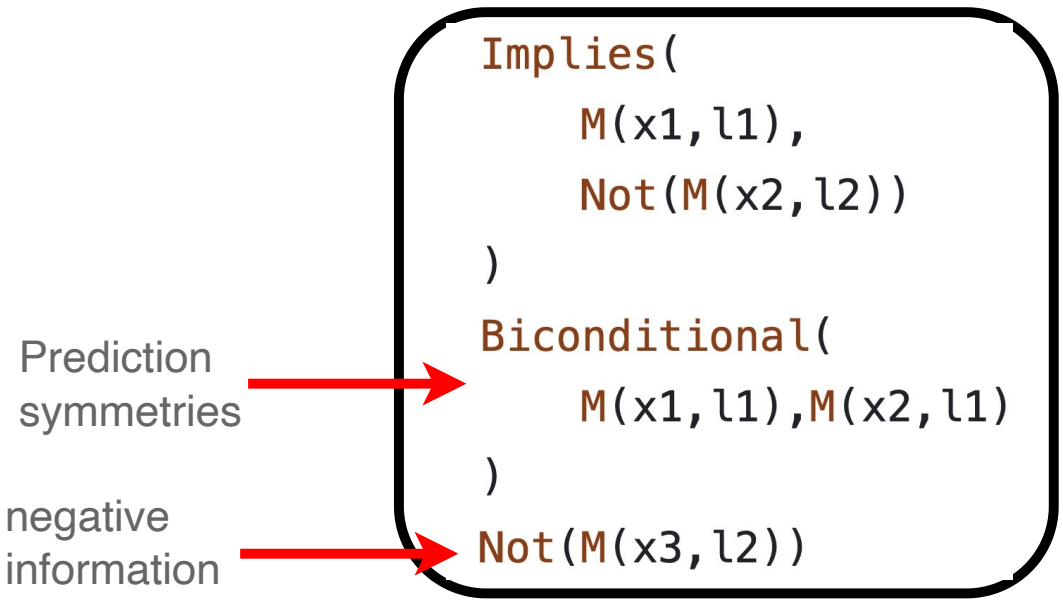


$$\underbrace{\theta(\ \texttt{M(x,l)}\ )}_{\text{proposition weight}} = p_M(l \mid x)$$

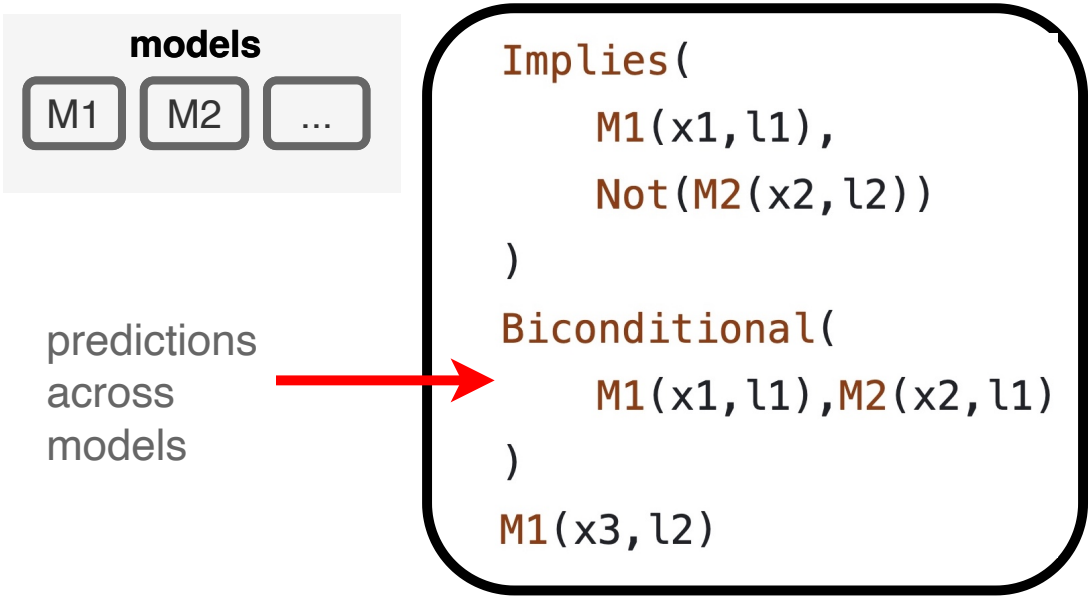
Declarative Modeling: Predictions as Propositions



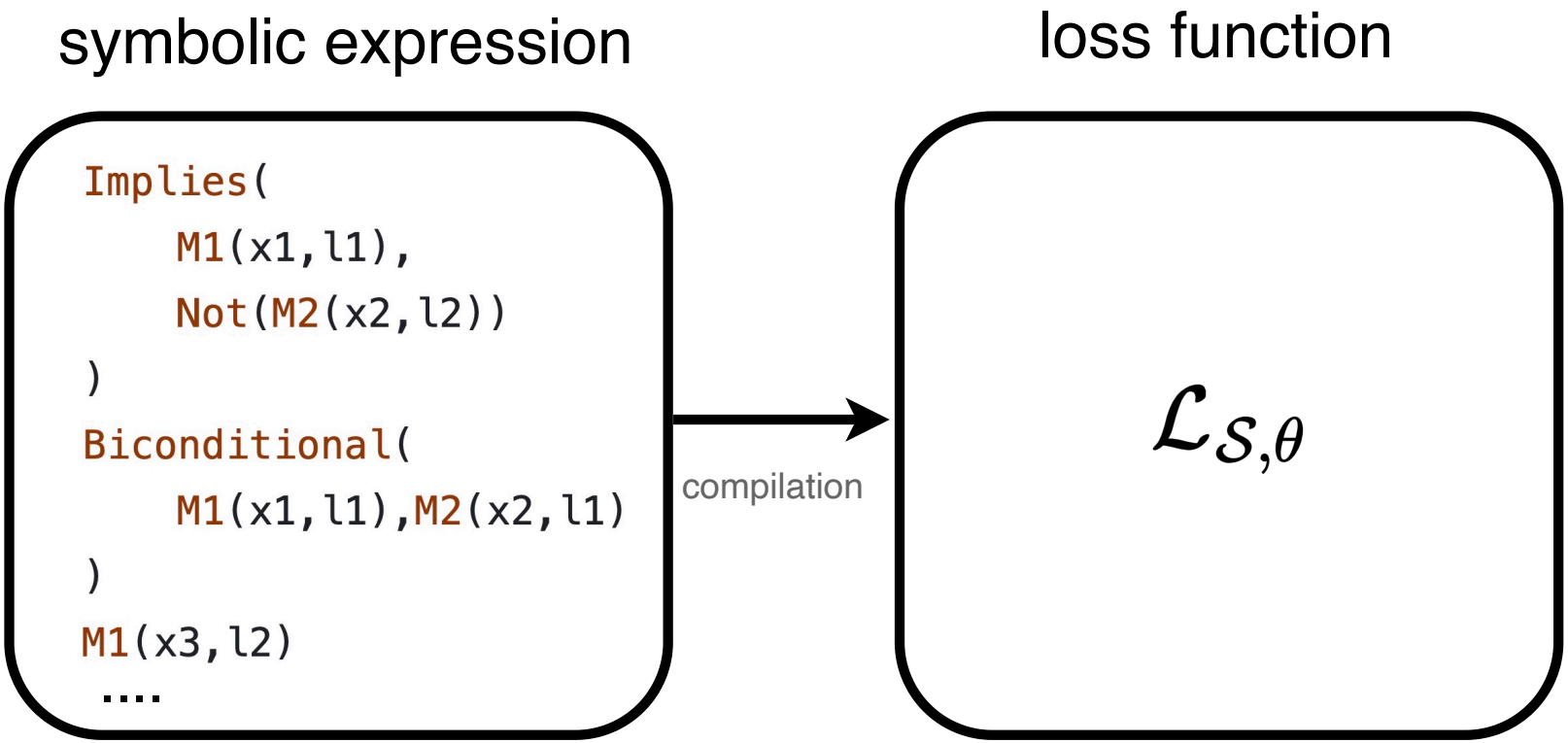
Declarative Modeling: Programs as constraints on predictions



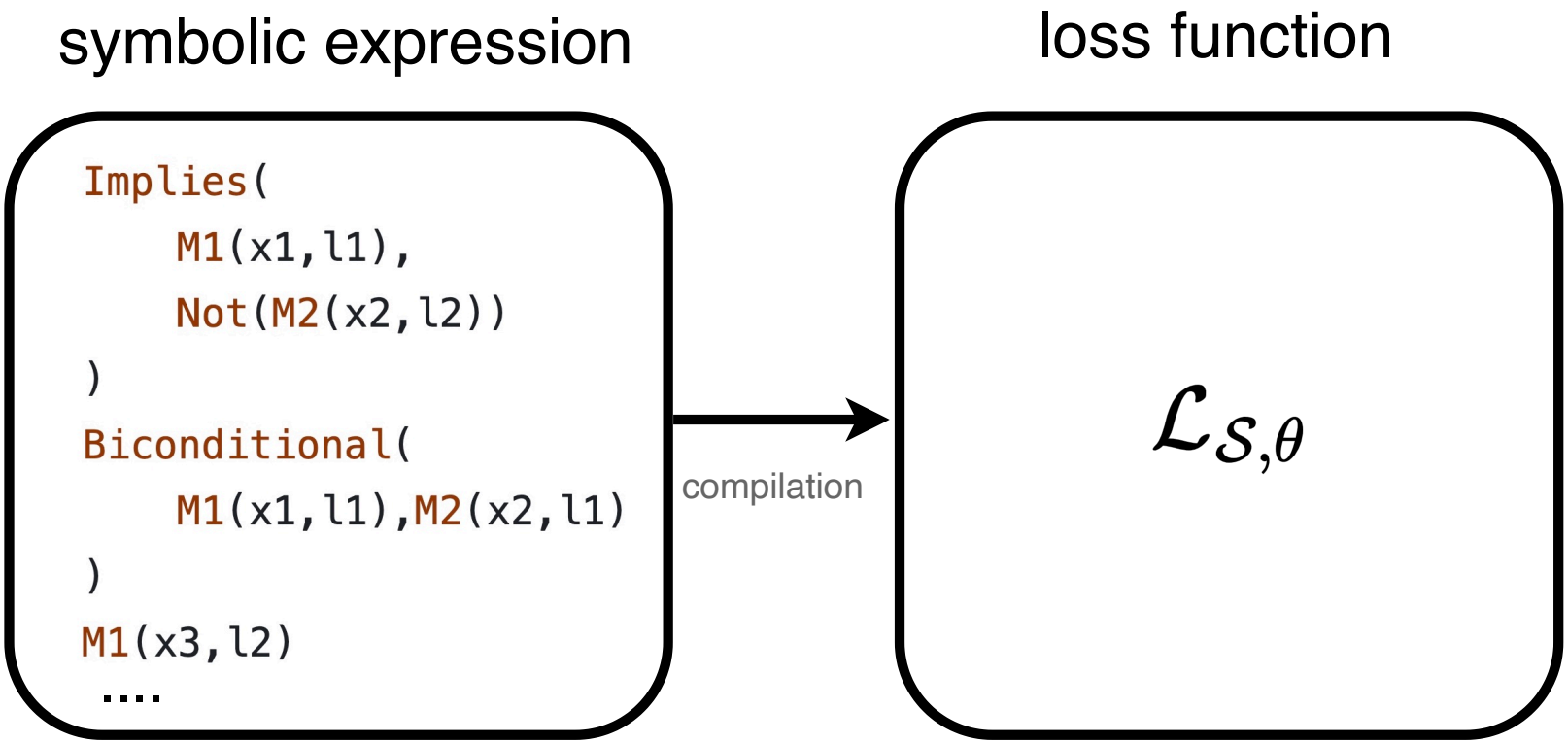
Declarative Modeling: Programs as constraints on predictions



Declarative Model Programming



Declarative Model Programming



- Say what we want models to do (without saying how), compilation into a learning or (inference) algorithm.

A motivating example

Classifying inferential relations between texts

USER	Imagining the situation: "A man was on his bike" would it be reasonable to conclude (answer yes/no) that "A person moved"?
ASSISTANT	Yes

- **Natural Language Inference** (**NLI**): Given a **premise** *(first)*, is it reasonable to conclude that the **hypothesis** *(second)* is likely to be true?

Classifying inferential relations between texts

USER	Imagining the situation: "A man was on his bike" would it be reasonable to conclude (answer yes/no) that "A person moved"?
ASSISTANT	Yes
USER	Why do you think this?
ASSISTANT	I reached this conclusion because the statement implies that the man was actively using his bike, suggesting movement. Riding a bike typically involves pedaling and propelling oneself forward, which would involve movement on the part of the person.

Datasets for NLI

- **Datasets**: SNLI (570k pairs), MultiNLI (433k pairs, many genres).
- **Labels**: `entailment` (**E**), `contradiction` (**C**) and `neutral` (**N**).

Example data in Hugging Face `datasets`

```
from datasets import load_dataset   ## pip install datasets

## Stanford Natural Language Inference dataset
snli_dataset = load_dataset("snli")
## Multi-genre NLI dataset
mnli_dataset = load_dataset("multi_nli")

## a training instance
snli_dataset["train"][3]
## => {"premise": "Children smiling and waving at camera",
##     "hypothesis": "They are smiling at their parents",
##     "label": 1}          ## 1 = neutral
```

An NLI model in PyTorch and Hugging Face Transformers

```
1  import torch     ## pip install torch transformers
2  from transformers import \
3       AutoModelForSequenceClassification as hf_model
4
5  class NLIModel(torch.nn.Module):
6      def __init__(self, model_name):
7          super().__init__()
8          self.model = hf_model.from_pretrained(
9              model_name,     ## pre-trained encoder
10             num_labels=3)   ## + a new head: 3 labels for NLI
11
12     def forward(self, features):      ## forward pass
13         output = self.model(**features)
14         prob = output.logits.softmax(dim=-1)
15         return (output.loss, prob)
```

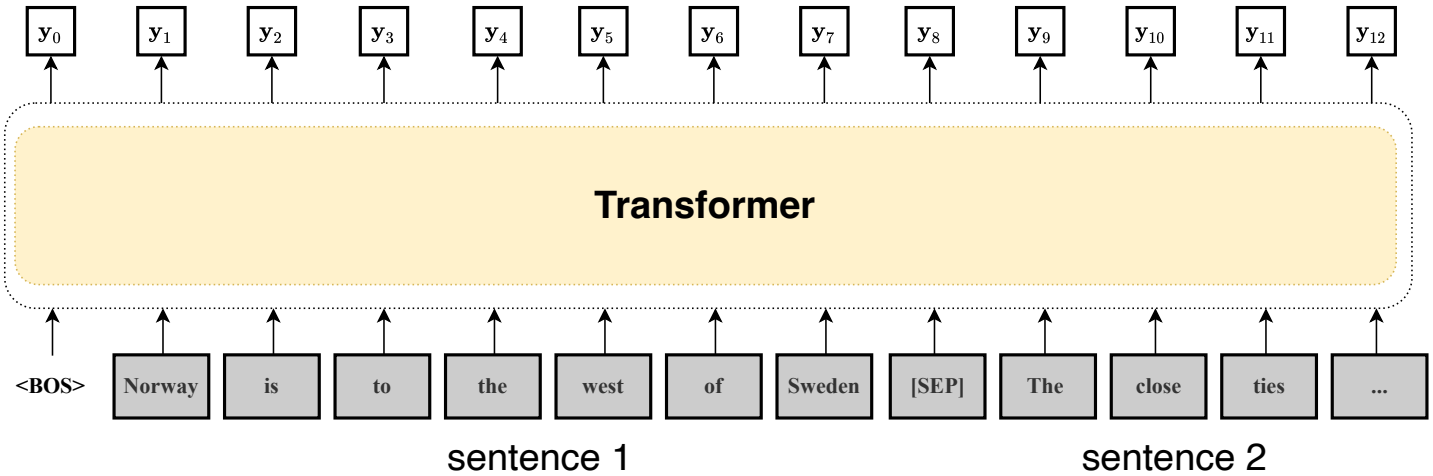
- A pre-trained **encoder** + a new *classification head* (lines 8–10).

An NLI model in PyTorch and Hugging Face Transformers

```
1  from transformers import AutoTokenizer as hf_tok
2
3  nli_model = NLIModel("roberta-base")    ## pre-trained encoder
4  tokenizer = hf_tok.from_pretrained("roberta-base")
5
6  ## prepare data and labels
7  p = "Children smiling and waving at camera"
8  h = "They are smiling at their parents"
9  batch = tokenizer([(p, h)], return_tensors="pt")
10 labels = torch.tensor([1])                ## 1 = neutral
11
12 ## run the forward pass
13 loss, prob = nli_model({
14     "input_ids": batch.input_ids,
15     "labels"   : labels,
16 })
17 prob    ## => tensor([[0.33, 0.30, 0.36]]), the head is untrained
```

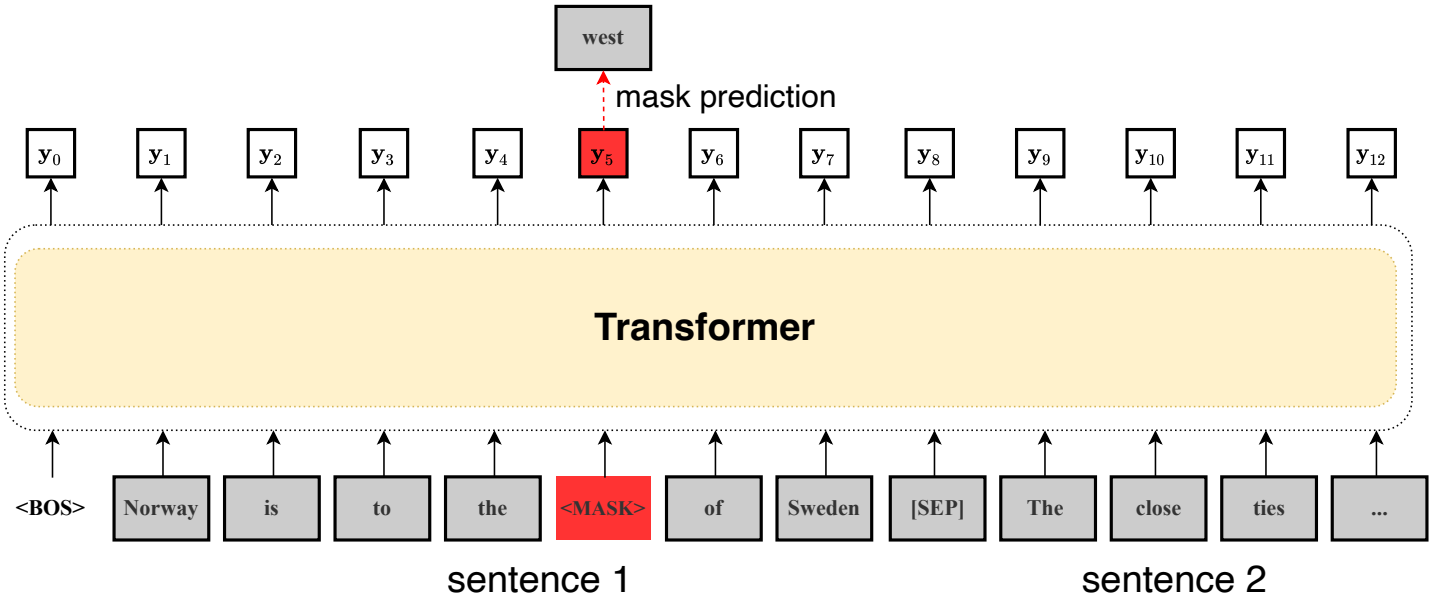
- Premise + hypothesis = **one input**: <s> *p* </s></s> *h* </s> (line 9).

Pre-training encoders: **masked words**



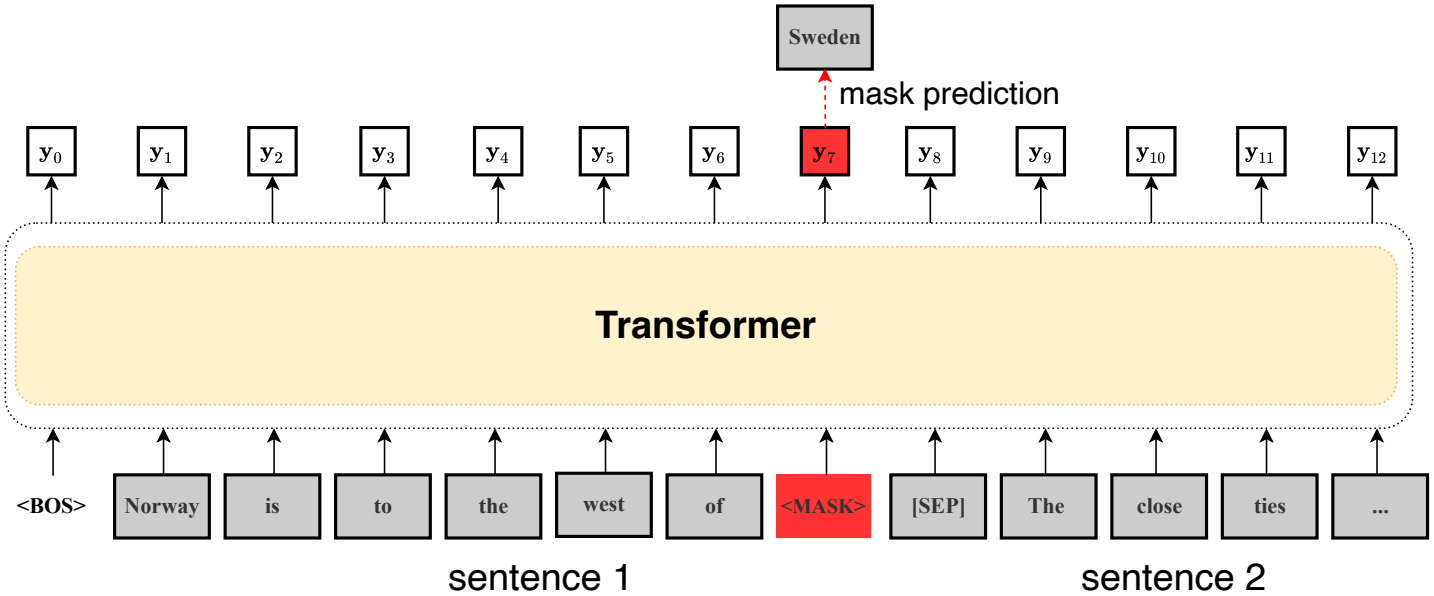
- **Recall** (lecture 2): encoders see every word (full attention).

Pre-training encoders: **masked words**



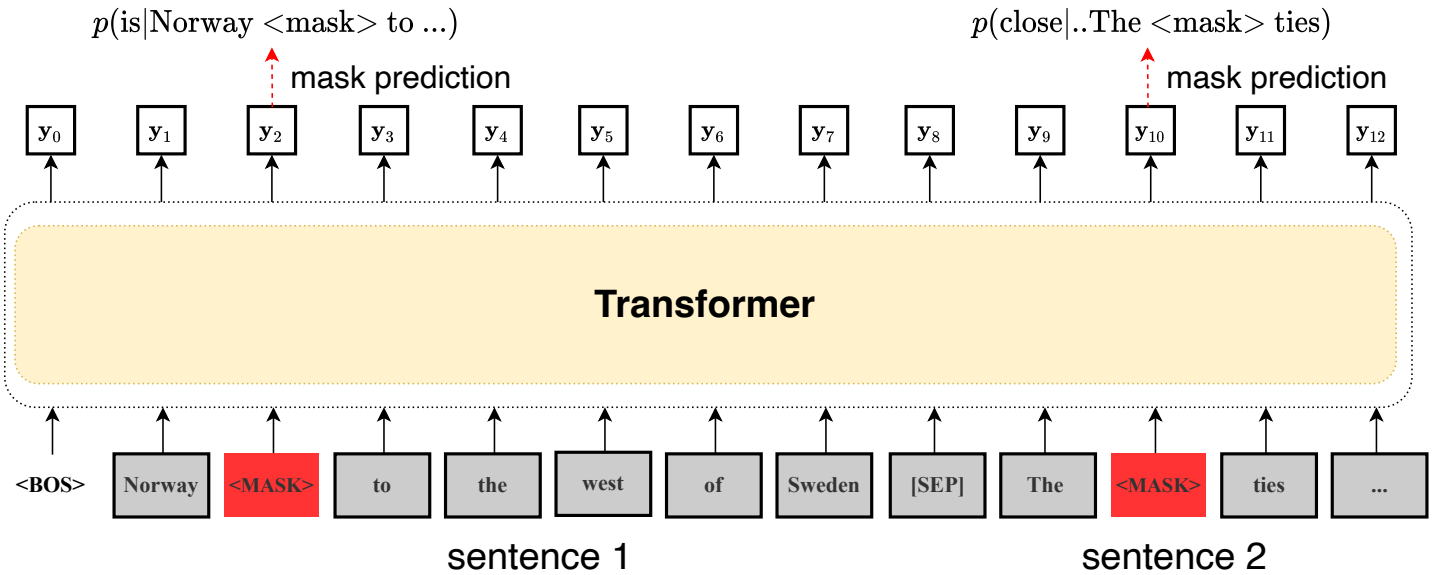
- **Recall** (lecture 2): encoders see every word (full attention).
- **Masked language objective**: hide words, predict them.

Pre-training encoders: **masked words**

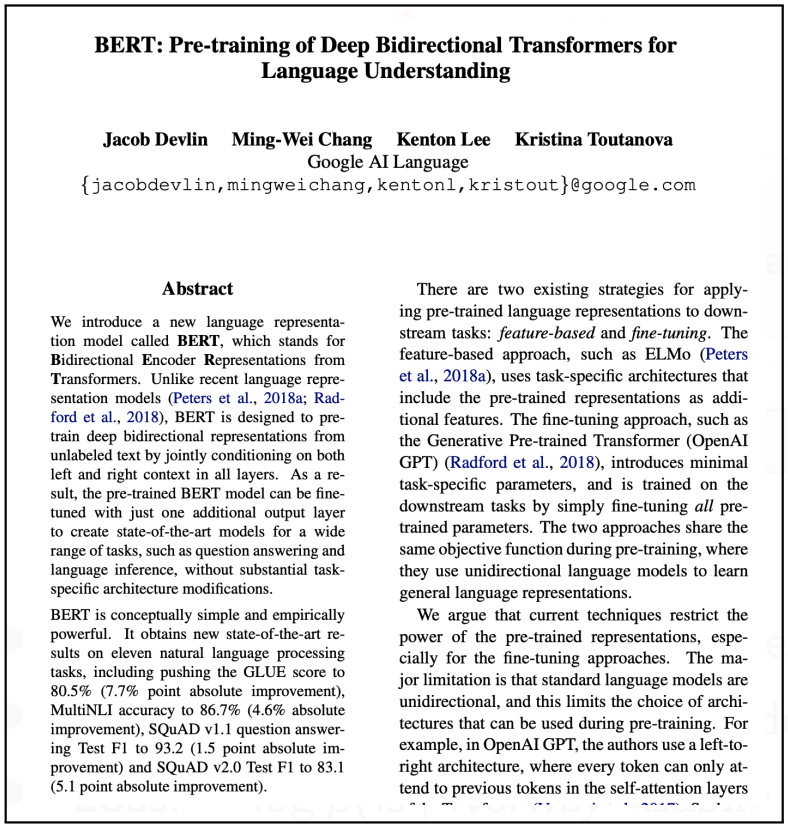


- **Recall** (lecture 2): encoders see every word (full attention).
- **Masked language objective**: hide words, predict them.

Pre-training encoders: **masked words**



- **Recall** (lecture 2): encoders see every word (full attention).
- **Masked language objective**: hide words, predict them.
- **Loss**: $-\log p(\textit{is} \mid \textit{Norway}\ \texttt{<mask>}\ \textit{to}\ldots)$



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

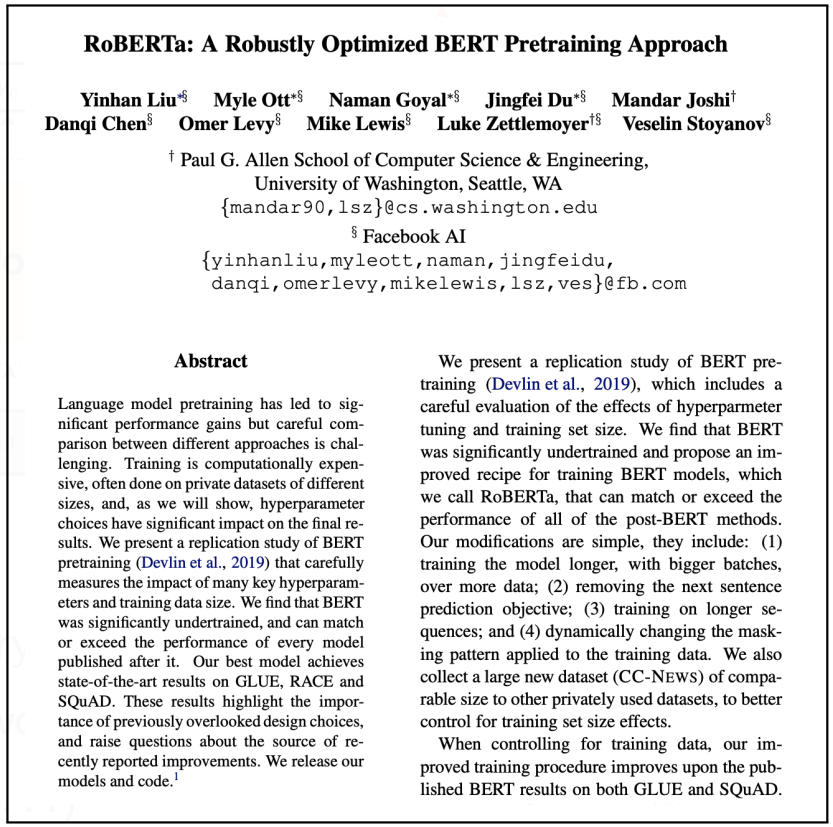
Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers



RoBERTa: A Robustly Optimized BERT Pretraining Approach

Yinhan Liu[*§] **Myle Ott**[*§] **Naman Goyal**[*§] **Jingfei Du**[*§] **Mandar Joshi**[†]
Danqi Chen[§] **Omer Levy**[§] **Mike Lewis**[§] **Luke Zettlemoyer**[†§] **Veselin Stoyanov**[§]

[†] Paul G. Allen School of Computer Science & Engineering,
University of Washington, Seattle, WA
{mandar90,lsz}@cs.washington.edu

[§] Facebook AI
{yinhanliu,myleott,naman,jingfeidu,
danqi,omerlevy,mikelewis,lsz,ves}@fb.com

Abstract

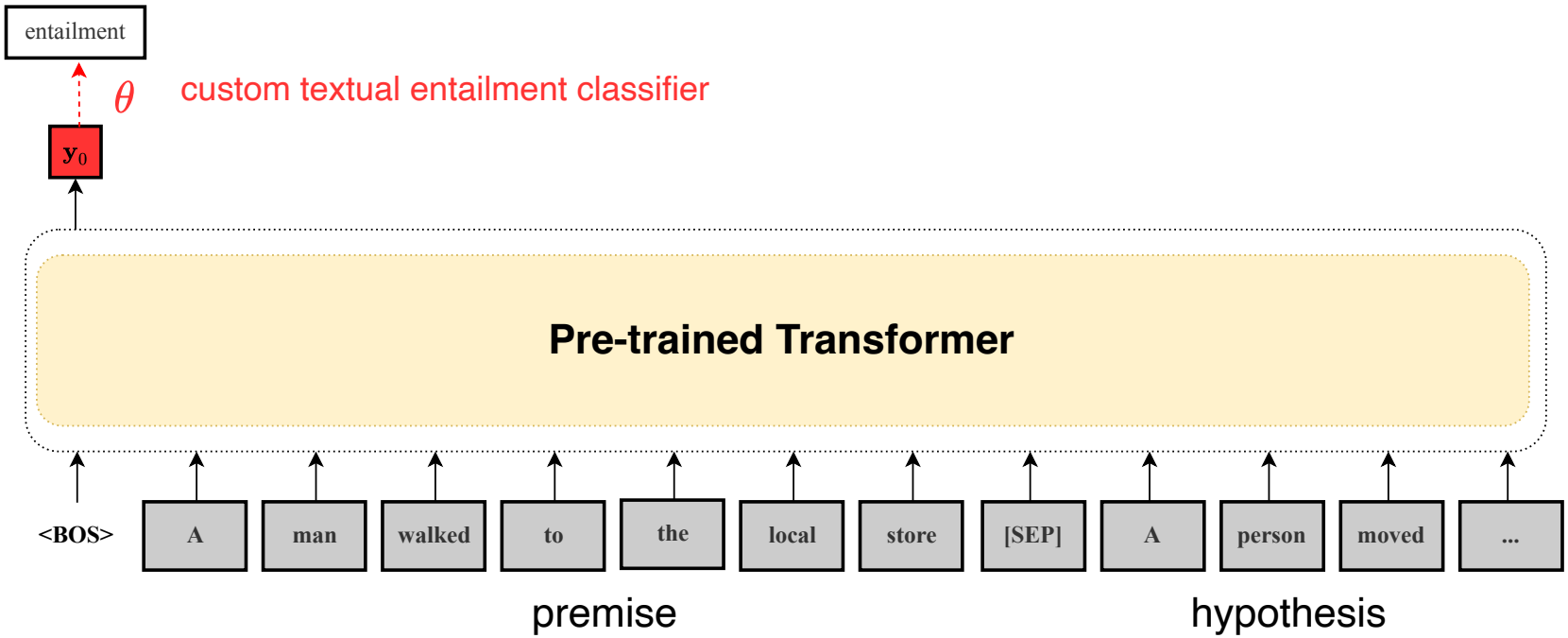
Language model pretraining has led to significant performance gains but careful comparison between different approaches is challenging. Training is computationally expensive, often done on private datasets of different sizes, and, as we will show, hyperparameter choices have significant impact on the final results. We present a replication study of BERT pretraining (Devlin et al., 2019) that carefully measures the impact of many key hyperparameters and training data size. We find that BERT was significantly undertrained, and can match or exceed the performance of every model published after it. Our best model achieves state-of-the-art results on GLUE, RACE and SQuAD. These results highlight the importance of previously overlooked design choices, and raise questions about the source of recently reported improvements. We release our models and code.[1]

We present a replication study of BERT pretraining (Devlin et al., 2019), which includes a careful evaluation of the effects of hyperparmeter tuning and training set size. We find that BERT was significantly undertrained and propose an improved recipe for training BERT models, which we call RoBERTa, that can match or exceed the performance of all of the post-BERT methods. Our modifications are simple, they include: (1) training the model longer, with bigger batches, over more data; (2) removing the next sentence prediction objective; (3) training on longer sequences; and (4) dynamically changing the masking pattern applied to the training data. We also collect a large new dataset (CC-NEWS) of comparable size to other privately used datasets, to better control for training set size effects.

When controlling for training data, our improved training procedure improves upon the published BERT results on both GLUE and SQuAD.

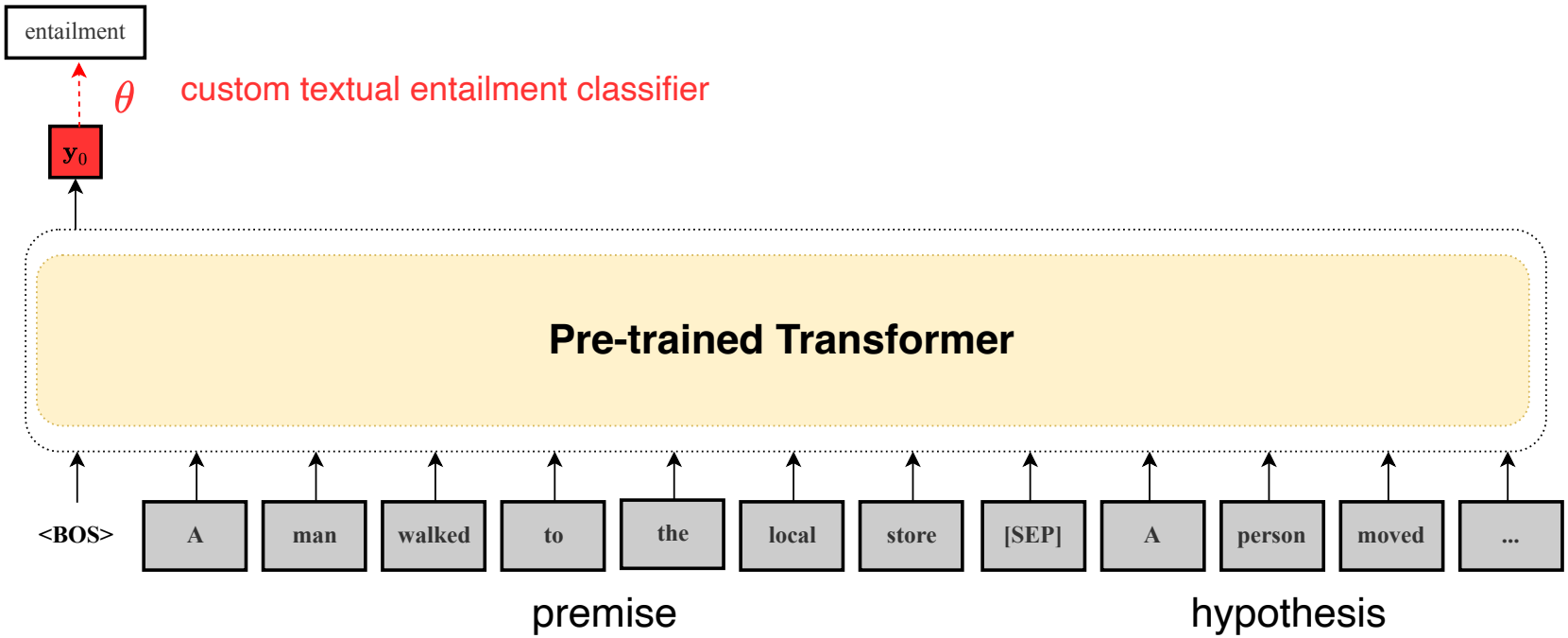
BERT and RoBERTa (Devlin et al., 2018; Liu et al., 2019)

Then: **fine-tuning** the encoder for a task



- **New head**: a classifier on $\mathbf{y}_1$ (`num_labels=3`).

Then: **fine-tuning** the encoder for a task



- **New head**: a classifier on $\mathbf{y}_1$ (`num_labels=3`).
- **Fine-tune**: train all weights on NLI pairs.

Model Development Process: Classification

```
1 nli_model = NLIModel("roberta-base")   ## encoder + new head
```

1. Define a custom model with a base **pre-trained** transformer model and a set of additional classification parameters; couple with a dataset.

2. **Fine-tune** the new parameters and the full transformer using this dataset.

Model Development Process: Classification

```
1 nli_model = NLIModel("roberta-base")   ## encoder + new head
```

1. Define a custom model with a base **pre-trained** transformer model and a set of additional classification parameters; couple with a dataset.

2. **Fine-tune** the new parameters and the full transformer using this dataset.

   The idea: We can leverage the pre-trained knowledge of the model and apply it to new tasks; introduce new task format.

Model Development Process: Classification

```
nli_model = NLIModel("roberta-base")    ## encoder + new head
```

1. Define a custom model with a base **pre-trained** transformer model and a set of additional classification parameters; couple with a dataset.

2. **Fine-tune** the new parameters and the full transformer using this dataset.

   Variations: parameter efficient tuning, adapters; **Alternatives**: Zero/few-shot modeling, prompting. *discussed later*

Model Development Process: Classification

```
nli_model = NLIModel("roberta-base")   ## encoder + new head
```

1. Define a custom model with a base **pre-trained** transformer model and a set of additional classification parameters; couple with a dataset.
2. **Fine-tune** the new parameters and the full transformer using this dataset.

note: Dated problem and approach

Variations: parameter efficient tuning, adapters; **Alternatives**: Zero/few-shot modeling, prompting. *discussed later*

Interim summary

1. **Declarative programming**

   say **what** we want, not **how**

   a solver works out the how

Interim summary

1. **Declarative programming**

   say **what** we want, not **how**

   a solver works out the how

2. **Declarative modeling for ML**

   model predictions as **symbols**

   relations as **constraints**

Interim summary

1. **Declarative programming**
   say **what** we want, not **how**
   a solver works out the how

2. **Declarative modeling for ML**
   model predictions as **symbols**
   relations as **constraints**

3. **Running example: NLI**
   the relation between two texts
   labels: **E**, **C**, **N**

Interim summary

1. **Declarative programming**
   say **what** we want, not **how**
   a solver works out the how

2. **Declarative modeling for ML**
   model predictions as **symbols**
   relations as **constraints**

3. **Running example: NLI**
   the relation between two texts
   labels: **E**, **C**, **N**

4. **Encoder models**
   masked words, then **fine-tune**
   RoBERTa + a new NLI head

Interim summary

1. **Declarative programming**
   say **what** we want, not **how**
   a solver works out the how

2. **Declarative modeling for ML**
   model predictions as **symbols**
   relations as **constraints**

3. **Running example: NLI**
   the relation between two texts
   labels: **E**, **C**, **N**

4. **Encoder models**
   masked words, then **fine-tune**
   RoBERTa + a new NLI head

Questions?

What we want a model to do (semantics)

What rules should an ideal NLI model follow?

- **Predictions as propositions** We can symbolically represent NLI predictions as follows:

$$\mathbf{E}(p, h), \mathbf{C}(p, h), \mathbf{N}(p, h)$$

What rules should an ideal NLI model follow?

- **Predictions as propositions** We can symbolically represent NLI predictions as follows:

$$\mathbf{E}(p, h), \mathbf{C}(p, h), \mathbf{N}(p, h)$$

Denoting an *entailment* (**E**), *neutral* (**N**) or *contradict* (**C**) between any premise/hypothesis pair (p, h).

What rules should an ideal NLI model follow?

- **Predictions as propositions** We can symbolically represent NLI predictions as follows:

$$\mathbf{E}(p, h), \mathbf{C}(p, h), \mathbf{N}(p, h)$$

Denoting an *entailment* (**E**), *neutral* (**N**) or *contradict* (**C**) between any premise/hypothesis pair (p, h).

Example rules on predictions:

$$\forall(p, h).\, \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$$
$$\forall(p, h).\, \mathbf{E}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$
$$\forall(p, h).\, \mathbf{N}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

(example rules from Li et al. (2019); Minervini and Riedel (2018))

What rules should an ideal NLI model follow?

- **Example rules**

$$\forall(p, h).\, \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{E}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{N}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

For the pair

p :John is wearing a green shirt

h :John is wearing a yellow shirt,

contradiction is a symmetric relation, flipping the order yields the same prediction.

What rules should an ideal NLI model follow?

- **Example rules**

$$\forall(p, h).\, \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{E}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{N}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

For the pair

p :A man is wearing a **big** green shirt

h :A man is wearing a green shirt.

there shouldn't be a contradiction if I flip the order for an entailment or a neutral prediction.

What rules should an ideal NLI model follow?

- **Example rules**

$$\forall(p, h).\, \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{E}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

$$\forall(p, h).\, \mathbf{N}(p, h) \rightarrow \neg\mathbf{C}(h, p)$$

For the pair

p :A man is wearing a **big** green shirt

h :A man is wearing a green shirt.

Li et al. (2019) found such constraints to be routinely violated, e.g., BERT violates contradiction symmetry around 20% of time.

What rules should an ideal NLI model follow?

- **More rules**

  Transitivity relations Given the triple (p, h, z)

$$\forall(p, h, z).$$
$$((\mathbf{E}(p, h) \wedge \mathbf{E}(h, z) \rightarrow \mathbf{E}(p, z)) \wedge$$
$$(\mathbf{E}(p, h) \wedge \mathbf{C}(h, z) \rightarrow \mathbf{C}(p, z)) \wedge$$
$$(\mathbf{N}(p, h) \wedge \mathbf{E}(h, z) \rightarrow \neg\mathbf{C}(p, z)) \wedge$$
$$(\mathbf{N}(p, h) \wedge \mathbf{C}(h, z) \rightarrow \neg\mathbf{E}(p, z)))$$

What rules should an ideal NLI model follow?

- **More rules**

  Transitivity relations Given the triple (p, h, z)

$$\forall(p,h,z).$$
$$((\mathbf{E}(p,h) \wedge \mathbf{E}(h,z) \rightarrow \mathbf{E}(p,z)) \wedge$$
$$(\mathbf{E}(p,h) \wedge \mathbf{C}(h,z) \rightarrow \mathbf{C}(p,z)) \wedge$$
$$(\mathbf{N}(p,h) \wedge \mathbf{E}(h,z) \rightarrow \neg\mathbf{C}(p,z)) \wedge$$
$$(\mathbf{N}(p,h) \wedge \mathbf{C}(h,z) \rightarrow \neg\mathbf{E}(p,z)))$$

  Uniqueness constraints: there should be only one prediction.

$$\forall(p,h).$$
$$(\mathbf{E}(p,h) \vee \mathbf{C}(p,h) \vee \mathbf{N}(p,h)) \wedge$$
$$(\neg\mathbf{E}(p,h) \vee \neg\mathbf{C}(p,h)) \wedge$$
$$(\neg\mathbf{E}(p,h) \vee \neg\mathbf{N}(p,h)) \wedge$$
$$(\neg\mathbf{N}(p,h) \vee \neg\mathbf{C}(p,h))$$

What rules should an ideal NLI model follow?

- **More rules**

Transitivity relations Given the triple (p, h, z)

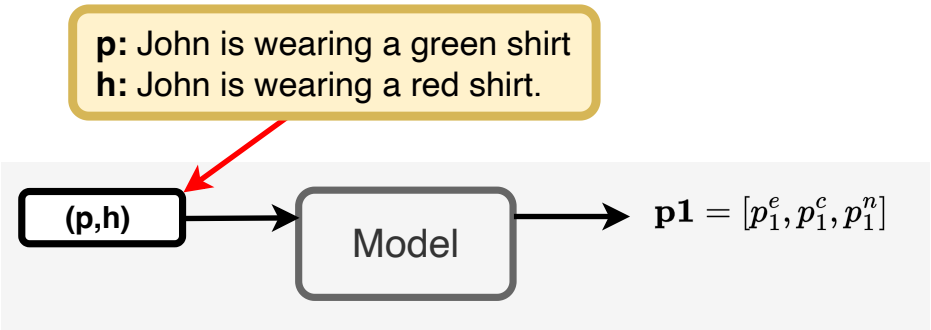
$$\forall(p, h, z).$$
$$((\mathbf{E}(p, h) \wedge \mathbf{E}(h, z) \rightarrow \mathbf{E}(p, z)) \wedge$$
$$(\mathbf{E}(p, h) \wedge \mathbf{C}(h, z) \rightarrow \mathbf{C}(p, z)) \wedge$$
$$(\mathbf{N}(p, h) \wedge \mathbf{E}(h, z) \rightarrow \neg\mathbf{C}(p, z)) \wedge$$
$$(\mathbf{N}(p, h) \wedge \mathbf{C}(h, z) \rightarrow \neg\mathbf{E}(p, z)))$$

note: Does not require labeled data!

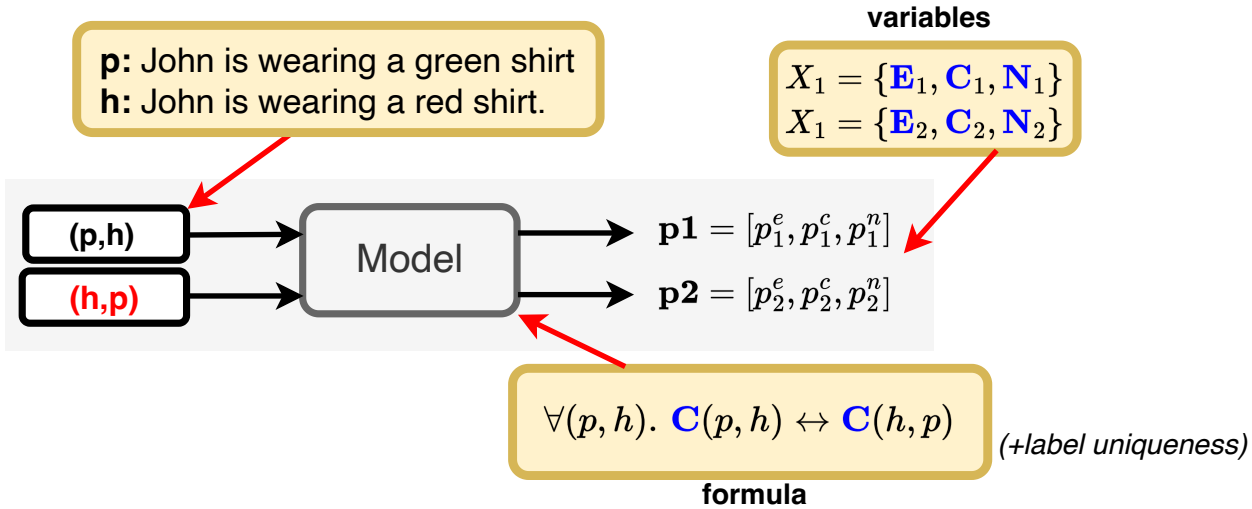
Uniqueness constraints: there should be only one prediction.

$$\forall(p, h).$$
$$(\mathbf{E}(p, h) \vee \mathbf{C}(p, h) \vee \mathbf{N}(p, h)) \wedge$$
$$(\neg\mathbf{E}(p, h) \vee \neg\mathbf{C}(p, h)) \wedge$$
$$(\neg\mathbf{E}(p, h) \vee \neg\mathbf{N}(p, h)) \wedge$$
$$(\neg\mathbf{N}(p, h) \vee \neg\mathbf{C}(p, h))$$

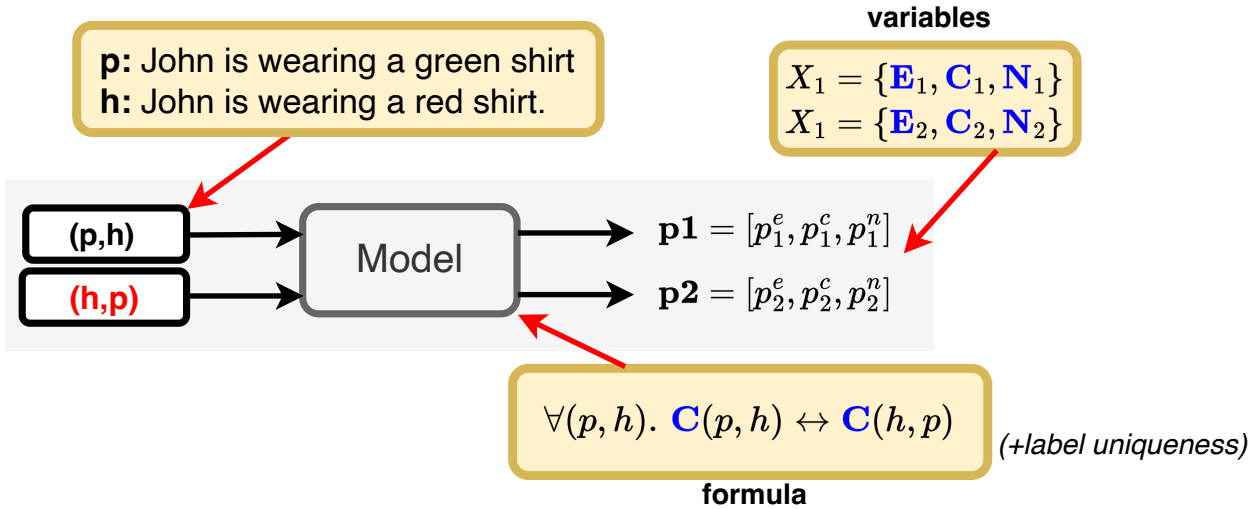
Modeling with declarative knowledge: Semantics



Modeling with declarative knowledge: Semantics



Modeling with declarative knowledge: Semantics



Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
w_1	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	yes	yes
w_2	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	no	no
w_3	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	no	no
w_4	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
w_5	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
w_6	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no
w_7	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
w_8	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
w_9	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no

count: 5

Modeling with declarative knowledge: Semantics

Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
w_1	$\neg E_1, C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	yes	yes
w_2	$\neg E_1, C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	no	no
w_3	$\neg E_1, C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	no	no
w_4	$\neg E_1, \neg C_1, N_1, \neg E_2, C_2, \neg N_2$	no	no
w_5	$\neg E_1, \neg C_1, N_1, \neg E_2, \neg C_2, N_2$	yes	no
w_6	$\neg E_1, \neg C_1, N_1, E_2, \neg C_2, \neg N_2$	yes	no
w_7	$E_1, \neg C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	no	no
w_8	$E_1, \neg C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	yes	no
w_9	$E_1, \neg C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	yes	no
		count: 5	

Modeling with declarative knowledge: Semantics

Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
w_1	$\neg E_1, C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	yes	yes
w_2	$\neg E_1, C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	no	no
w_3	$\neg E_1, C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	no	no
w_4	$\neg E_1, \neg C_1, N_1, \neg E_2, C_2, \neg N_2$	no	no
w_5	$\neg E_1, \neg C_1, N_1, \neg E_2, \neg C_2, N_2$	yes	no
w_6	$\neg E_1, \neg C_1, N_1, E_2, \neg C_2, \neg N_2$	yes	no
w_7	$E_1, \neg C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	no	no
w_8	$E_1, \neg C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	yes	no
w_9	$E_1, \neg C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	yes	no
		count: 5	

What we care about: worlds **w** where our target formula F is true (denoted as $\mathbf{w} \models F$)

Modeling with declarative knowledge: Semantics

Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
$\mathbf{w}_1$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	yes	yes
$\mathbf{w}_2$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	no	no
$\mathbf{w}_3$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_4$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_5$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_6$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no
$\mathbf{w}_7$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_8$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_9$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no
		count: 5	

What we care about: worlds **w** where our target formula F is true (denoted as $\mathbf{w} \models F$)

$$\textsc{count}(F) := \sum_{\mathbf{w} \models F} 1$$

Modeling with declarative knowledge: Semantics

Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
$\mathbf{w}_1$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	yes	yes
$\mathbf{w}_2$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	no	no
$\mathbf{w}_3$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_4$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_5$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_6$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no
$\mathbf{w}_7$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	no	no
$\mathbf{w}_8$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_9$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	yes	no
		count: 5	

With prediction scores: weight each row by them.

$$s_\theta(\mathbf{w}) = \prod_{X \in \mathbf{w}} P_\theta(X) \prod_{\neg X \in \mathbf{w}} \big(1 - P_\theta(X)\big) \qquad \text{score of a row}$$

Modeling with declarative knowledge: Semantics

Thinking about our formula F in conventional semantic terms.

	Worlds	Satisfies F?	Correct?
$\mathbf{w}_1$	$\neg \mathbf{E}_1, \mathbf{C}_1, \neg \mathbf{N}_1, \neg \mathbf{E}_2, \mathbf{C}_2, \neg \mathbf{N}_2$	yes	yes
$\mathbf{w}_2$	$\neg \mathbf{E}_1, \mathbf{C}_1, \neg \mathbf{N}_1, \neg \mathbf{E}_2, \neg \mathbf{C}_2, \mathbf{N}_2$	no	no
$\mathbf{w}_3$	$\neg \mathbf{E}_1, \mathbf{C}_1, \neg \mathbf{N}_1, \mathbf{E}_2, \neg \mathbf{C}_2, \neg \mathbf{N}_2$	no	no
$\mathbf{w}_4$	$\neg \mathbf{E}_1, \neg \mathbf{C}_1, \mathbf{N}_1, \neg \mathbf{E}_2, \mathbf{C}_2, \neg \mathbf{N}_2$	no	no
$\mathbf{w}_5$	$\neg \mathbf{E}_1, \neg \mathbf{C}_1, \mathbf{N}_1, \neg \mathbf{E}_2, \neg \mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_6$	$\neg \mathbf{E}_1, \neg \mathbf{C}_1, \mathbf{N}_1, \mathbf{E}_2, \neg \mathbf{C}_2, \neg \mathbf{N}_2$	yes	no
$\mathbf{w}_7$	$\mathbf{E}_1, \neg \mathbf{C}_1, \neg \mathbf{N}_1, \neg \mathbf{E}_2, \mathbf{C}_2, \neg \mathbf{N}_2$	no	no
$\mathbf{w}_8$	$\mathbf{E}_1, \neg \mathbf{C}_1, \neg \mathbf{N}_1, \neg \mathbf{E}_2, \neg \mathbf{C}_2, \mathbf{N}_2$	yes	no
$\mathbf{w}_9$	$\mathbf{E}_1, \neg \mathbf{C}_1, \neg \mathbf{N}_1, \mathbf{E}_2, \neg \mathbf{C}_2, \neg \mathbf{N}_2$	yes	no
		count: 5	

With prediction scores: weight each row by them.

$$s_\theta(\mathbf{w}) = \prod_{X \in \mathbf{w}} P_\theta(X) \prod_{\neg X \in \mathbf{w}} \big(1 - P_\theta(X)\big) \qquad \text{score of a row}$$

$$\mathrm{WMC}(F, \theta) = \sum_{\mathbf{w} \models F} s_\theta(\mathbf{w}) \qquad \text{rows satisfying } F$$

Modeling with declarative knowledge: Semantics

- Suppose for F that we have weights θ:

  $P_\theta(E_1), P_\theta(C_1), P_\theta(N_1), P_\theta(E_2), P_\theta(C_2), P_\theta(N_2) = [0.3, 0.2, 0.5, 0.1, 0.8, 0.1]$

  then

	Worlds	Satisfies F?	Score
$\mathbf{w}_1$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	True	0.0454
$\mathbf{w}_2$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	False	0.0013
$\mathbf{w}_3$	$\neg\mathbf{E}_1, \mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	False	0.0013
$\mathbf{w}_4$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	False	0.1814
$\mathbf{w}_5$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	True	0.005
$\mathbf{w}_6$	$\neg\mathbf{E}_1, \neg\mathbf{C}_1, \mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	True	0.005
$\mathbf{w}_7$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \mathbf{C}_2, \neg\mathbf{N}_2$	False	0.0778
$\mathbf{w}_8$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \neg\mathbf{E}_2, \neg\mathbf{C}_2, \mathbf{N}_2$	True	0.0022
$\mathbf{w}_9$	$\mathbf{E}_1, \neg\mathbf{C}_1, \neg\mathbf{N}_1, \mathbf{E}_2, \neg\mathbf{C}_2, \neg\mathbf{N}_2$	True	0.0022

WMC: $\approx$0.05976

e.g., score of $\mathbf{w}_1 = (1 - 0.3) * 0.2 * (1 - 0.5) * (1 - 0.1) * 0.8 * (1 - 0.1)$

Modeling with declarative knowledge: Semantics

- Suppose for F that we have weights θ:

  $P_\theta(E_1), P_\theta(C_1), P_\theta(N_1), P_\theta(E_2), P_\theta(C_2), P_\theta(N_2) = [0.3, 0.2, 0.5, 0.1, 0.8, 0.1]$

  then

	Worlds	Satisfies F?	Score
w_1	$\neg E_1, C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	True	0.0454
w_2	$\neg E_1, C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	False	0.0013
w_3	$\neg E_1, C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	False	0.0013
w_4	$\neg E_1, \neg C_1, N_1, \neg E_2, C_2, \neg N_2$	False	0.1814
w_5	$\neg E_1, \neg C_1, N_1, \neg E_2, \neg C_2, N_2$	True	0.005
w_6	$\neg E_1, \neg C_1, N_1, E_2, \neg C_2, \neg N_2$	True	0.005
w_7	$E_1, \neg C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	False	0.0778
w_8	$E_1, \neg C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	True	0.0022
w_9	$E_1, \neg C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	True	0.0022

WMC: $\approx$0.05976

  Total count $\mathrm{WMC}(F) = 0.05976$

Modeling with declarative knowledge: Semantics

- Suppose for F that we have weights θ:

  $$P_\theta(E_1), P_\theta(C_1), P_\theta(N_1), P_\theta(E_2), P_\theta(C_2), P_\theta(N_2) = [0.3, 0.2, 0.5, 0.1, 0.8, 0.1]$$

  then

	Worlds	Satisfies F?	Score
w_1	$\neg E_1, C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	True	0.0454
w_2	$\neg E_1, C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	False	0.0013
w_3	$\neg E_1, C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	False	0.0013
w_4	$\neg E_1, \neg C_1, N_1, \neg E_2, C_2, \neg N_2$	False	0.1814
w_5	$\neg E_1, \neg C_1, N_1, \neg E_2, \neg C_2, N_2$	True	0.005
w_6	$\neg E_1, \neg C_1, N_1, E_2, \neg C_2, \neg N_2$	True	0.005
w_7	$E_1, \neg C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	False	0.0778
w_8	$E_1, \neg C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	True	0.0022
w_9	$E_1, \neg C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	True	0.0022

WMC: ≈ 0.05976

Intuitively: the higher the count, the more our rule is satisfied.

Modeling with declarative knowledge: Semantics

- Suppose for F that we have weights θ:

  $P_\theta(E_1), P_\theta(C_1), P_\theta(N_1), P_\theta(E_2), P_\theta(C_2), P_\theta(N_2) = [0.3, 0.2, 0.5, 0.1, 0.8, 0.1]$

  then

	Worlds	Satisfies F?	Score
$\mathbf{w}_1$	$\neg E_1, C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	True	0.0454
$\mathbf{w}_2$	$\neg E_1, C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	False	0.0013
$\mathbf{w}_3$	$\neg E_1, C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	False	0.0013
$\mathbf{w}_4$	$\neg E_1, \neg C_1, N_1, \neg E_2, C_2, \neg N_2$	False	0.1814
$\mathbf{w}_5$	$\neg E_1, \neg C_1, N_1, \neg E_2, \neg C_2, N_2$	True	0.005
$\mathbf{w}_6$	$\neg E_1, \neg C_1, N_1, E_2, \neg C_2, \neg N_2$	True	0.005
$\mathbf{w}_7$	$E_1, \neg C_1, \neg N_1, \neg E_2, C_2, \neg N_2$	False	0.0778
$\mathbf{w}_8$	$E_1, \neg C_1, \neg N_1, \neg E_2, \neg C_2, N_2$	True	0.0022
$\mathbf{w}_9$	$E_1, \neg C_1, \neg N_1, E_2, \neg C_2, \neg N_2$	True	0.0022

WMC: ≈ 0.05976

As a probability: the scores $s_\theta(\mathbf{w})$ add up to 1 over **all** worlds, so $\text{WMC}(F) = P_\theta(F)$, the probability that the model's predictions satisfy F.

Computing wmc(F) in symprog

```
from symprog import Parser   ## pip install symprog

@Parser(language="constraint")
def program():
    E1: 0.3; C1: 0.2; N1: 0.5   ## model on (p, h)
    E2: 0.1; C2: 0.8; N2: 0.1   ## model on (h, p)

    Biconditional(C1, C2)       ## symmetry of C

    E1 or C1 or N1              ## label uniqueness
    not (E1 and C1); not (E1 and N1); not (C1 and N1)
    E2 or C2 or N2
    not (E2 and C2); not (E2 and N2); not (C2 and N2)

program.inference().inference   ## => 0.0598, WMC(F) = P(F)
```

Questions?

Compiling to Loss

A Semantic Loss Function for Deep Learning with Symbolic Knowledge

Jingyi Xu [1] **Zilu Zhang** [2] **Tal Friedman** [1] **Yitao Liang** [1] **Guy Van den Broeck** [1]

Abstract

This paper develops a novel methodology for using symbolic knowledge in deep learning. From first principles, we derive a semantic loss function that bridges between neural output vectors and logical constraints. This loss function captures how close the neural network is to satisfying the constraints on its output. An experimental evaluation shows that it effectively guides the learner to achieve (near-)state-of-the-art results on semi-supervised multi-class classification. Moreover, it significantly increases the ability of the neural network to predict structured objects, such as rankings and paths. These discrete concepts are tremendously difficult to learn, and benefit from a tight integration of deep learning and symbolic reasoning methods.

This paper considers learning in domains where we have symbolic knowledge connecting the different outputs of a neural network. This knowledge takes the form of a constraint (or sentence) in Boolean logic. It can be as simple as an exactly-one constraint for one-hot output encodings, or as complex as a structured output prediction constraint for intricate combinatorial objects such as rankings, subgraphs, or paths. Our goal is to augment neural networks with the ability to learn how to make predictions subject to these constraints, and use the symbolic knowledge to improve the learning performance.

Most neuro-symbolic approaches aim to simulate or learn symbolic reasoning in an end-to-end deep neural network, or capture symbolic knowledge in a vector-space embedding. This choice is partly motivated by the need for smooth *differentiable* models; adding symbolic reasoning code (e.g., SAT solvers) to a deep learning pipeline destroys this property. Unfortunately, while making rea-

Semantic loss (Xu et al., 2018)

Semantic loss

- **Recall**: the probability of a formula is its weighted model count,

$$P_\theta(F) = \text{WMC}(F, \theta) \qquad \text{the model's predictions satisfy } F$$

Semantic loss

- **Recall**: the probability of a formula is its weighted model count,

$$P_\theta(F) = \mathrm{WMC}(F, \theta) \qquad \text{the model's predictions satisfy } F$$

- The **semantic loss** $\mathcal{L}_{\mathsf{sl}}$ (Xu et al., 2018) for F is:

$$\mathcal{L}_{\mathsf{sl}}(F, \theta) := -\log P_\theta(F) = -\log \mathrm{WMC}(F, \theta)$$

Semantic loss

- **Recall**: the probability of a formula is its weighted model count,

$$P_\theta(F) = \mathrm{WMC}(F, \theta) \qquad \text{the model's predictions satisfy } F$$

- The **semantic loss** $\mathcal{L}_{\mathsf{sl}}$ (Xu et al., 2018) for F is:

$$\mathcal{L}_{\mathsf{sl}}(F, \theta) := -\log P_\theta(F) = -\log \mathrm{WMC}(F, \theta)$$

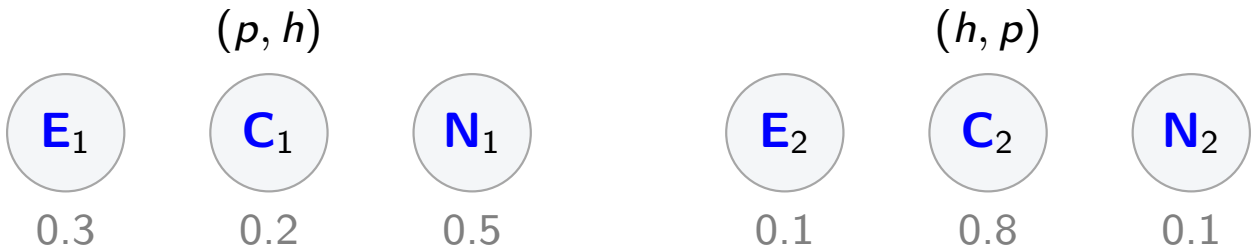
> The semantic loss is ... [equal]... to a negative logarithm of the probability of generating a state that satisfies the constraint when sampling values according to p. Hence, it is the self-information (or 'surprise') of obtaining an assignment that satisfies the constraint... (Xu et al., 2018)

Semantic loss: the underlying probabilistic model



one independent coin per label, **no edges**

Semantic loss: the underlying probabilistic model



one independent coin per label, **no edges**

- **Fully factorized**: the joint is a **product** of one distribution per variable,

$$p_\theta(\mathbf{w}) = \prod_{i=1}^{n} P_\theta(X_i = \mathbf{w}_i) \qquad \text{e.g., } p_\theta(\mathbf{w}_1) = 0.7 \cdot 0.2 \cdots 0.9 \approx 0.045$$

Semantic loss: the underlying probabilistic model



one independent coin per label, **no edges**

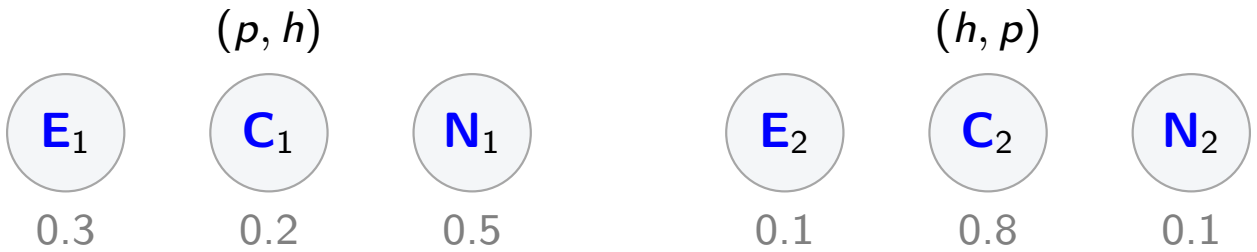
- **Fully factorized**: the joint is a **product** of one distribution per variable,

$$p_\theta(\mathbf{w}) = \prod_{i=1}^{n} P_\theta(X_i = \mathbf{w}_i) \qquad \text{e.g., } p_\theta(\mathbf{w}_1) = 0.7 \cdot 0.2 \cdots 0.9 \approx 0.045$$

- So the row **score is a probability**, $s_\theta(\mathbf{w}) = p_\theta(\mathbf{w})$, and

$$P_\theta(F) = \sum_{\mathbf{w} \models F} p_\theta(\mathbf{w}) = \mathrm{WMC}(F, \theta).$$

Semantic loss: the underlying probabilistic model



one independent coin per label, **no edges**

- **Fully factorized**: the joint is a **product** of one distribution per variable,

$$p_\theta(\mathbf{w}) = \prod_{i=1}^{n} P_\theta(X_i = \mathbf{w}_i) \qquad \text{e.g., } p_\theta(\mathbf{w}_1) = 0.7 \cdot 0.2 \cdots 0.9 \approx 0.045$$

- So the row **score is a probability**, $s_\theta(\mathbf{w}) = p_\theta(\mathbf{w})$, and

$$P_\theta(F) = \sum_{\mathbf{w} \models F} p_\theta(\mathbf{w}) = \mathrm{WMC}(F, \theta).$$

The labels are **independent** given the input; only the loss ties them together (see van Krieken et al., 2024).

Special cases

Semantic loss: special cases

- Suppose we have unlabeled (p, h) pairs that we want to learn from, how? via **uniqueness constraints** or for each instance (simpl. $P(p, h)$ to P):

$$F := \big(\mathbf{E} \wedge \neg \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N}\big)$$

Semantic loss: special cases

- Suppose we have unlabeled (p, h) pairs that we want to learn from, how? via **uniqueness constraints** or for each instance (simpl. $P(p, h)$ to P):

$$F := \big(\mathbf{E} \wedge \neg \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N}\big)$$

Intuition: A model must confidently assign a consistent class, even to unlabeled examples (Xu et al., 2018), **semi-supervised** learning.

Semantic loss: special cases

- Suppose we have unlabeled (p, h) pairs that we want to learn from, how? via **uniqueness constraints** or for each instance (simpl. $P(p, h)$ to P):

$$F := \big(\mathbf{E} \wedge \neg \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N}\big)$$

Intuition: A model must confidently assign a consistent class, even to unlabeled examples (Xu et al., 2018), **semi-supervised** learning.

We can compute this directly:

$$\mathcal{L}_{\text{uniq}}(\theta, F) = -\log \underbrace{\sum_{i=1}^{3} P_\theta(X_i) \cdot \prod_{j=1, j \neq i} (1 - P_\theta(X_i))}_{\text{direct } n^2}$$

Semantic loss: special cases

$$F := (\mathbf{E} \wedge \neg \mathbf{C} \wedge \neg \mathbf{N}) \vee (\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}) \vee (\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N})$$

We can compute this directly:

$$\mathcal{L}_{\text{uniq}}(\theta, F) = -\log \sum_{i=1}^{3} P_\theta(X_i) \cdot \prod_{j=1, j \neq i} (1 - P_\theta(X_i))$$

- Suppose for F that we have weights θ:

$$P_\theta(E), (C), P_\theta(N) = [0.3, 0.2, 0.5]$$

then the loss is equal to

$$-\log \left(\underbrace{(0.3 * 0.8 * 0.5)}_{\mathbf{E} \wedge \neg \mathbf{C} \wedge \neg \mathbf{N}} + (0.2 * 0.7 * 0.5) + (0.5 * 0.7 * 0.8) \right)$$

Model sketch in PyTorch: semi-supervised learning

```
unlabel_p = "John is wearing a red shirt"       ## an unlabeled pair
unlabel_h = "John is wearing a yellow shirt"
batch = tokenizer([(p, h), (unlabel_p, unlabel_h)],
                  return_tensors="pt", padding=True)
labels = torch.tensor([1, -100])              ## -100: no label

label_loss, probs = nli_model({**batch, "labels": labels})

unlab_probs = probs[1:]                       ## the unlabeled pair
wmc = torch.zeros_like(unlab_probs)
for i in range(3):                            ## one world per label
    negate       = 1 - unlab_probs
    negate[:, i] = unlab_probs[:, i]
    wmc[:, i]    = negate.prod(dim=-1)

wmc_loss = -torch.log(wmc.sum(dim=-1)).mean()
loss = label_loss + wmc_loss                  ## train on both
```

original implementation, pytorch version (version above)

Model sketch in PyTorch: semi-supervised learning

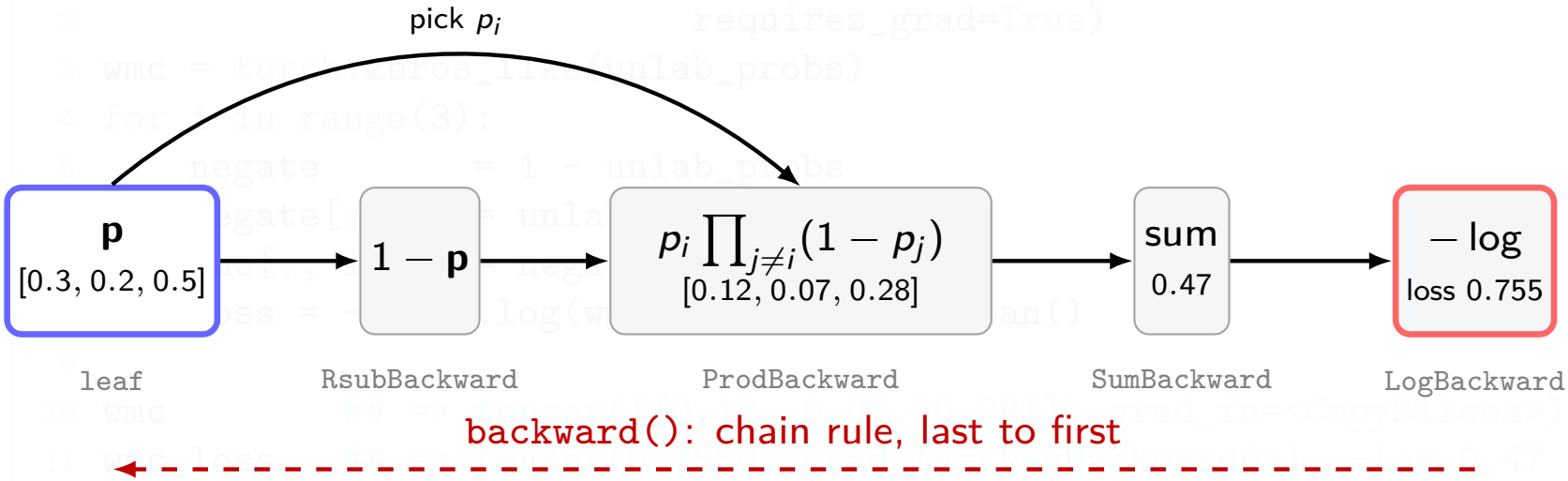
```
unlab_probs = torch.tensor([[0.3, 0.2, 0.5]],   ## our toy example
                           requires_grad=True)
wmc = torch.zeros_like(unlab_probs)
for i in range(3):
    negate       = 1 - unlab_probs
    negate[:, i] = unlab_probs[:, i]
    wmc[:, i]    = negate.prod(dim=-1)
wmc_loss = -torch.log(wmc.sum(dim=-1)).mean()

wmc         ## => tensor([[0.12, 0.07, 0.28]], grad_fn=<CopySlices>)
wmc_loss    ## => tensor(0.7550, grad_fn=<NegBackward0>), -log 0.47

wmc_loss.backward()       ## backpropagate
unlab_probs.grad          ## => tensor([[ 0.21,  0.32, -0.38]])
```

some notes on autodiff, more notes.

What PyTorch records: a **computation graph**



$\partial \mathcal{L}/\partial \mathbf{p} = [0.21, 0.32, -0.38]$

to lower the loss: **raise** N, **lower** E and C (one confident label)

Gradients in symprog

```
from symprog import Parser   ## pip install symprog

@Parser(language="constraint")
def program():
    E: 0.3; C: 0.2; N: 0.5                ## our toy example
    E or C or N                           ## label uniqueness
    not (E and C); not (E and N); not (C and N)

program.inference().inference   ## => 0.47
program.grad()                  ## gradients of P(F)
## => {'weights': tensor([-0.1500, -0.1000,  0.1800])}
```

Under the hood: symbolic gradients

`symprog` uses **computer algebra** (SymPy): get the formula, not a number.

In [1]:
```
import sympy                          ## computer algebra
P = program.inference(symbolic=True).inference
P
```

Out[1]: $N\,(1-C)\,(1-E)+(1-N)\,(C\,(1-E)+E\,(1-C))$

Under the hood: symbolic gradients

`symprog` uses **computer algebra** (SymPy): get the formula, not a number.

In [1]:
```
import sympy                          ## computer algebra
P = program.inference(symbolic=True).inference
P
```

Out[1]: $N\,(1-C)\,(1-E)+(1-N)\,(C\,(1-E)+E\,(1-C))$

In [2]:
```
sympy.expand(P)                       ## linear in each weight
```

Out[2]: $3CEN-2CE-2CN+C-2EN+E+N$

Under the hood: symbolic gradients

`symprog` uses **computer algebra** (SymPy): get the formula, not a number.

In [1]:
```
import sympy                          ## computer algebra
P = program.inference(symbolic=True).inference
P
```

Out[1]: $N(1-C)(1-E)+(1-N)(C(1-E)+E(1-C))$

In [2]:
```
sympy.expand(P)                       ## linear in each weight
```

Out[2]: $3CEN-2CE-2CN+C-2EN+E+N$

In [3]:
```
grad = program.grad(symbolic=True)["weights"]
grad                                  ## dP/dC, dP/dE, dP/dN
```

Out[3]:
$$\begin{bmatrix} -N(1-E)+(1-2E)(1-N) \\ -N(1-C)+(1-2C)(1-N) \\ -C(1-E)-E(1-C)+(1-C)(1-E) \end{bmatrix}$$

Under the hood: symbolic gradients

`symprog` uses **computer algebra** (SymPy): get the formula, not a number.

In [1]:
```
import sympy                          ## computer algebra
P = program.inference(symbolic=True).inference
P
```

Out[1]: $N(1-C)(1-E)+(1-N)(C(1-E)+E(1-C))$

In [2]:
```
sympy.expand(P)                       ## linear in each weight
```

Out[2]: $3CEN-2CE-2CN+C-2EN+E+N$

In [3]:
```
grad = program.grad(symbolic=True)["weights"]
grad                                  ## dP/dC, dP/dE, dP/dN
```

Out[3]:
$$\begin{bmatrix} -N(1-E)+(1-2E)(1-N) \\ -N(1-C)+(1-2C)(1-N) \\ -C(1-E)-E(1-C)+(1-C)(1-E) \end{bmatrix}$$

In [4]:
```
E, C, N = sympy.symbols("E C N", positive=True)
grad.subs({E: 0.3, C: 0.2, N: 0.5}).T   ## as before
```

Out[4]: $\begin{bmatrix} -0.15 & -0.1 & 0.18 \end{bmatrix}$

Semantic loss: other special cases

- **Supervised learning**: suppose we have two gold annotations $\mathbf{E}_1(p_1, h_1)$ and $\mathbf{C}_2(p_2, h_2)$ *(no additional variables)* and the formula:

$$F := \mathbf{E}_1(p_1, h_1) \wedge \mathbf{C}_2(p_2, h_2).$$

Semantic loss: other special cases

- **Supervised learning**: suppose we have two gold annotations $\mathbf{E}_1(p_1, h_1)$ and $\mathbf{C}_2(p_2, h_2)$ *(no additional variables)* and the formula:

$$F := \mathbf{E}_1(p_1, h_1) \wedge \mathbf{C}_2(p_2, h_2).$$

Performing counting of this gives the following:

	Worlds	**Satisfies F?**
$\mathbf{w}_1$	$\mathbf{E}_1, \mathbf{C}_2$	True
$\mathbf{w}_2$	$\neg\mathbf{E}_1, \mathbf{C}_2$	False
$\mathbf{w}_3$	$\mathbf{E}_1, \neg\mathbf{C}_2$	False
$\mathbf{w}_4$	$\neg\mathbf{E}_1, \neg\mathbf{C}_2$	False

Semantic loss: other special cases

- **Supervised learning**: suppose we have two gold annotations $\mathbf{E}_1(p_1, h_1)$ and $\mathbf{C}_2(p_2, h_2)$ *(no additional variables)* and the formula:

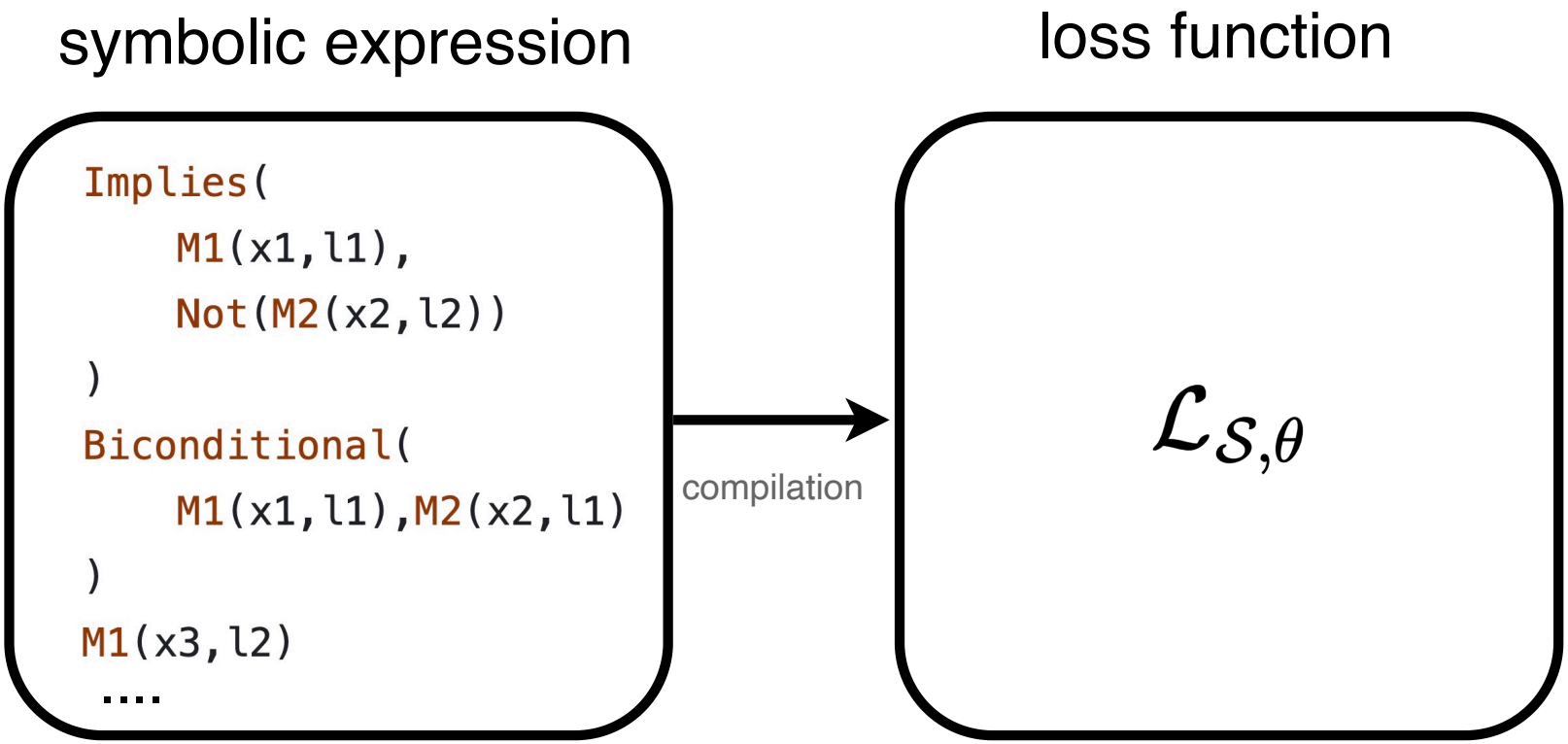
$$F := \mathbf{E}_1(p_1, h_1) \wedge \mathbf{C}_2(p_2, h_2).$$

Performing counting of this gives the following:

	Worlds	**Satisfies F?**
$\mathbf{w}_1$	$\mathbf{E}_1, \mathbf{C}_2$	True
$\mathbf{w}_2$	$\neg\mathbf{E}_1, \mathbf{C}_2$	False
$\mathbf{w}_3$	$\mathbf{E}_1, \neg\mathbf{C}_2$	False
$\mathbf{w}_4$	$\neg\mathbf{E}_1, \neg\mathbf{C}_2$	False

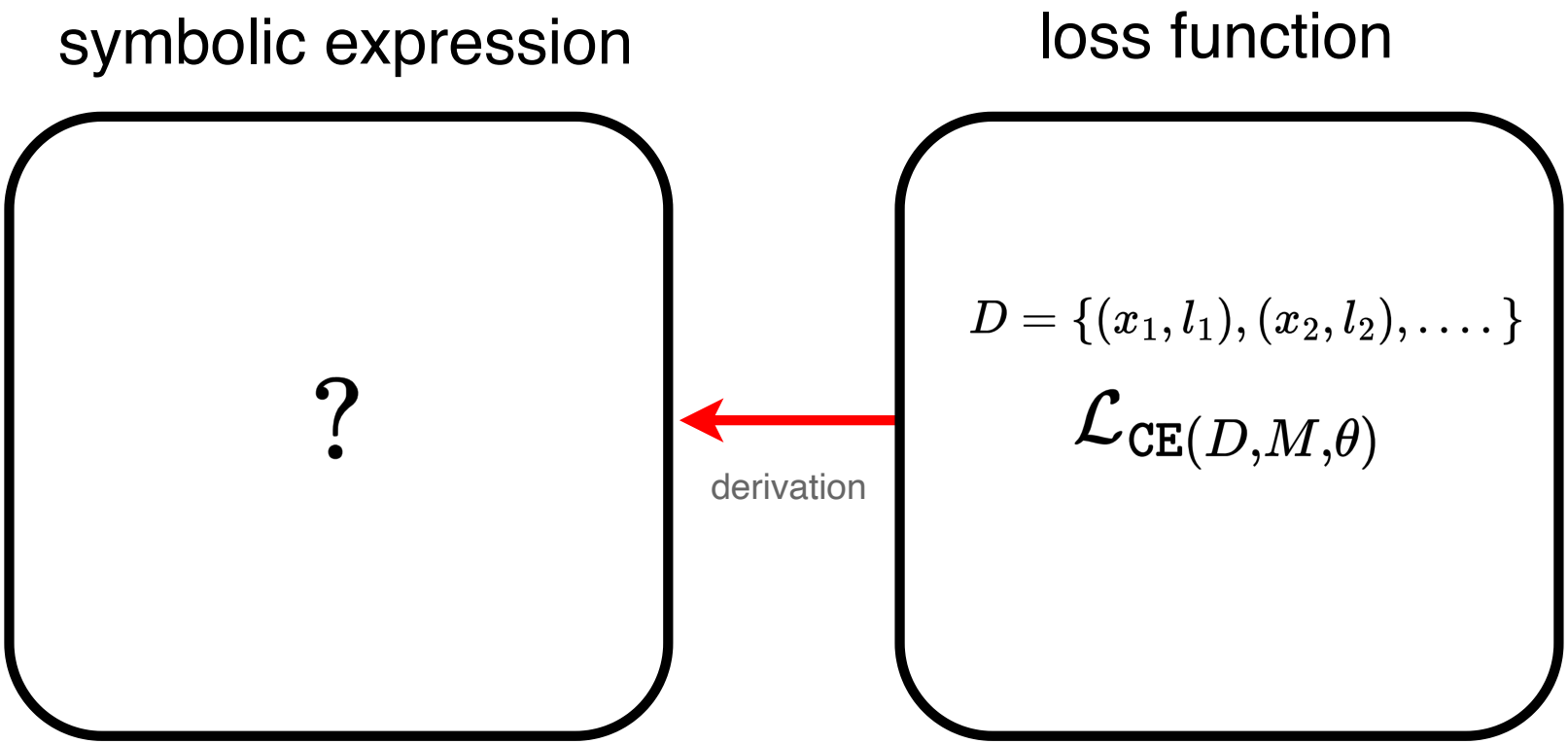
where only a single model gets counted, yielding a semantic loss of $-\log(\mathbf{p}_1 * \mathbf{p}_2)$ equal to the **cross-entropy loss**.

Going in the reverse direction



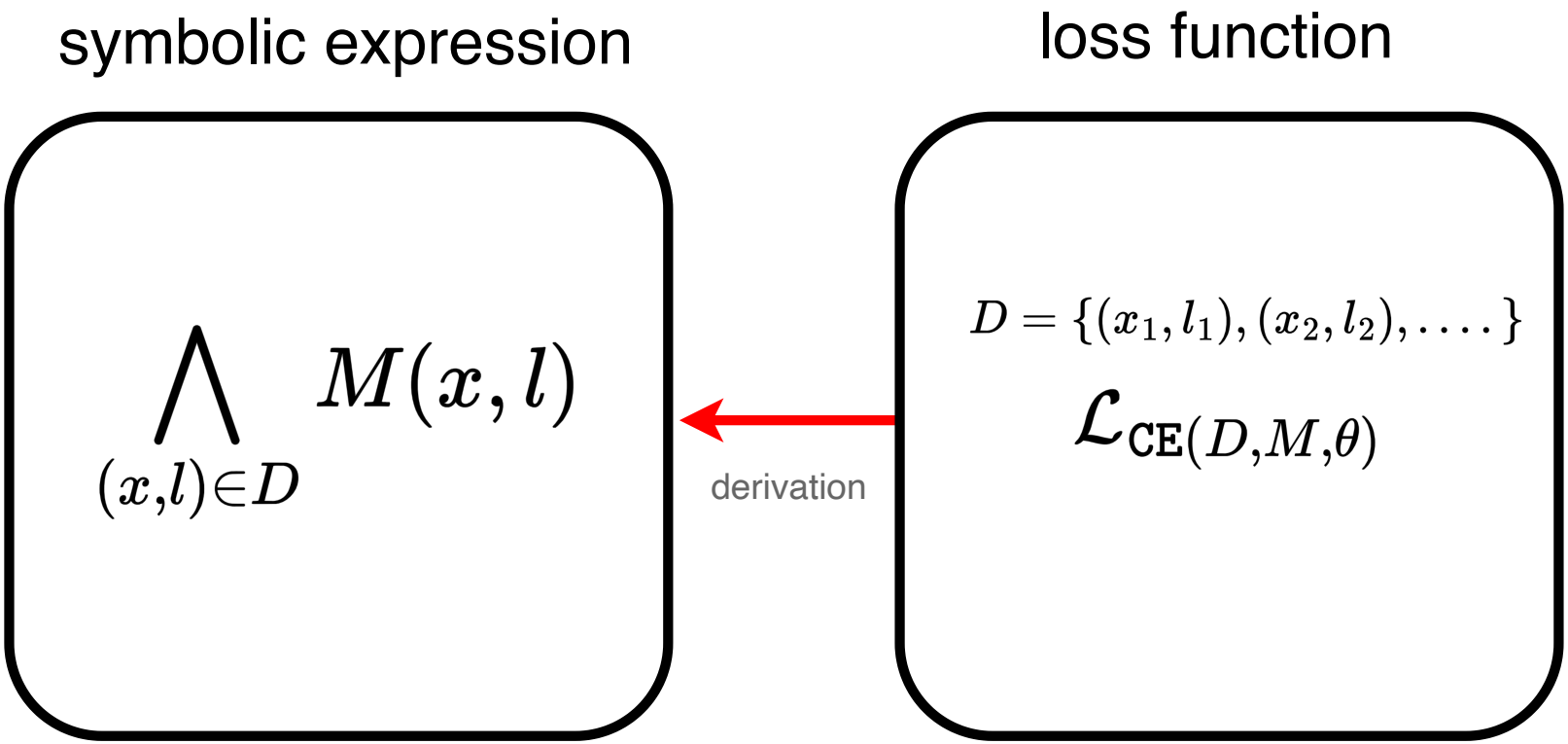
- **So far**: from a symbolic specification to a loss, **compilation**.

Going in the reverse direction



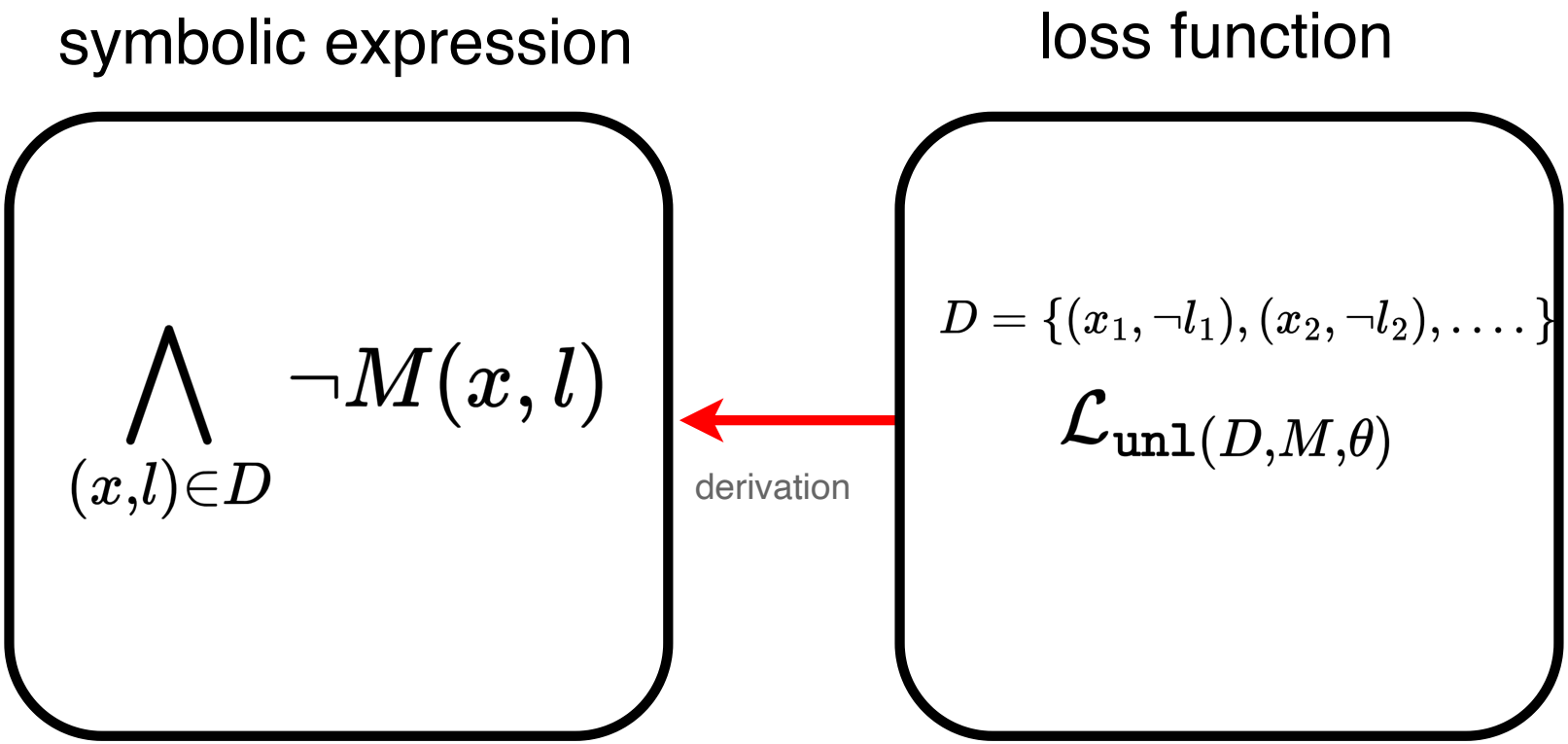
- We can also try to decompile from a known loss function to a symbolic expression, help to understand the semantics of known loss functions.

Going in the reverse direction



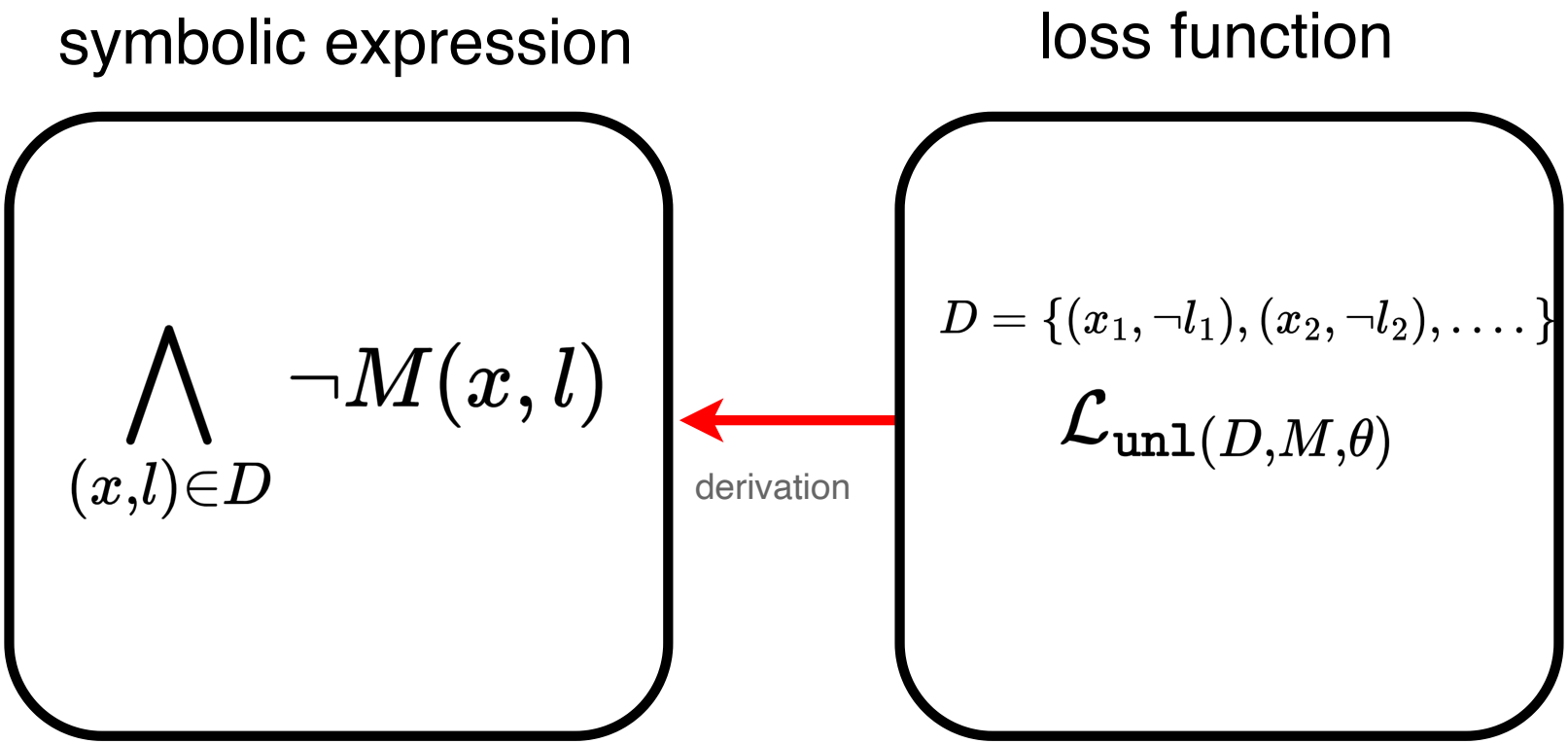
- We can also try to decompile from a known loss function to a symbolic expression, help to understand the semantics of known loss functions.

Going in the reverse direction



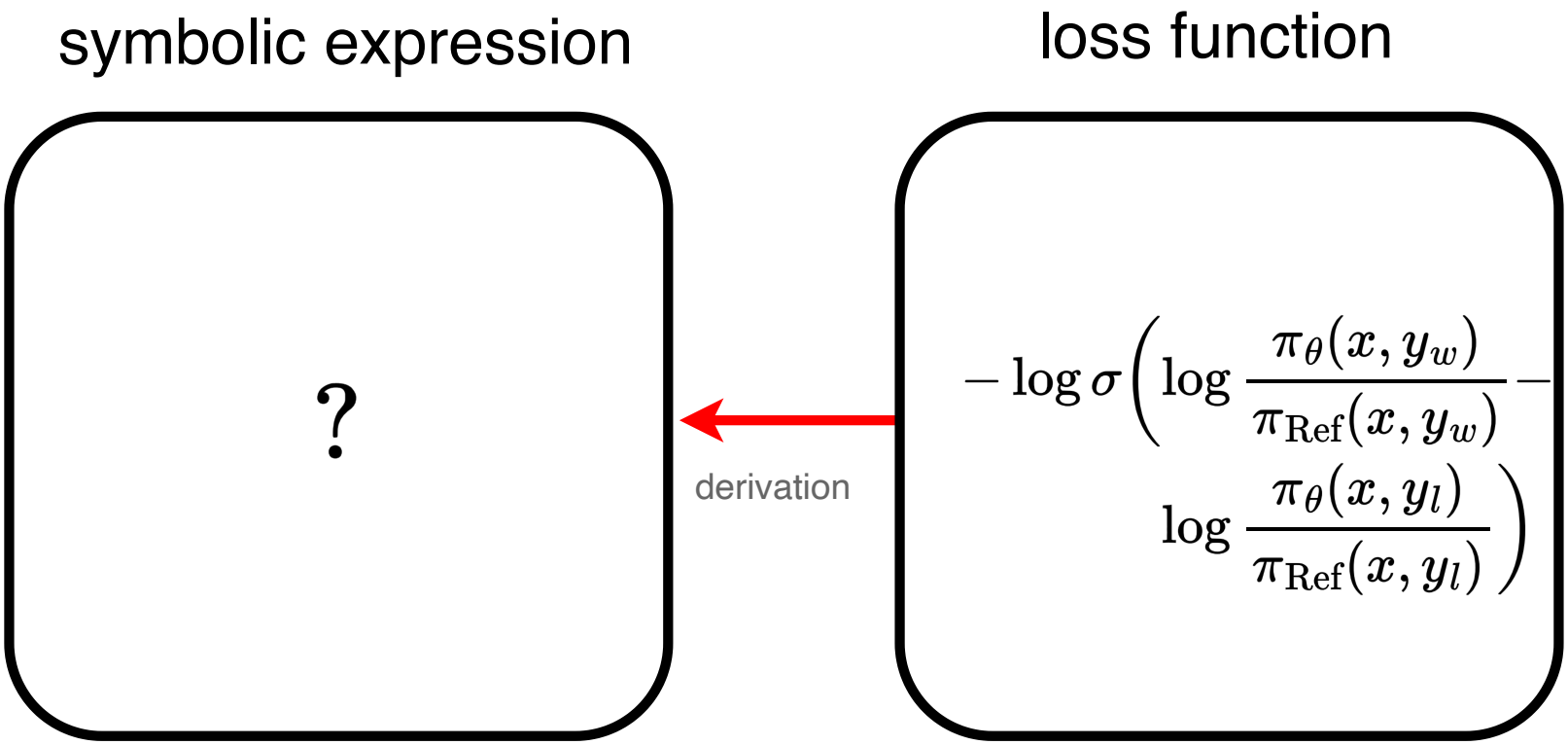
- We can also try to decompile from a known loss function to a symbolic expression, help to understand the semantics of known loss functions.

Going in the reverse direction



observation: standard loss (e.g., cross entropy) functions have a fairly simple logical semantics (Li et al., 2019; Giannini et al., 2023).

Going in the reverse direction



Going in the reverse direction

symbolic expression

loss function

?

$$-\log \sigma\left(\log \frac{\pi_\theta(x, y_w)}{\pi_{\text{Ref}}(x, y_w)} - \log \frac{\pi_\theta(x, y_l)}{\pi_{\text{Ref}}(x, y_l)}\right)$$

Decompilation

from known losses to their semantics

starting next week

Formal properties of semantic loss

Recall: $\mathcal{L}_{\text{sl}}(F, \theta) := -\log P_\theta(F)$

(for any θ; Xu et al., 2018)

Formal properties of semantic loss

Recall: $\mathcal{L}_{\text{sl}}(F, \theta) := -\log P_\theta(F)$

monotonicity if $F_1 \models F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) \geq \mathcal{L}_{\text{sl}}(F_2, \theta)$

a stronger formula has a larger loss

(for any θ; Xu et al., 2018)

Formal properties of semantic loss

Recall: $\mathcal{L}_{\text{sl}}(F, \theta) := -\log P_\theta(F)$

monotonicity if $F_1 \models F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) \geq \mathcal{L}_{\text{sl}}(F_2, \theta)$

a stronger formula has a larger loss

equivalence if $F_1 \equiv F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) = \mathcal{L}_{\text{sl}}(F_2, \theta)$

only the meaning matters, not how it is written

(for any θ; Xu et al., 2018)

Formal properties of semantic loss

Recall: $\mathcal{L}_{\text{sl}}(F, \theta) := -\log P_\theta(F)$

monotonicity	if $F_1 \models F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) \geq \mathcal{L}_{\text{sl}}(F_2, \theta)$ a stronger formula has a larger loss
equivalence	if $F_1 \equiv F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) = \mathcal{L}_{\text{sl}}(F_2, \theta)$ only the meaning matters, not how it is written
differentiability	continuous, differentiable, monotone in each $P_\theta(X)$ we can train with gradient descent

(for any θ; Xu et al., 2018)

Formal properties of semantic loss

Recall: $\mathcal{L}_{\text{sl}}(F, \theta) := -\log P_\theta(F)$

monotonicity if $F_1 \models F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) \geq \mathcal{L}_{\text{sl}}(F_2, \theta)$

a stronger formula has a larger loss

equivalence if $F_1 \equiv F_2$, then $\mathcal{L}_{\text{sl}}(F_1, \theta) = \mathcal{L}_{\text{sl}}(F_2, \theta)$

only the meaning matters, not how it is written

differentiability continuous, differentiable, monotone in each $P_\theta(X)$

we can train with gradient descent

(for any θ; Xu et al., 2018)

Why it matters: we can reason about how different losses relate, both semantically and in how they behave.

Interim summary

With semantic loss, **logic is a language for losses**.

1. Ordinary ML (Grandvalet and Bengio, 2004; Welleck et al., 2019)

$\bigwedge_{(t,P)\in D}$	$P(t)$	cross-entropy
$\bigwedge_{(t,P)\in D_{\text{neg}}}$	$\neg P(t)$	unlikelihood
$\bigwedge_{t\in D_{\text{unl}}}$	$\bigvee_i \left(P_i(t) \wedge \bigwedge_{j\neq i} \neg P_j(t) \right)$	semi-supervised

Interim summary

With semantic loss, **logic is a language for losses**.

1. Ordinary ML (Grandvalet and Bengio, 2004; Welleck et al., 2019)

$\bigwedge_{(t,P)\in D}$	$P(t)$	cross-entropy
$\bigwedge_{(t,P)\in D_{\text{neg}}}$	$\neg P(t)$	unlikelihood
$\bigwedge_{t\in D_{\text{unl}}}$	$\bigvee_i \left(P_i(t) \wedge \bigwedge_{j\neq i} \neg P_j(t) \right)$	semi-supervised

2. Beyond ordinary ML

$\bigwedge_{(t,t')\in D}$	$P(t) \leftrightarrow P(t')$	symmetry
$\bigwedge_{(t_1,t_2)\in D}$	$P_1(t_1) \rightarrow P_2(t_2)$	pair rules

Interim summary

With semantic loss, **logic is a language for losses**.

1. Ordinary ML (Grandvalet and Bengio, 2004; Welleck et al., 2019)

$\bigwedge_{(t,P)\in D}$	$P(t)$	cross-entropy
$\bigwedge_{(t,P)\in D_{\text{neg}}}$	$\neg P(t)$	unlikelihood
$\bigwedge_{t\in D_{\text{unl}}}$	$\bigvee_i \left(P_i(t) \wedge \bigwedge_{j\neq i} \neg P_j(t) \right)$	semi-supervised

2. Beyond ordinary ML

$\bigwedge_{(t,t')\in D}$	$P(t) \leftrightarrow P(t')$	symmetry
$\bigwedge_{(t_1,t_2)\in D}$	$P_1(t_1) \rightarrow P_2(t_2)$	pair rules

> Anything we can write in logic can be a **loss**, *in principle*.

Interim summary: from rules to losses

1. **Running example: NLI**

   rules about **structure**, not labels

   e.g., **C** is symmetric

Interim summary: from rules to losses

1. **Running example: NLI**

   rules about **structure**, not labels

   e.g., **C** is symmetric

2. **Probability of a formula**

   $$P_\theta(F) = \mathrm{WMC}(F, \theta)$$

   a sum over worlds

Interim summary: from rules to losses

1. Running example: NLI

rules about **structure**, not labels

e.g., **C** is symmetric

2. Probability of a formula

$$P_\theta(F) = \text{WMC}(F, \theta)$$

a sum over worlds

3. Semantic loss

$$\mathcal{L}_{\text{sl}}(F, \theta) = -\log P_\theta(F)$$

a regularizer: $\mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}$

Interim summary: from rules to losses

1. Running example: NLI

rules about **structure**, not labels

e.g., **C** is symmetric

2. Probability of a formula

$$P_\theta(F) = \mathrm{WMC}(F, \theta)$$

a sum over worlds

3. Semantic loss

$$\mathcal{L}_{\mathsf{sl}}(F, \theta) = -\log P_\theta(F)$$

a regularizer: $\mathcal{L}_{\mathsf{CE}} + \lambda \mathcal{L}_{\mathsf{sl}}$

4. Special cases

ordinary ML falls out

cross-entropy, semi-sup.

Interim summary: from rules to losses

1. Running example: NLI

rules about **structure**, not labels

e.g., **C** is symmetric

2. Probability of a formula

$$P_\theta(F) = \mathrm{WMC}(F, \theta)$$

a sum over worlds

3. Semantic loss

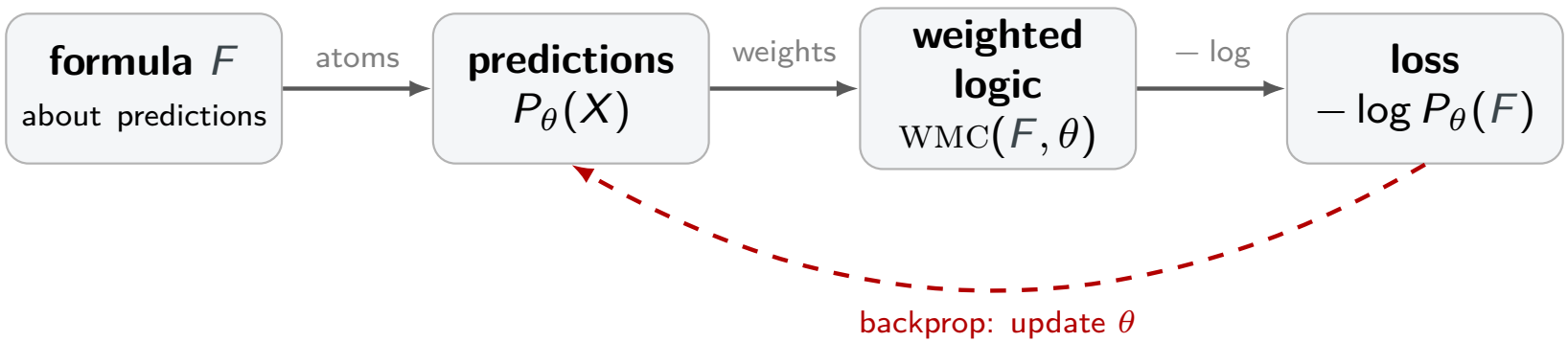
$$\mathcal{L}_{\mathsf{sl}}(F, \theta) = -\log P_\theta(F)$$

a regularizer: $\mathcal{L}_{\mathsf{CE}} + \lambda \mathcal{L}_{\mathsf{sl}}$

4. Special cases

ordinary ML falls out

cross-entropy, semi-sup.



Quick check: true or false?

1. the symmetry rule for **C** needs gold labels
2. $P_\theta(F) = \mathrm{WMC}(F, \theta)$
3. predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$
4. if $F_1 \models F_2$, then F_1 has the smaller loss
5. for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy
6. minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels
7. computing WMC is easy for any formula

Quick check: true or false?

1. the symmetry rule for **C** needs gold labels **false** only structure
2. $P_\theta(F) = \mathrm{WMC}(F, \theta)$
3. predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$
4. if $F_1 \models F_2$, then F_1 has the smaller loss
5. for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy
6. minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels
7. computing WMC is easy for any formula

Quick check: true or false?

1. the symmetry rule for **C** needs gold labels — **false** — only structure
2. $P_\theta(F) = \mathrm{WMC}(F, \theta)$ — **true** — one coin per label
3. predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$
4. if $F_1 \models F_2$, then F_1 has the smaller loss
5. for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy
6. minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels
7. computing WMC is easy for any formula

Quick check: true or false?

1. the symmetry rule for **C** needs gold labels — **false** — only structure
2. $P_\theta(F) = \mathrm{WMC}(F, \theta)$ — **true** — one coin per label
3. predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$ — **true** — $-\log 1 = 0$
4. if $F_1 \models F_2$, then F_1 has the smaller loss
5. for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy
6. minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels
7. computing WMC is easy for any formula

Quick check: true or false?

1. the symmetry rule for **C** needs gold labels — **false** — only structure
2. $P_\theta(F) = \mathrm{WMC}(F, \theta)$ — **true** — one coin per label
3. predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$ — **true** — $-\log 1 = 0$
4. if $F_1 \models F_2$, then F_1 has the smaller loss — **false** — larger or equal
5. for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy
6. minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels
7. computing WMC is easy for any formula

Quick check: true or false?

1.	the symmetry rule for **C** needs gold labels	**false**	only structure
2.	$P_\theta(F) = \mathrm{WMC}(F, \theta)$	**true**	one coin per label
3.	predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$	**true**	$-\log 1 = 0$
4.	if $F_1 \models F_2$, then F_1 has the smaller loss	**false**	larger or equal
5.	for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy	**true**	$-\log p$
6.	minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels		
7.	computing WMC is easy for any formula		

Quick check: true or false?

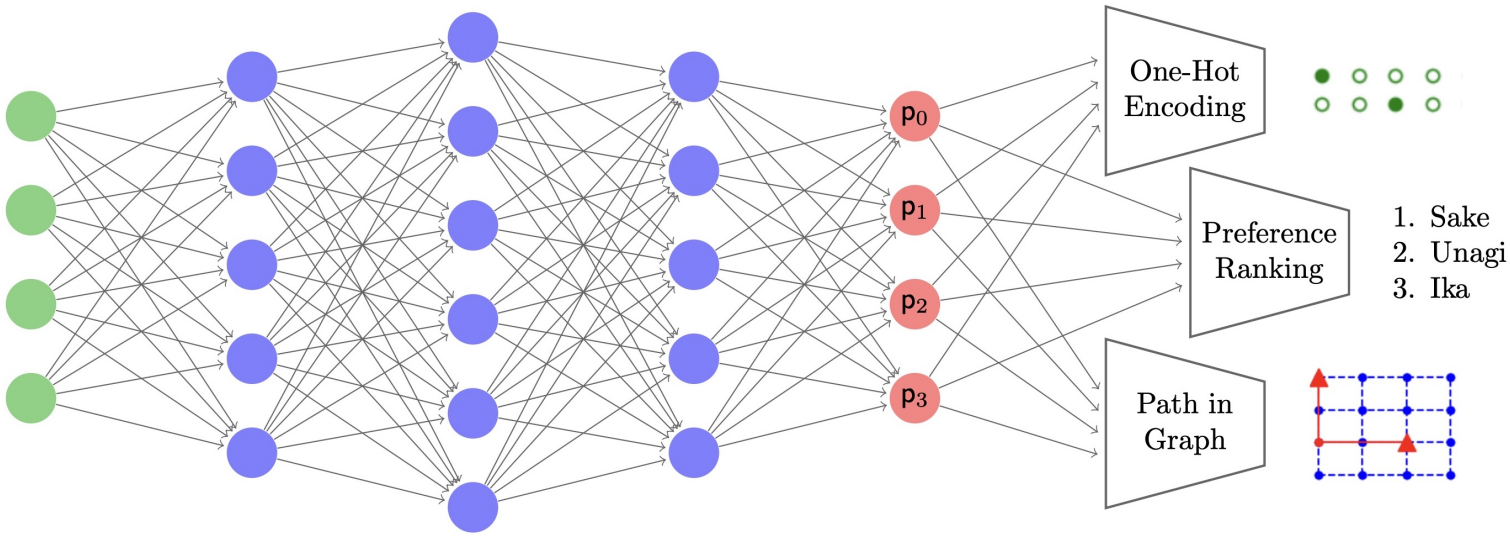
1.	the symmetry rule for **C** needs gold labels	**false**	only structure
2.	$P_\theta(F) = \mathrm{WMC}(F, \theta)$	**true**	one coin per label
3.	predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$	**true**	$-\log 1 = 0$
4.	if $F_1 \models F_2$, then F_1 has the smaller loss	**false**	larger or equal
5.	for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy	**true**	$-\log p$
6.	minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels	**false**	never predict **C**
7.	computing WMC is easy for any formula		

Quick check: true or false?

1.	the symmetry rule for **C** needs gold labels	**false**	only structure
2.	$P_\theta(F) = \mathrm{WMC}(F, \theta)$	**true**	one coin per label
3.	predictions that always satisfy F: $\mathcal{L}_{\mathsf{sl}} = 0$	**true**	$-\log 1 = 0$
4.	if $F_1 \models F_2$, then F_1 has the smaller loss	**false**	larger or equal
5.	for one labeled example, $\mathcal{L}_{\mathsf{sl}}$ is cross-entropy	**true**	$-\log p$
6.	minimizing $\mathcal{L}_{\mathsf{sl}}$ alone finds the right labels	**false**	never predict **C**
7.	computing WMC is easy for any formula	**false**	#P-hard

Questions?

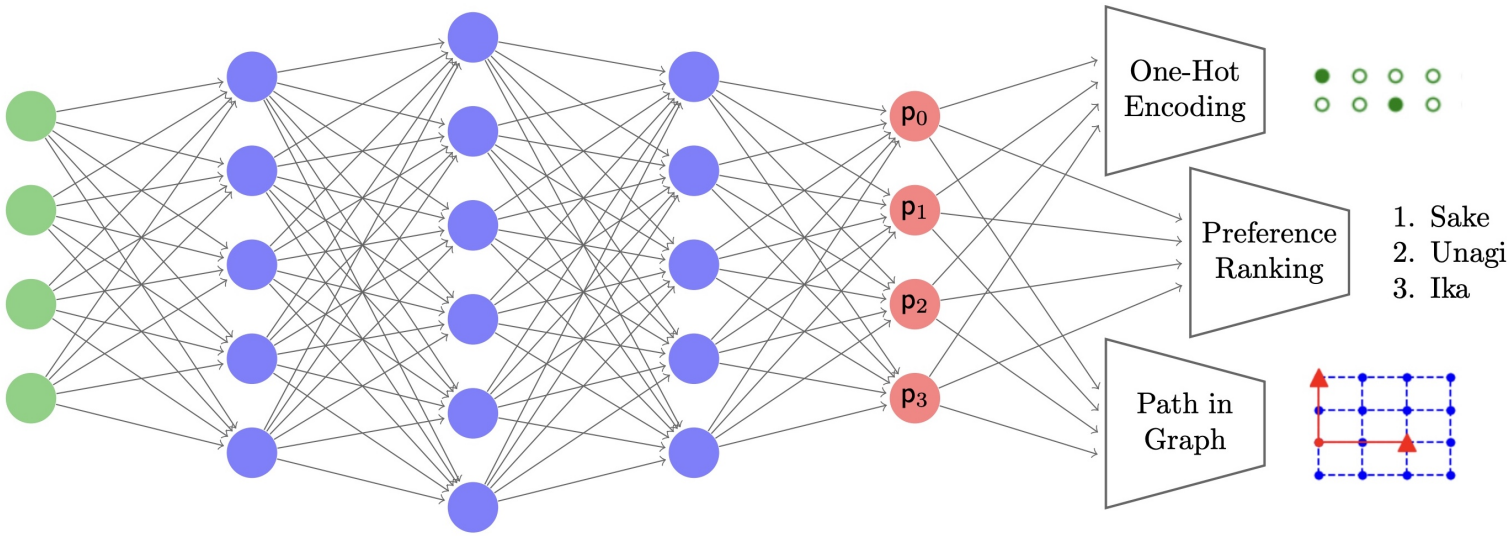
Other uses of semantic loss



from Xu et al. (2018)

- Can be used for *structured prediction*, jointly predicting many output variables for building structured objects (Ahmed et al., 2022).

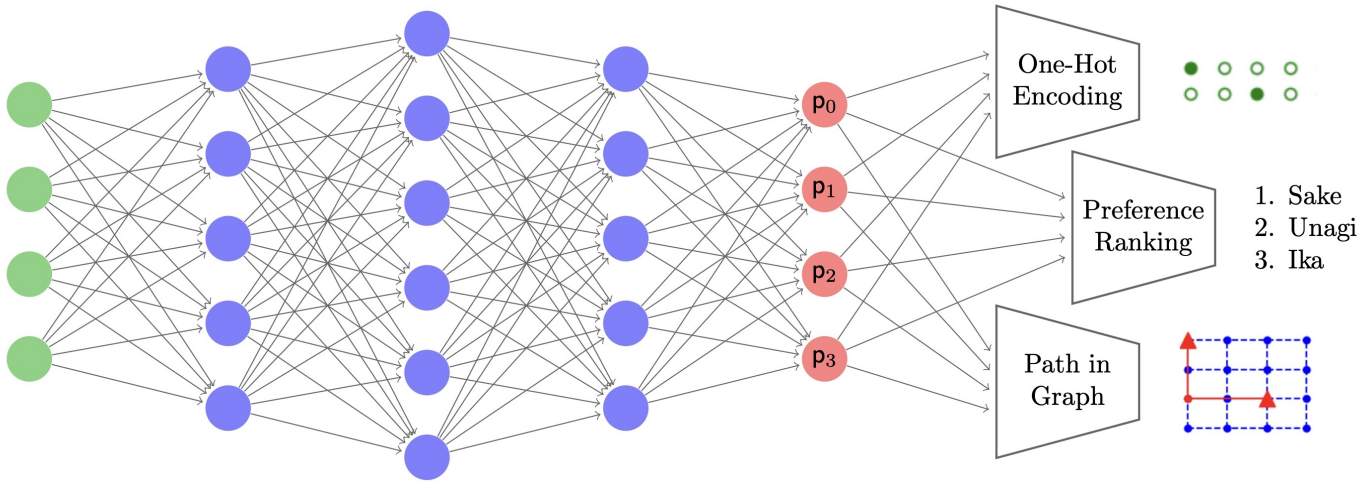
Other uses of semantic loss



from Xu et al. (2018)

- Can be used for *structured prediction*, jointly predicting many output variables for building structured objects (Ahmed et al., 2022).

Other uses of semantic loss



- **Preference learning**: Ranking preferences, e.g., (Choi et al., 2015) given variables P_1, P_2, P_3, binary matrix $P_{i,j}$ $(i, j \in \{1, ..., 3\})$ and a total order:

$$\forall i.((P_{i,1} \wedge \neg P_{i,2}, \wedge \neg P_{i,3}) \vee$$
$$(\neg P_{i,1} \wedge P_{i,2}, \wedge \neg P_{i,3}) \vee$$
$$(\neg P_{i,1} \wedge \neg P_{i,2}, \wedge P_{i,3}))$$

$$\forall j.((P_{1,j} \wedge \neg P_{2,j} \wedge \neg P_{3,j}) \vee$$
$$(\neg P_{1,j} \wedge P_{2,j} \wedge \neg P_{3,j}) \vee$$
$$(\neg P_{1,j} \wedge \neg P_{2,j} \wedge P_{3,j}))$$

How the semantic loss is used

```
1 loss_weight = 0.3                                    ## lambda
2 wmc_loss = -torch.log(wmc.sum(dim=-1)).mean()        ## semantic loss
3 loss = label_loss + loss_weight * wmc_loss           ## L_CE + lambda L_sl
```

- Usually coupled with ordinary loss, or as a *regularizer*

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}(F)$$

How the semantic loss is used

```
loss_weight = 0.3                                    ## lambda
wmc_loss = -torch.log(wmc.sum(dim=-1)).mean()        ## semantic loss
loss = label_loss + loss_weight * wmc_loss           ## L_CE + lambda L_sl
```

- Usually coupled with ordinary loss, or as a *regularizer*

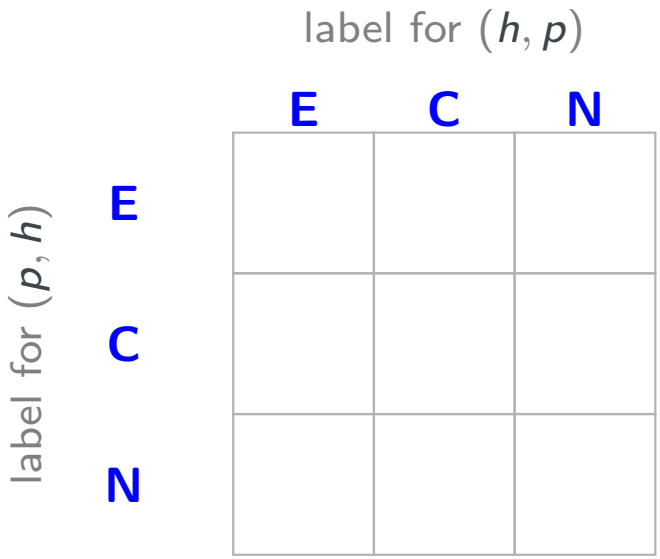
$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}(F)$$

From Marra et al. (2021):

> [This] class of NeSy is model-based. They use logic to define a loss function (usually a regularization term) for neural networks. The networks compute scores for a set of atoms... At each training step the logic-based loss function checks whether the assigned scores satisfy the logical theory and computes a corresponding penalty. **Logical inference is therefore turned into a learning problem** (i.e. **"learning to satisfy"**)

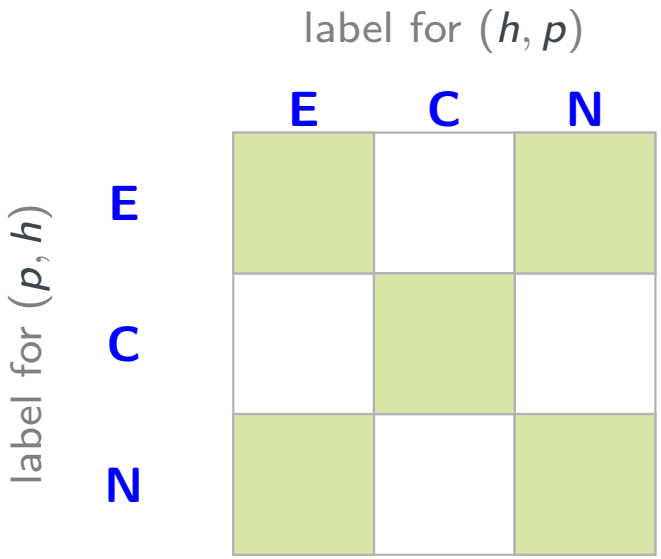
When "learning to satisfy" can go wrong

Rule: $\forall(p, h).\ \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$ (one label per pair). **Gold**: both are **C**.



When "learning to satisfy" can go wrong

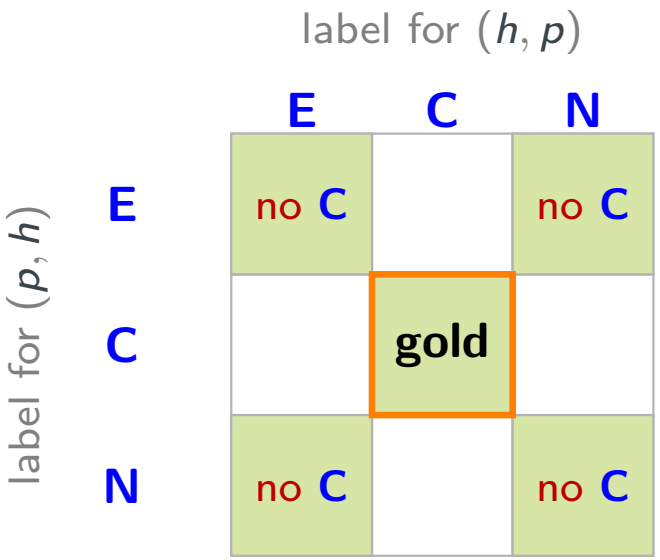
Rule: $\forall(p, h)$. **C**$(p, h) \leftrightarrow$ **C**(h, p) (one label per pair). **Gold**: both are **C**.



satisfies the rule: 5 of 9

When "learning to satisfy" can go wrong

Rule: $\forall(p, h).\ \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$ (one label per pair). **Gold**: both are **C**.

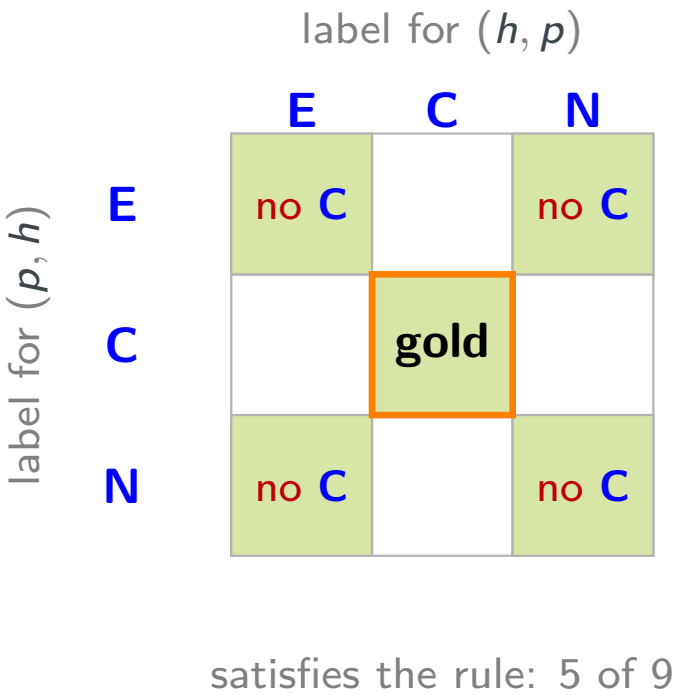


satisfies the rule: 5 of 9

4 of 5 satisfying worlds never predict **C**.

When "learning to satisfy" can go wrong

Rule: $\forall(p, h).\ \mathbf{C}(p, h) \leftrightarrow \mathbf{C}(h, p)$ (one label per pair). **Gold**: both are **C**.



satisfies the rule: 5 of 9

4 of 5 satisfying worlds never predict **C**.

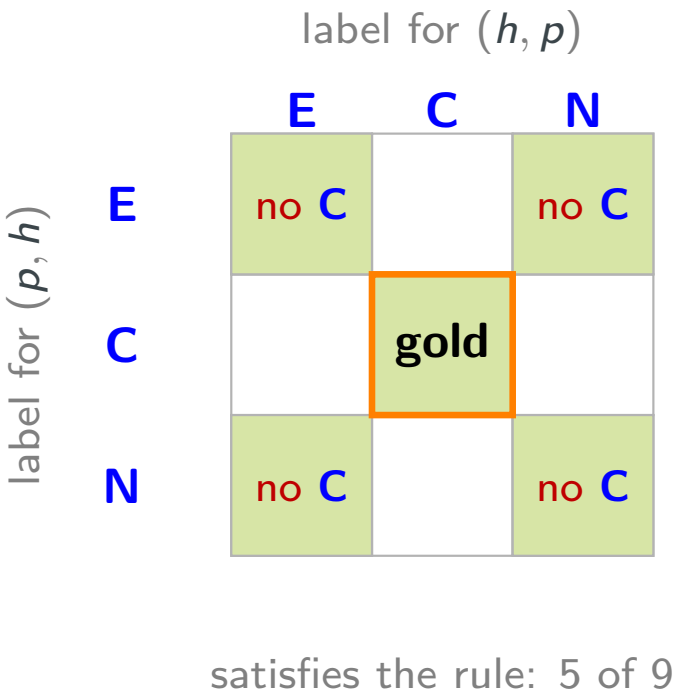
Two models, scored by the rule:

	predicts	$P_\theta(F)$
A	**C**, **C** (correct)	0.660
B	**E**, **N** (never **C**)	0.668

The rule alone cannot tell them apart.

When "learning to satisfy" can go wrong

Rule: $\forall(p, h)$. **C**$(p, h) \leftrightarrow$ **C**(h, p) (one label per pair). **Gold**: both are **C**.



satisfies the rule: 5 of 9

4 of 5 satisfying worlds never predict **C**.

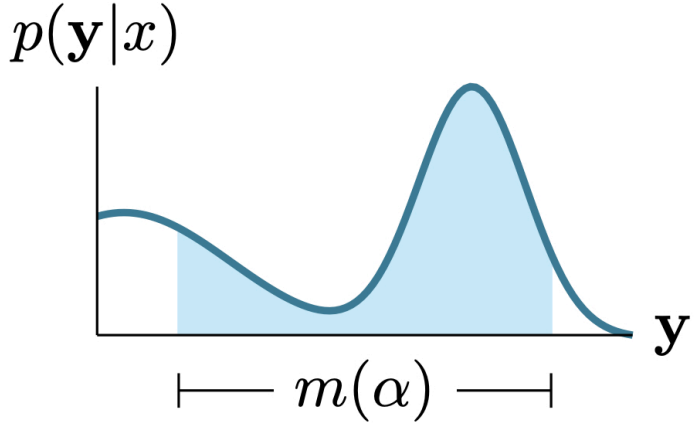
Two models, scored by the rule:

	predicts	$P_\theta(F)$
A	**C**, **C** (correct)	0.660
B	**E**, **N** (never **C**)	0.668

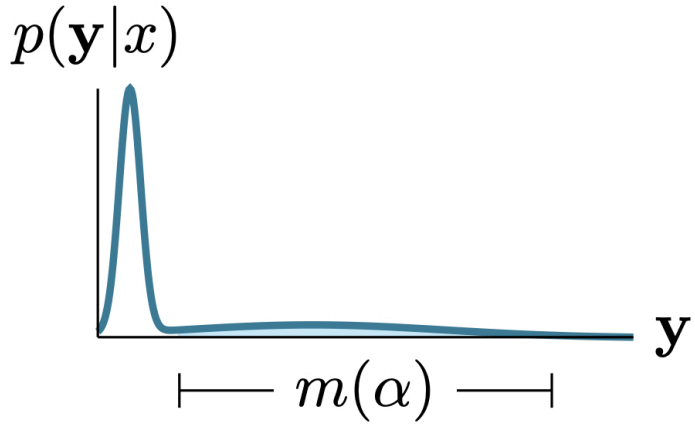
The rule alone cannot tell them apart.

So: a **regularizer**, $\mathcal{L}_{\text{CE}} + \lambda\mathcal{L}_{\text{sl}}$; the labels pick out the right world.

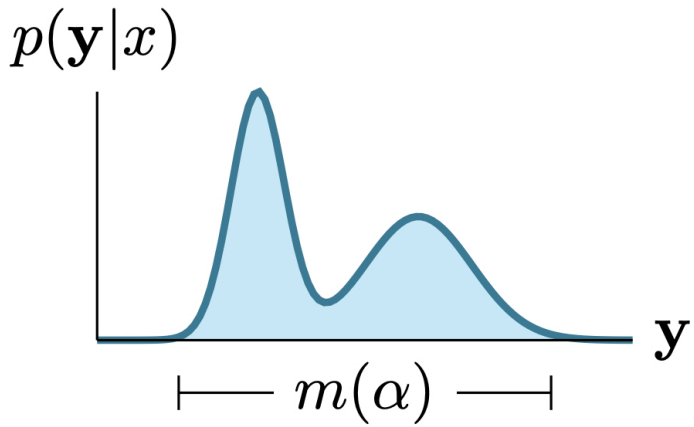
Another view

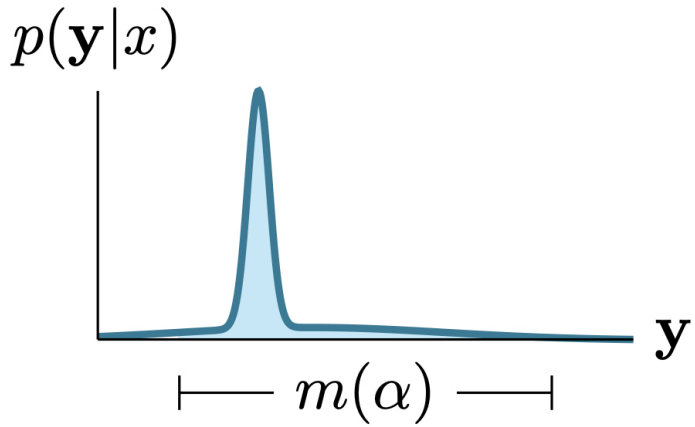


(a) Network uncertain over both valid and invalid predictions



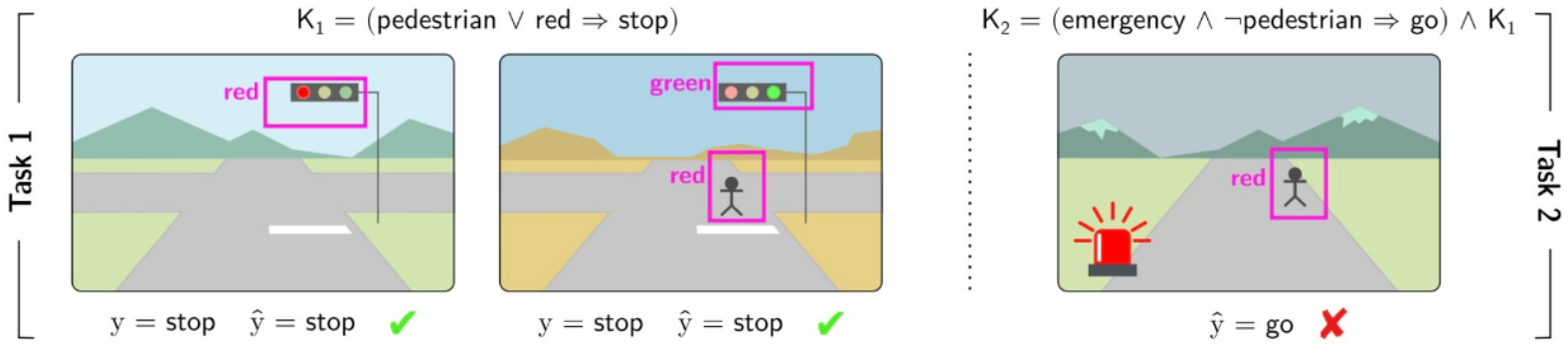
(b) Network allocating most mass to one invalid prediction





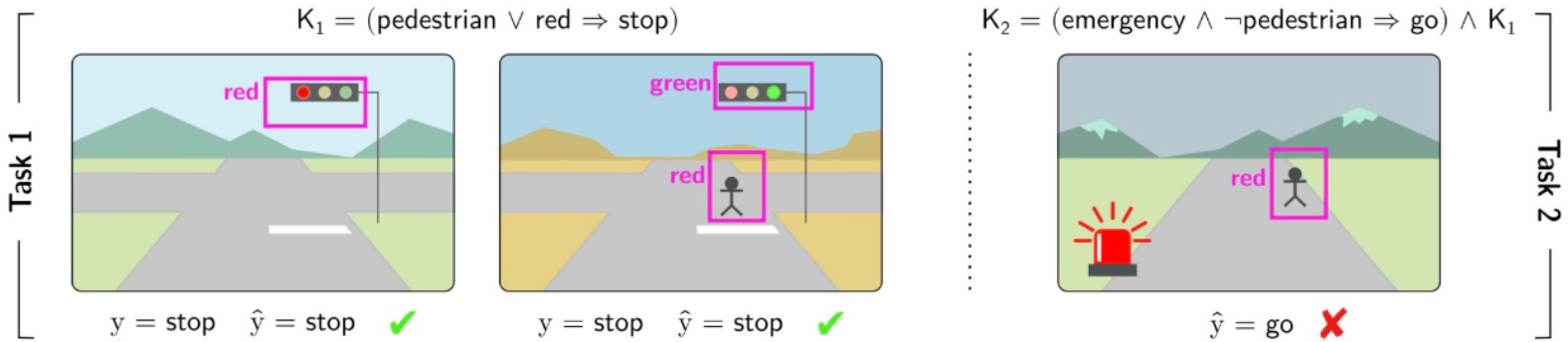
From Ahmed et al. (2022)

Another example: reasoning shortcuts



from Marconato et al. (2024)

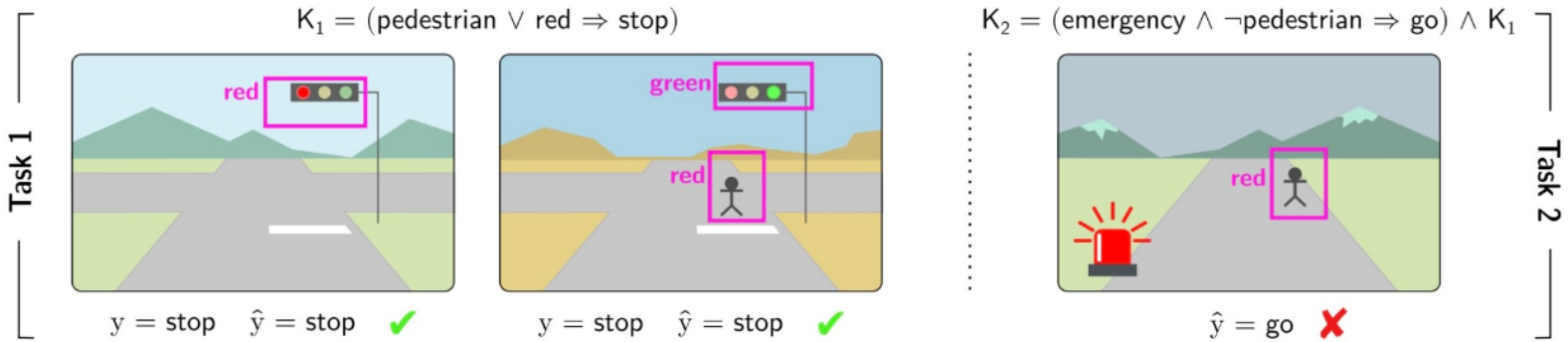
Another example: reasoning shortcuts



from Marconato et al. (2024)

- **Rule**: (*pedestrian* $\vee$ *red*) $\rightarrow$ *stop*; training only checks the label *stop*.

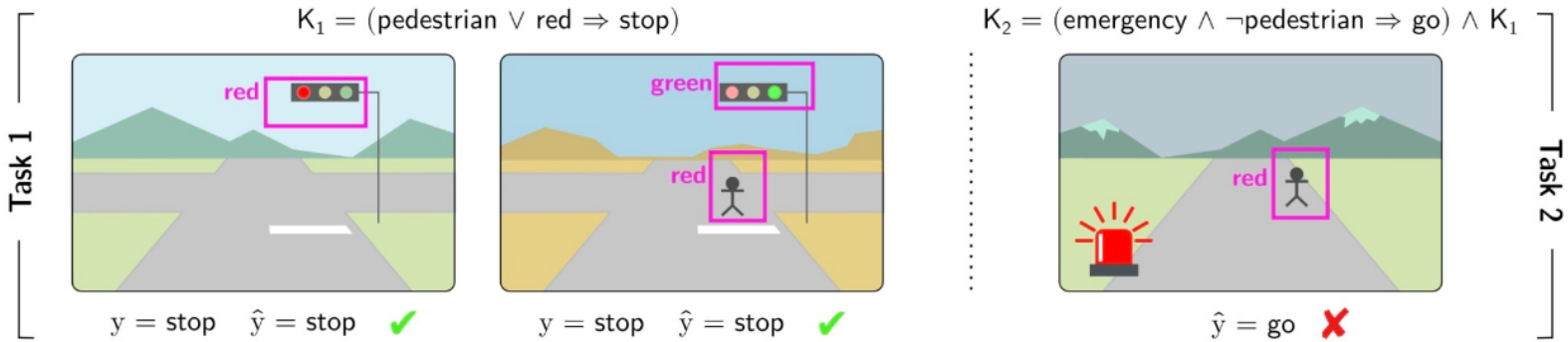
Another example: reasoning shortcuts



from Marconato et al. (2024)

- **Rule**: (*pedestrian* $\vee$ *red*) $\rightarrow$ *stop*; training only checks the label *stop*.
- **Shortcut**: see a pedestrian as "*red*": **right label, wrong concept**.

Another example: reasoning shortcuts



from Marconato et al. (2024)

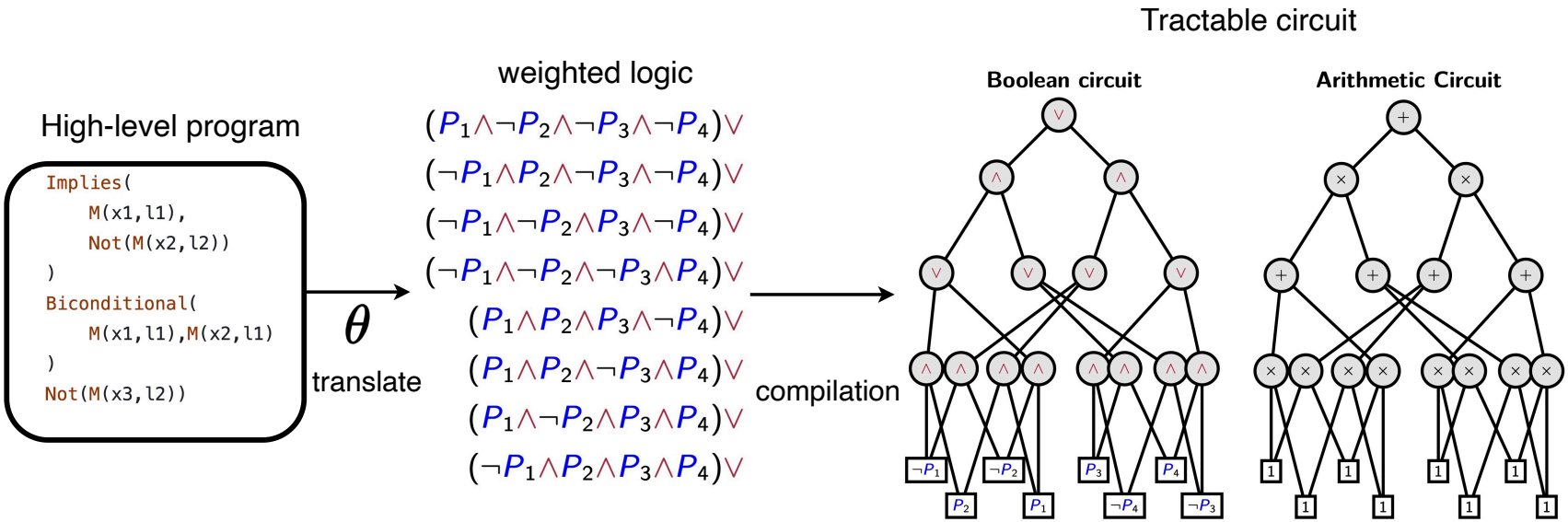
- **Rule**: (*pedestrian* $\vee$ *red*) $\rightarrow$ *stop*; training only checks the label *stop*.
- **Shortcut**: see a pedestrian as "*red*": **right label, wrong concept**.
- **Reuse**: add (*emergency* $\wedge$ $\neg$*pedestrian*) $\rightarrow$ *go*: *go* ✗.

The practical bottleneck

computing $\mathrm{WMC}(F, \theta)$

a sum over all 2^n worlds (lecture 1): #P-hard in general

How this works in practice



- **Knowledge compilation**: tractable representations for efficient logical and probabilistic inference (Darwiche and Marquis, 2002; Marquis, 2008).

Compilation in `symprog`: the circuit underneath

Every query first **compiles** the formula into a circuit (an SDD).

In [1]:
```
@Parser(language="constraint")
def program():
    a: 0.4; b: 0.7
    a or b

program.inference().inference
```

Out[1]:
```
tensor(0.8200)
```

Compilation in `symprog`: the circuit underneath

Every query first **compiles** the formula into a circuit (an SDD).

In [1]:
```
@Parser(language="constraint")
def program():
    a: 0.4; b: 0.7
    a or b

program.inference().inference
```

Out[1]: `tensor(0.8200)`

In [2]:
```
program.inference(mode="Formula").inference
```

Out[2]: `b | (a & ~b)` the compiled circuit

Compilation in `symprog`: the circuit underneath

Every query first **compiles** the formula into a circuit (an SDD).

In [1]:
```
@Parser(language="constraint")
def program():
    a: 0.4; b: 0.7
    a or b

program.inference().inference
```

Out[1]: `tensor(0.8200)`

In [2]:
```
program.inference(mode="Formula").inference
```

Out[2]: `b | (a & ~b)` the compiled circuit

In [3]:
```
program.inference(symbolic=True).inference
```

Out[3]: $a\,(1-b)+b$ as arithmetic

Compilation in `symprog`: the circuit underneath

Every query first **compiles** the formula into a circuit (an SDD).

In [1]:
```
@Parser(language="constraint")
def program():
    a: 0.4; b: 0.7
    a or b

program.inference().inference
```

Out[1]: `tensor(0.8200)`

In [2]:
```
program.inference(mode="Formula").inference
```

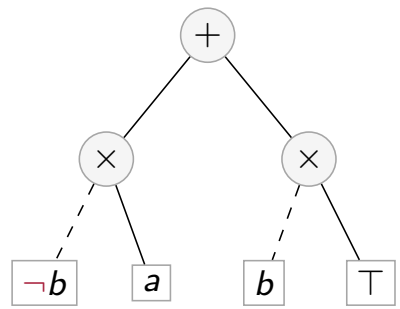
Out[2]: `b | (a & ~b)` the compiled circuit

In [3]:
```
program.inference(symbolic=True).inference
```

Out[3]: $a\,(1-b)+b$ as arithmetic

In [4]:
```
import graphviz
graphviz.Source(
    program.solver.formula.dot)
```

Out[4]:



Compilation in `symprog`: the circuit underneath

Every query first **compiles** the formula into a circuit (an SDD).

In [1]:
```
@Parser(language="constraint")
def program():
    a: 0.4; b: 0.7
    a or b

program.inference().inference
```

Out[1]: `tensor(0.8200)`

In [2]:
```
program.inference(mode="Formula").inference
```

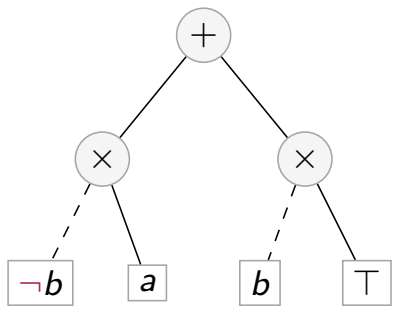
Out[2]: `b | (a & ~b)` the compiled circuit

In [3]:
```
program.inference(symbolic=True).inference
```

Out[3]: $a\,(1-b)+b$ as arithmetic

In [4]:
```
import graphviz
graphviz.Source(
    program.solver.formula.dot)
```

Out[4]:



The circuit is **deterministic**: its two branches cannot both be true, so WMC = one bottom-up pass, $0.4 \cdot 0.3 + 0.7 = 0.82$.

Creating tractable representations

- **Uniqueness constraint** would be commonly written in *conjunctive normal form* (CNF):

$$\underbrace{(\mathbf{E} \vee \mathbf{N} \vee \mathbf{C})}_{\text{clause}} \wedge (\neg \mathbf{E} \vee \neg \mathbf{N}) \wedge (\neg \mathbf{N} \vee \neg \mathbf{C}) \wedge (\neg \mathbf{E} \vee \neg \mathbf{N})$$

Creating tractable representations

- **Uniqueness constraint** would be commonly written in *conjunctive normal form* (CNF):

$$\underbrace{(\mathbf{E} \vee \mathbf{N} \vee \mathbf{C})}_{\text{clause}} \wedge (\neg \mathbf{E} \vee \neg \mathbf{N}) \wedge (\neg \mathbf{N} \vee \neg \mathbf{C}) \wedge (\neg \mathbf{E} \vee \neg \mathbf{N})$$

We could also write it in the following form:

$$\big(\mathbf{E} \wedge \neg \mathbf{N} \wedge \neg \mathbf{C}\big) \vee \big(\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N}\big)$$

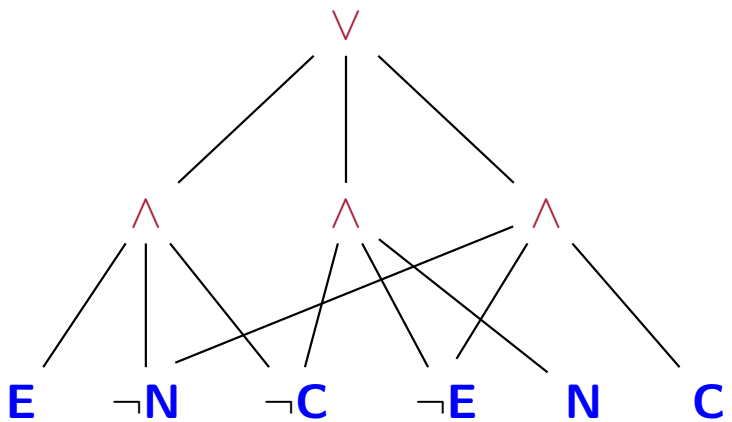
Creating tractable representations

- **Uniqueness constraint** would be commonly written in *conjunctive normal form* (CNF):

$$\underbrace{(\mathbf{E} \vee \mathbf{N} \vee \mathbf{C})}_{\text{clause}} \wedge (\neg \mathbf{E} \vee \neg \mathbf{N}) \wedge (\neg \mathbf{N} \vee \neg \mathbf{C}) \wedge (\neg \mathbf{E} \vee \neg \mathbf{N})$$

We could also write it in the following form:

$$\big(\mathbf{E} \wedge \neg \mathbf{N} \wedge \neg \mathbf{C}\big) \vee \big(\neg \mathbf{E} \wedge \mathbf{C} \wedge \neg \mathbf{N}\big) \vee \big(\neg \mathbf{E} \wedge \neg \mathbf{C} \wedge \mathbf{N}\big)$$



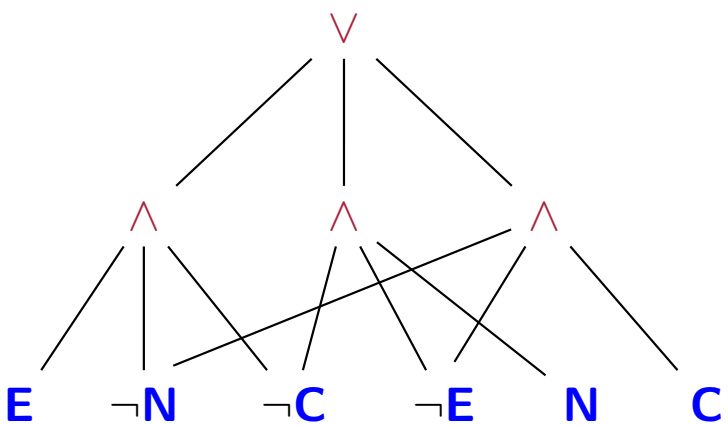
Creating tractable representations

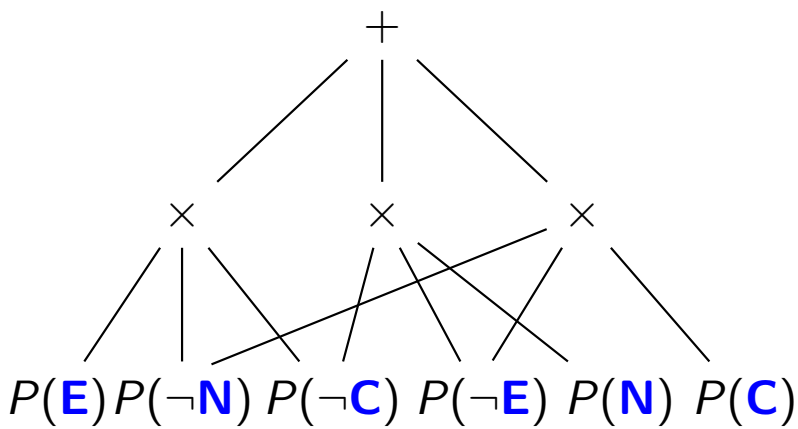
- **Uniqueness constraint** would be commonly written in *conjunctive normal form* (CNF):

$$\underbrace{(\mathbf{E}\vee\mathbf{N}\vee\mathbf{C})}_{\text{clause}}\wedge(\neg\mathbf{E}\vee\neg\mathbf{N})\wedge(\neg\mathbf{N}\vee\neg\mathbf{C})\wedge(\neg\mathbf{E}\vee\neg\mathbf{N})$$

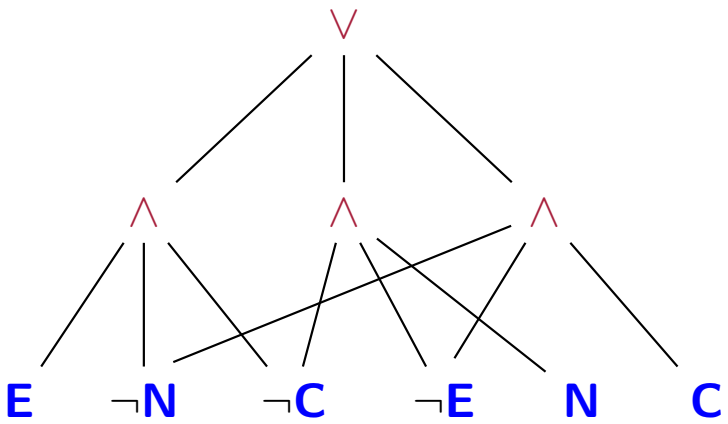
We could also write it in the following form:

$$\big(\mathbf{E}\wedge\neg\mathbf{N}\wedge\neg\mathbf{C}\big)\vee\big(\neg\mathbf{E}\wedge\mathbf{C}\wedge\neg\mathbf{N}\big)\vee\big(\neg\mathbf{E}\wedge\neg\mathbf{C}\wedge\mathbf{N}\big)$$

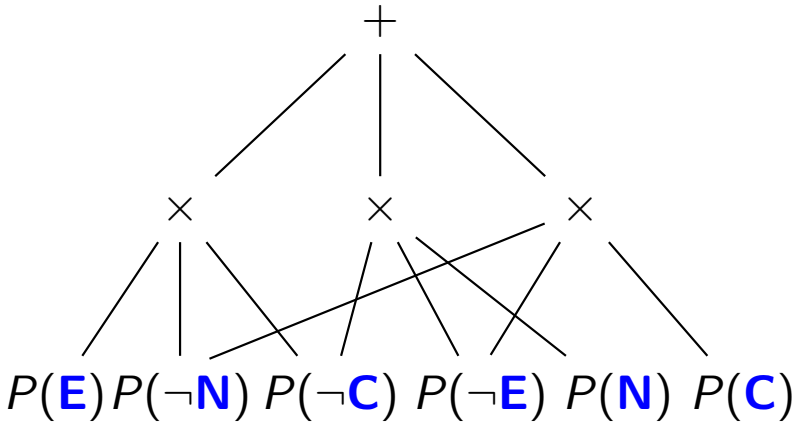




Creating tractable representations

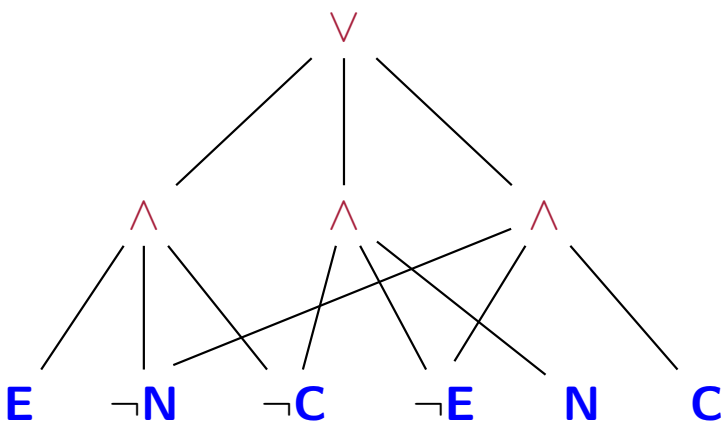


Boolean circuit

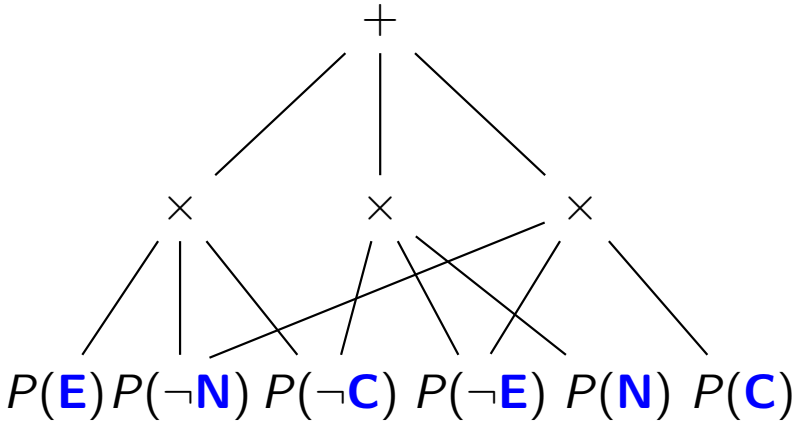


Arithmetic circuit

Creating tractable representations



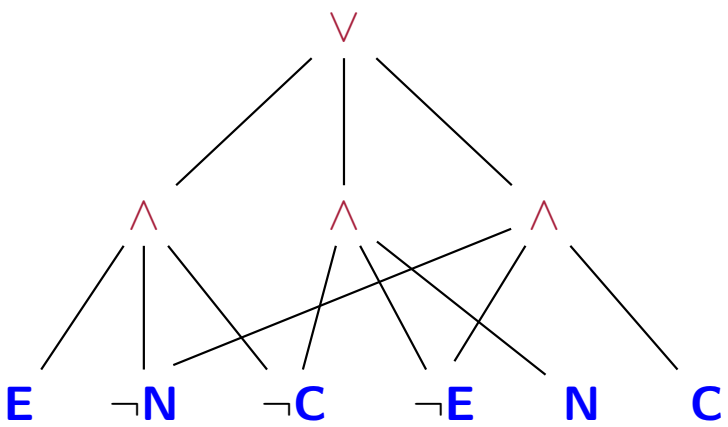
Boolean circuit



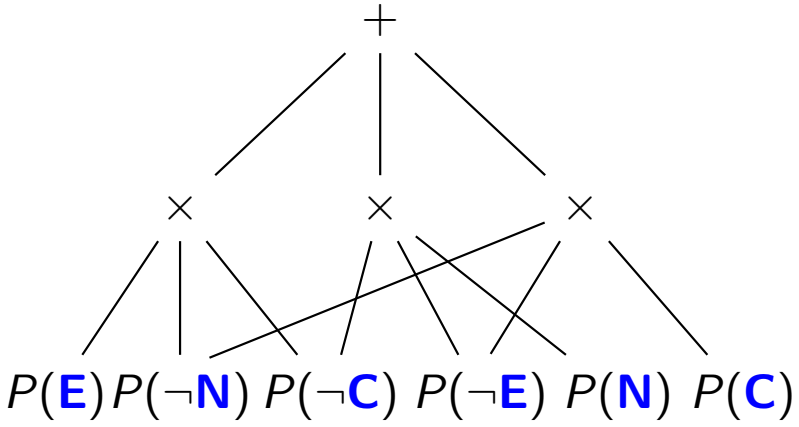
Arithmetic circuit

- Facilitate solving these algorithmic problems (SAT, WMC) efficiently.
  Two key properties:

Creating tractable representations



Boolean circuit



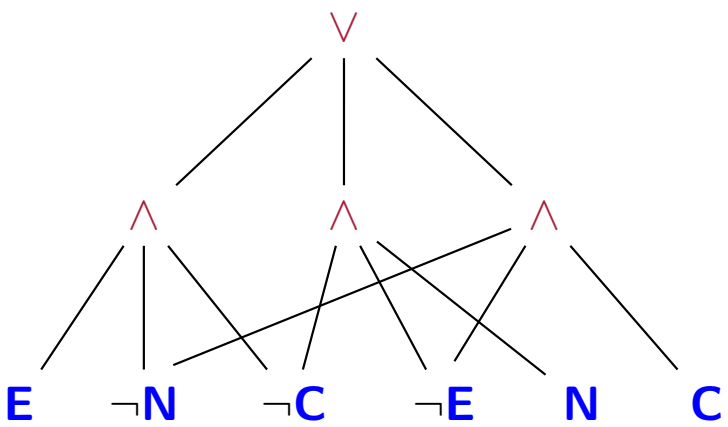
Arithmetic circuit

- Facilitate solving these algorithmic problems (SAT, WMC) efficiently.

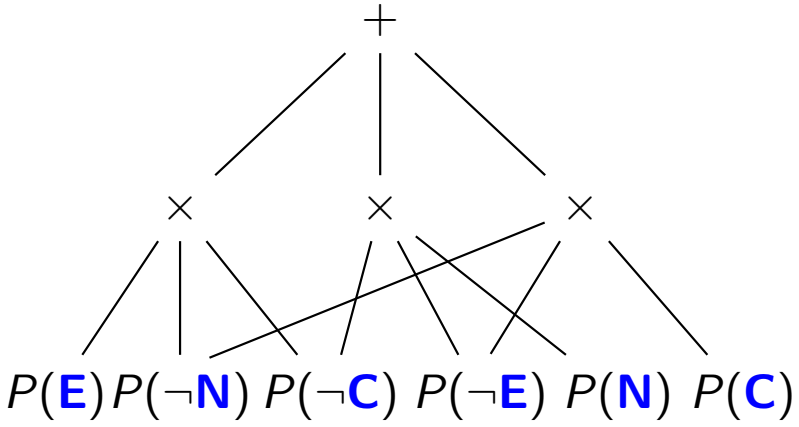
  Two key properties:

  decomposability: For conjunction nodes ∧, children never share the same variables.

Creating tractable representations



Boolean circuit



Arithmetic circuit

- Facilitate solving these algorithmic problems (SAT, WMC) efficiently.

  Two key properties:

  decomposability: For conjunction nodes $\wedge$, children never share the same variables.

  determinism: For each disjunction node $\vee$, the children are logically inconsistent.

5 Tractable Representations

```
1 from pysat.examples.genhard import PHP    # pip install python-sat
2 from pysat.solvers import Solver
3 cnf = PHP(nof_holes=10) # Try to place 11 pigeons into 10 holes.
4 with Solver(name="g3", bootstrap_with=cnf.clauses,use_timer=True) as solver:
5     print("SAT" if solver.solve() else "UNSAT")
6     print(f"Time: {solver.time():.3f}s")
7     print(solver.accum_stats())
```

Figure 4: Computing a deceptively easy but notoriously hard problem for **SAT** solvers (Haken, 1985) (Pigeonhole; **line 3**) in `pysat`, which provides access to a wide variety of top-performing **SAT** solvers in Python.

As detailed in the last section and Aside 3, computing **SAT** and its many variants is computationally very challenging. Nonetheless, modern **SAT** solvers, which are used in a wide variety of industrial applications, have been a tremendous engineering success, as they are now capable of scaling to millions of variables (see example in Figure 4 for how to use such solvers via `pysat`). Rather than focus on these engineering details – not all of which are extensible to scaling up the other counting problems we discussed in the last section – we focus on an approach to implementing logical and probabilistic inference procedures using a set of techniques called **knowledge compilation** (Darwiche & Marquis, 2002).

Fundamentally, knowledge compilation is again about normal forms: the idea is to transform our logical formulas into a representation, known as a **tractable representation**, where our target problems can be computed more easily (ideally, in linear time over the new representation). Broadly, knowledge compilation of the kind we will talk about will ask questions of the form: *Given my algorithmic problem involving Boolean formulas, what form must my formulas take such that I can tractably compute my target problem exactly?*

Example 13. *Taking once again our running example*

$$\mathsf{F} = (\neg \mathsf{P}_1 \vee \mathsf{P}_2) \wedge \mathsf{P}_3,$$

More in the course notes, Section 5

we will come back to this

Avoiding the bottleneck

An alternative formalism: fuzzy logic

- Allows for truth values to be **real-valued**, implements Boolean operators as real-valued functions (t-norms). For example:

An alternative formalism: fuzzy logic

- Allows for truth values to be **real-valued**, implements Boolean operators as real-valued functions (t-norms). For example:

Boolean logic	$\mathcal{R}$**-Product**
$a \wedge b$	$\mathbf{a} \cdot \mathbf{b}$
$\neg a$	$1 - \mathbf{a}$
$a \vee b$	$\mathbf{a} + \mathbf{b} - \mathbf{a} \cdot \mathbf{b}$
$a \rightarrow b$	$\min(1, \frac{\mathbf{b}}{\mathbf{a}})$

An alternative formalism: fuzzy logic

- Allows for truth values to be **real-valued**, implements Boolean operators as real-valued functions (t-norms). For example:

Boolean logic	$\mathcal{R}$**-Product**
$a \wedge b$	$\mathbf{a} \cdot \mathbf{b}$
$\neg a$	$1 - \mathbf{a}$
$a \vee b$	$\mathbf{a} + \mathbf{b} - \mathbf{a} \cdot \mathbf{b}$
$a \to b$	$\min(1, \frac{\mathbf{b}}{\mathbf{a}})$

For example:

$$F := \mathbf{E}(p_1, h_1) \wedge \mathbf{C}_2(p_2, h_2).$$

An alternative formalism: fuzzy logic

- Allows for truth values to be **real-valued**, implements Boolean operators as real-valued functions (t-norms). For example:

Boolean logic	$\mathcal{R}$**-Product**
$a \wedge b$	$\mathbf{a} \cdot \mathbf{b}$
$\neg a$	$1 - \mathbf{a}$
$a \vee b$	$\mathbf{a} + \mathbf{b} - \mathbf{a} \cdot \mathbf{b}$
$a \to b$	$\min(1, \frac{\mathbf{b}}{\mathbf{a}})$

For example:

$$F := \mathbf{E}(p_1, h_1) \wedge \mathbf{C}_2(p_2, h_2).$$

Can define the loss to be

$$-\log\left(P_\theta(\mathbf{E}(p_1, h_1)) \cdot P_\theta(\mathbf{C}_2(p_2, h_2)\right)$$

.

An alternative formalism: fuzzy logic

Name	Boolean Logic	Product	Gödel	Łukasiewicz
Negation	$\neg A$	$1-a$	$1-a$	$1-a$
T-norm	$A \wedge B$	ab	$\min(a,b)$	$\max(0, a+b-1)$
T-conorm	$A \vee B$	$a+b-ab$	$\max(a,b)$	$\min(1, a+b)$
Residuum	$A \rightarrow B$	$\min\left(1, \frac{b}{a}\right)$	$\begin{cases} 1, & \text{if b} \geq \text{a}, \\ b, & \text{else} \end{cases}$	$\min(1, 1-a+b)$

Table 1: Mapping discrete statements to differentiable functions using t-norms. Literals are upper-cased (e.g. A) while real-valued probabilities are lower-cased (e.g. a). Here, differentiable forms are from a mixture of R-fuzzy logic and S-fuzzy logic. In this paper, we focus on the product t-norm.

An alternative formalism: fuzzy logic

Name	Boolean Logic	Product	Gödel	Łukasiewicz
Negation	$\neg A$	$1 - a$	$1 - a$	$1 - a$
T-norm	$A \wedge B$	ab	$\min(a, b)$	$\max(0, a + b - 1)$
T-conorm	$A \vee B$	$a + b - ab$	$\max(a, b)$	$\min(1, a + b)$
Residuum	$A \to B$	$\min\left(1, \frac{b}{a}\right)$	$\begin{cases} 1, & \text{if b} \geq \text{a}, \\ b, & \text{else} \end{cases}$	$\min(1, 1 - a + b)$

Table 1: Mapping discrete statements to differentiable functions using t-norms. Literals are upper-cased (e.g. A) while real-valued probabilities are lower-cased (e.g. a). Here, differentiable forms are from a mixture of R-fuzzy logic and S-fuzzy logic. In this paper, we focus on the product t-norm.

- Lots of work on the optimization properties of different fuzzy systems of logic (Grespan et al., 2021; Van Krieken et al., 2022).

An alternative formalism: fuzzy logic

Name	Boolean Logic	Product	Gödel	Łukasiewicz
Negation	$\neg A$	$1-a$	$1-a$	$1-a$
T-norm	$A \wedge B$	ab	$\min(a,b)$	$\max(0, a+b-1)$
T-conorm	$A \vee B$	$a+b-ab$	$\max(a,b)$	$\min(1, a+b)$
Residuum	$A \rightarrow B$	$\min\left(1, \frac{b}{a}\right)$	$\begin{cases} 1, & \text{if b} \geq \text{a}, \\ b, & \text{else} \end{cases}$	$\min(1, 1-a+b)$

Table 1: Mapping discrete statements to differentiable functions using t-norms. Literals are upper-cased (e.g. A) while real-valued probabilities are lower-cased (e.g. a). Here, differentiable forms are from a mixture of R-fuzzy logic and S-fuzzy logic. In this paper, we focus on the product t-norm.

- Lots of work on the optimization properties of different fuzzy systems of logic (Grespan et al., 2021; Van Krieken et al., 2022).

  trouble: Loses virtually all the nice semantic properties we discussed.

Aside 5: Multi-valued logic and fuzzy logic. Classical logic is *bivalent*: every proposition is either true or false. Non-classical logics that have more than two truth values were already studied in the early 20th century by Łukasiewicz (Łukasiewicz, 1920) and Gödel (Gödel, 1932), among others (Kleene, 1952). In more modern times, Zadeh (1975) introduced *fuzzy logic*, which initially aimed to help formalize vague notions and imprecise information. Accordingly, every atom takes a truth value in $[0, 1]$ and the Boolean operators take the form of real-valued functions (or triangular norms (Klement et al., 2013)) as shown below for different systems of fuzzy logic (Li et al., 2019):

	Boolean	Gödel	Product	Łukasiewicz
negation	$\neg P_1$	$1 - x$	$1 - x$	$1 - x$
conjunction	$P_1 \wedge P_2$	$\min(x, y)$	xy	$\max(0, x + y - 1)$
disjunction	$P_1 \vee P_2$	$\max(x, y)$	$x + y - xy$	$\min(1, x + y)$
implication	$P_1 \rightarrow P_2$	1 if $x \leq y$, else y	$\min(1, y/x)$	$\min(1, 1 - x + y)$

Note that all three logics agree with classical logic when truth values are 0 or 1, but differ in between: for $x = y = 0.5$, for instance, $P_1 \wedge P_2$ receives the values 0.5, 0.25 and 0.

Because these functions are cheap to evaluate and (almost everywhere) differentiable, fuzzy logics are popular in machine learning as a way of turning logical constraints into loss functions (Grespan et al., 2021; Badreddine et al., 2022; Van Krieken et al., 2022) (for a critical discussion see Marra et al. (2021)).

More in the course notes, Aside 5

cheap, differentiable connectives: no counting, no WMC bottleneck

Conclusion: what you should know

Declarative modeling

- ✓ say **what**, not how
- ✓ predictions as propositions
- ✓ rules as constraints (NLI)

Conclusion: what you should know

Declarative modeling

- ✓ say **what**, not how
- ✓ predictions as propositions
- ✓ rules as constraints (NLI)

Semantic loss

- ✓ $P_\theta(F) = \mathrm{WMC}(F, \theta)$
- ✓ $\mathcal{L}_{\mathsf{sl}} = -\log P_\theta(F)$
- ✓ special case: cross-entropy

Conclusion: what you should know

Declarative modeling

- ✓ say **what**, not how
- ✓ predictions as propositions
- ✓ rules as constraints (NLI)

In practice

- ✓ a regularizer: $\mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}$
- ✓ beware of shortcuts
- ✓ fully factorized model

Semantic loss

- ✓ $P_\theta(F) = \text{WMC}(F, \theta)$
- ✓ $\mathcal{L}_{\text{sl}} = -\log P_\theta(F)$
- ✓ special case: cross-entropy

Conclusion: what you should know

Declarative modeling

- ✓ say **what**, not how
- ✓ predictions as propositions
- ✓ rules as constraints (NLI)

In practice

- ✓ a regularizer: $\mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}$
- ✓ beware of shortcuts
- ✓ fully factorized model

Semantic loss

- ✓ $P_\theta(F) = \text{WMC}(F, \theta)$
- ✓ $\mathcal{L}_{\text{sl}} = -\log P_\theta(F)$
- ✓ special case: cross-entropy

Tractability

- ✓ WMC is #P-hard in general
- ✓ compile into circuits
- ✓ decomposable, deterministic

Conclusion: what you should know

Declarative modeling

- ✓ say **what**, not how
- ✓ predictions as propositions
- ✓ rules as constraints (NLI)

In practice

- ✓ a regularizer: $\mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{sl}}$
- ✓ beware of shortcuts
- ✓ fully factorized model

Semantic loss

- ✓ $P_\theta(F) = \text{WMC}(F, \theta)$
- ✓ $\mathcal{L}_{\text{sl}} = -\log P_\theta(F)$
- ✓ special case: cross-entropy

Tractability

- ✓ WMC is #P-hard in general
- ✓ compile into circuits
- ✓ decomposable, deterministic

Next time: decompilation, from known losses to their semantics.

References I

Ahmed, K., Wang, E., Chang, K.-W., and Van den Broeck, G. (2022). Neuro-symbolic entropy regularization. In *Uncertainty in Artificial Intelligence*, pages 43–53. PMLR.

Choi, A., Van den Broeck, G., and Darwiche, A. (2015). Tractable learning for structured probability spaces: A case study in learning preference distributions. In *Proceedings of 24th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2861–2868.

Darwiche, A. and Marquis, P. (2002). A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Giannini, F., Diligenti, M., Maggini, M., Gori, M., and Marra, G. (2023). T-norms driven loss functions for machine learning. *Applied Intelligence*, 53(15):18775–18789.

Grandvalet, Y. and Bengio, Y. (2004). Semi-supervised learning by entropy minimization. *Advances in neural information processing systems*, 17.

Grespan, M. M., Gupta, A., and Srikumar, V. (2021). Evaluating relaxations of logic for neural networks: A comprehensive study. *arXiv preprint arXiv:2107.13646*.

Li, T., Gupta, V., Mehta, M., and Srikumar, V. (2019). A logic-driven framework for consistency of neural models. *arXiv preprint arXiv:1909.00126*.

References II

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Marconato, E., Teso, S., Vergari, A., and Passerini, A. (2024). Not all neuro-symbolic concepts are created equal: Analysis and mitigation of reasoning shortcuts. *Advances in Neural Information Processing Systems*, 36.

Marquis, P. (2008). Knowledge compilation: a sightseeing tour. *Tutorial notes, ECAI*, 8.

Marra, G., Dumančić, S., Manhaeve, R., and De Raedt, L. (2021). From statistical relational to neural symbolic artificial intelligence: a survey. *arXiv preprint arXiv:2108.11451*.

Minervini, P. and Riedel, S. (2018). Adversarially regularising neural nli models to integrate logical background knowledge. *arXiv preprint arXiv:1808.08609*.

Van Krieken, E., Acar, E., and Van Harmelen, F. (2022). Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602.

van Krieken, E., Minervini, P., Ponti, E. M., and Vergari, A. (2024). On the independence assumption in neurosymbolic learning. *arXiv preprint arXiv:2404.08458*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008.

References III

Weiss, G., Goldberg, Y., and Yahav, E. (2021). Thinking like transformers. In *International Conference on Machine Learning*, pages 11080–11090. PMLR.

Welleck, S., Kulikov, I., Roller, S., Dinan, E., Cho, K., and Weston, J. (2019). Neural text generation with unlikelihood training. In *International Conference on Learning Representations*.

Xu, J., Zhang, Z., Friedman, T., Liang, Y., and Broeck, G. (2018). A Semantic Loss Function for Deep Learning with Symbolic Knowledge. In *International Conference on Machine Learning*, pages 5498–5507.

Yang, A. and Chiang, D. (2024). Counting like transformers: Compiling temporal counting logic into softmax transformers. *arXiv preprint arXiv:2404.04393*.

Yang, A., Chiang, D., and Angluin, D. (2024). Masked hard-attention transformers recognize exactly the star-free languages. *Advances in Neural Information Processing Systems*, 37:10202–10235.